

Data Science for High School

Technology Service Center, University of Delaware

Table of contents

Preface	10
Why This Book Exists	10
What This Book Covers	10
Before You Dive In	11
How to Use This Book	12
Definitions	12
Instructor Insights	12
Optional Asides	12
Knowledge Checks	13
Try With ChatGPT	13
Your Turn	13
Using AI Responsibly	15
The One Rule: Tutor, Not Ghostwriter	15
Always Verify	15
Ask Your Teacher	16
I Unit I: Foundations	17
1 What Is Data?	18
1.1 Learning Objectives	18
1.2 The Big Question	18
1.3 Structured vs. Unstructured Data	19
1.4 You Are a Data Source	20
1.5 A Different Kind of Data Source	21
1.6 Data Has a Point of View	22
1.7 Your Turn: The 72-Hour Data Log	24
1.8 Roadmap: Where We Go Next	24
2 Data Wrangling with Spreadsheets	25
2.1 Learning Objectives	25
2.2 Why Spreadsheets First?	25
2.3 Sorting and Filtering	26
2.4 Formulas That Do the Counting For You	27

2.5	Pivot Tables: Grouping Made Visual	29
2.6	Your Turn: Summarize a Real Dataset	31
2.7	Roadmap: From Tables to Pictures	31
3	Data Visualization Basics	32
3.1	Learning Objectives	32
3.2	From Tables to Pictures	32
3.3	Dataset vs. Dashboard	33
3.4	Choosing the Right Chart	34
3.5	Meet CODAP: Charts Without Code	35
3.6	Hands-On: Building Your First Chart in CODAP	35
3.7	Comparing Multiple Variables	36
3.8	Build Your Own Dashboard	38
3.9	How Charts Can Mislead	38
3.10	Your Turn: Two Charts, One Dataset	39
3.11	Roadmap: When Even the Dashboard Isn't Enough	40
II	Unit II: Programming for Data Science	41
4	Python & Pandas: Working with Big Data	42
4.1	Learning Objectives	42
4.2	The Challenge: When Spreadsheets Fail	42
4.3	What is Python?	43
4.3.1	The LEGO Metaphor for Logic	43
4.3.2	The “iPhone” of Coding	43
4.4	Why Learn Python?	43
4.5	Python as a Superpower	44
4.5.1	Your First Step into the Console	44
4.6	Roadmap for Part 1	45
Part 2:	Getting Started with PyCharm	46
	Learning Objectives	46
	Python is the Language; PyCharm is the Tool	46
	Setting Up Your Digital Workshop	47
	Installation Steps	47
	Creating Your First Project	47
	Naming and Saving Conventions	48
	Navigating the Interface	48
	Writing and Running Your First Code	49
	Understanding the Syntax	49
	Interpreting the Result	49

Part 3: Meet Your New Best Friend—Pandas	51
Learning Objectives	51
Meet Your New Best Friend: Pandas	51
Installing and Importing Pandas	51
Step 1: Installation via Terminal	52
Step 2: Importing into Your Code	52
Loading Real-World Data	52
The Exploration Toolkit: First Steps	53
1. The Quick Preview: <code>df.head()</code>	54
2. Checking the Scale: <code>df.shape</code>	54
Understanding Your Data: Report Cards and Snapshots	54
3. The Data Report Card: <code>df.info()</code>	54
4. The Statistics Snapshot: <code>df.describe()</code>	54
Case Study: The Delaware Crash Data	55
Your Turn: Explore a Dataset	55
Part 4: Filtering Like a Pro	57
Learning Objectives	57
From Sheets to Code: Why It Matters	57
Step 1: Cleaning the Data First	58
Step 2: Three Professional Filtering Methods	59
Method 1: The <code>.loc</code> Approach	59
Method 2: The <code>.query()</code> Method	60
Method 3: Direct Bracket Notation (Conditionals)	60
Comparison and Summary	61
Your Mission: Filter Like a Pro	62
5 ETL in Action: Cleaning & Preprocessing	63
5.1 Learning Objectives	63
5.2 What is ETL?	63
5.2.1 Visualizing the ETL Flow	64
5.3 How ETL Fits Into the Data Pipeline	64
5.3.1 The Data Journey	64
5.4 Why ETL Matters: Solving the Messy Data Problem	65
5.5 ETL in the Real World	65
5.6 Reflective Code: Programmatic Extraction	66
5.7 Putting It All Together: The ETL Mindset	67
Part 2: The Extraction Frontier	68
Learning Objectives	68
Why ETL is the Foundation of Data Science	68
Visualizing the Pipeline Logic	69

Phase 1: Extract — Where It All Begins	69
Finding Your Playground	69
Abstract Concept: The Extraction Web	70
Anatomy of an Activity Dataset	70
Hands-On: Loading Your Dataset with Pandas	71
Transformation Roadmap	72
Summary: The First Mile	72
Part 3: The Transform Engine Room	73
Transform: The Bridge Between Raw and Ready	73
Learning Objectives	73
The Purpose of Transformation	74
Visualizing the Transformation Pipeline	74
Technique 1: Normalization	74
The Formula	75
Reflective Code: Normalizing Activity Data	75
Technique 2: Sequential Data Transformation	75
From Chaos to Chronology	76
Reflective Code: Sorting the Timeline	76
Technique 3: Geospatial Data Transformation	77
The Tool: Haversine Formula	77
Reflective Code: Feature Engineering (Distance)	77
Summary: Shaping the Story	78
Part 4: Encoding, Merging, and the ETL Endgame	79
Teaching Machines to Speak Data	79
Learning Objectives	79
1. Encoding: The Language Bridge	80
Choosing the Right Encoding Logic	80
2. Label Encoding: Simple and Efficient	81
Reflective Code: Label Encoding Gender	81
3. One-Hot Encoding: Eliminating Bias	81
Reflective Code: Applying get_dummies()	82
4. Merging: From Hourly Noise to Daily Signal	82
The Aggregation Workflow	83
Reflective Code: Extracting the Daily Signal	83
The ETL Endgame: One Pipeline	84
Part 5: The Load Phase	85
Learning Objectives	85
What is the Load Phase?	85
Common Storage Destinations	86
The Data Life Cycle	86

Hands-On: Saving to CSV with Pandas	86
The Saving Workflow	87
Reflective Code: The Final Export	87
The Full Journey: ETL from Start to Finish	88
Pipeline Architecture	90
Key Takeaways	91
6 Where Data Comes From: APIs & Real-World Sources	92
6.1 Learning Objectives	92
6.2 Four Ways Data Reaches You	92
6.3 What Is an API?	94
6.4 JSON: The Language APIs Speak	95
6.5 Hands-On: Calling Your First API	96
6.6 Two Words Every API User Needs	98
6.7 Is It OK to Take This Data?	98
6.8 Your Turn	100
6.9 Roadmap: From Raw Data to Understanding	100
III Unit III: Understanding and Modeling Data	101
7 Descriptive Statistics & Data Ethics	102
7.1 Learning Objectives	102
7.2 The Number That Lies While Telling the Truth	102
7.3 Three Ways to Describe the Middle	102
7.3.1 The Mean: Everybody Into the Pool	103
7.3.2 The Median: The One in the Middle	104
7.3.3 The Mode: The Crowd Favorite	104
7.4 Spread: How Different Are the Numbers From Each Other?	105
7.5 Correlation Is Not Causation	107
7.6 Data Has a Point of View — Revisited	108
7.7 Your Data, Your Privacy	110
7.8 Your Turn: The Homework Dataset	111
7.9 Roadmap: From Describing to Predicting	112
8 Introduction to Machine Learning	113
8.1 Learning Objectives	113
8.2 From Describing to Predicting	113
8.3 What Is Machine Learning?	114
8.4 Regression vs. Classification	115
8.5 Train/Test Split and Overfitting	116
8.6 Hands-On: Predicting Heart Rate with Linear Regression	118
8.7 Why Did the Model Do So Poorly? And What Can We Try Instead?	121

8.8	Hands-On: Trying a Decision Tree	121
8.9	Comparing the Two Models	123
8.10	Your Turn: Two Models Enter	124
8.11	Roadmap: The Same Idea, a Billion Times Bigger	125
9	How AI Learns From Data	126
9.1	Learning Objectives	126
9.2	The Other End of the Prompt	126
9.3	What Is a Large Language Model?	127
9.4	Where Does the Training Data Come From?	128
9.5	Prompting Is a Skill	129
9.6	When AI Gets It Confidently Wrong	131
9.7	Your Turn: Catch One in the Wild	133
9.8	Roadmap: Everything, All at Once	134
IV	Unit IV: Communicating and Applying Data Science	135
10	Designing Your Own Data Science Project	136
10.1	Learning Objectives	136
10.2	Your Capstone Project	136
10.3	Step 1: Choose a Dataset	137
10.4	Step 2: Define a Research Question	138
10.5	Step 3: Load and Clean Your Data	139
10.6	Step 4: Analyze and Visualize	140
10.7	Step 5 (Optional): Build a Prediction Model	140
10.8	What Makes a Good Project?	141
10.9	Your Turn: Stake Your Claim	141
10.10	Roadmap: From Analysis to Story	142
11	Telling Your Data Story	143
11.1	Learning Objectives	143
11.2	Data Alone Isn't Enough	143
11.3	The Four Elements of a Data Story	144
11.4	Define Your Key Message	146
11.5	Using ChatGPT to Sharpen Your Story	147
11.5.1	1. Check Clarity	147
11.5.2	2. Make It Engaging	147
11.5.3	3. Polish Language	148
11.5.4	4. Simulate Audience Q&A	148
11.5.5	5. Match Visuals to Narrative	148
11.6	Preparing for Audience Questions	149
11.7	Your Turn: Present Your Data Story	150

11.8 You've Built a Full Pipeline	151
---	-----

Appendices 152

A Glossary 152

A.1 Algorithmic Bias	152
A.2 API	152
A.3 Bias (in data)	152
A.4 Big Data	153
A.5 Boolean Masking	153
A.6 Causation	153
A.7 Cell	153
A.8 Cell Reference	153
A.9 Classification	154
A.10 Correlation	154
A.11 Dashboard	154
A.12 Data	154
A.13 Data Logging	154
A.14 Data Preparation	155
A.15 Data Privacy	155
A.16 Data Storytelling	155
A.17 Data Visualization	155
A.18 DataFrame	155
A.19 Dataset	156
A.20 Decision Tree	156
A.21 Dual-Axis Chart	156
A.22 Encoding	156
A.23 Endpoint	157
A.24 ETL	157
A.25 Filter (spreadsheet)	157
A.26 Formula	157
A.27 Google Colab	158
A.28 Hallucination (AI)	158
A.29 Integrated Development Environment (IDE)	158
A.30 JSON	158
A.31 Key Message	158
A.32 Large Language Model (LLM)	159
A.33 Library	159
A.34 Machine Learning	159
A.35 Mean	159
A.36 Mean Squared Error (MSE)	159
A.37 Median	160

A.38 Mode	160
A.39 Overfitting	160
A.40 pip	160
A.41 Pivot Table	160
A.42 Prompt	161
A.43 Rate Limit	161
A.44 Regression	161
A.45 Research Question	161
A.46 R ² Score	162
A.47 String	162
A.48 Structured Data	162
A.49 The Load Phase	162
A.50 Trace	163
A.51 Train/Test Split	163
A.52 Training Data (AI)	163
A.53 Transform	163
A.54 Unstructured Data	163
B Getting Set Up	165
B.1 Option A: PyCharm (If You Can Install Software)	165
B.2 Option B: Google Colab (Chromebook-Friendly, Nothing to Install)	166
B.2.1 Step 1: Go to the Colab Website	166
B.2.2 Step 2: Create a New Notebook	166
B.2.3 Step 3: Run Your First Code	167
B.2.4 Step 4: Working with Data Files (Where Colab Differs from PyCharm)	167
B.3 Which Should I Use?	168
C Dataset & Tool Directory	170
C.1 Where to Find Data	170
C.2 Tools Used in This Book	170
D Image Credits	171

Preface

Technology Service Center, University of Delaware

Why This Book Exists

Think about the last hour you spent on your phone. Every video your feed served you, every song your playlist queued up, every “players near you” match in your favorite game — none of it was random. Behind each of those moments, a computer looked at data about you (and millions of people like you) and made a prediction. Data science is the skill of understanding how those predictions get made — and learning to make them yourself.

That matters for two reasons. First, decisions about you are already being made with data: what content you see, what ads follow you around, even what colleges and employers might one day learn about you. Understanding how that works turns you from a passive target into an informed participant. Second, the ability to *question, filter, and reshape* data is a genuine superpower. Whether you end up in medicine, music, sports, gaming, or a career that doesn’t exist yet, the people who can turn raw numbers into honest answers will be the ones others turn to.

The good news: you don’t need to be a math genius or a seasoned programmer to start. You need curiosity and a willingness to ask good questions. This book supplies the rest.

What This Book Covers

The book is organized into four units, each building on the last:

1. **Unit I: Foundations** — what data actually is, how to work with it in spreadsheets, and how to turn it into visualizations that tell the truth.
2. **Unit II: Programming for Data Science** — your introduction to Python and Pandas, the tools professionals use when a dataset (like Delaware’s nearly 540,000 traffic crash records) is too big for a spreadsheet to handle.
3. **Unit III: Understanding and Modeling Data** — statistics, ethics, machine learning, and modern AI: how computers find patterns, and how to judge whether those patterns can be trusted.

4. **Unit IV: Communicating and Applying Data Science** — a capstone project and the art of data storytelling, because an insight nobody understands is an insight nobody uses.

Along the way you'll work with real datasets — traffic crashes, open government data, and information pulled live from the web — not toy examples invented for a textbook.

Before You Dive In

Two short pages follow this one, and they're worth reading first. **How to Use This Book** explains the features you'll see in every chapter — callout boxes, code you can run yourself, and end-of-chapter roadmaps — and how to get the most out of them. **Using AI Responsibly** sets out how tools like ChatGPT and Claude fit into your learning: when they help you grow and when they quietly rob you of the skills you came here to build.

That's it. Turn the page, and let's start asking better questions of the data all around you.

How to Use This Book

This book is more than just reading — it’s built for *doing*. As you work through each chapter, you’ll notice the same few features showing up again and again. Here’s your field guide to each one, so the first time you meet one in Chapter 1, you’ll know exactly what to do with it.

Definitions

When a key vocabulary word appears for the first time, you’ll see a blue note box like this. Every one of these terms is also collected in the **Glossary** appendix at the back of the book, so you can always flip there for a refresher.

Dataset

A **dataset** is an organized collection of data — often arranged in rows and columns, like a giant spreadsheet — that we can explore, question, and analyze.

Instructor Insights

These green boxes are tips and shortcuts from someone who has taught this material before. Think of them as advice passed down from students who sat in your seat last year.

Instructor Insight: Type It Yourself

It’s tempting to copy and paste code examples, but typing them out yourself — typos and all — is how the patterns actually stick. The mistakes you make (and fix) are where the learning happens.

Optional Asides

Every so often you’ll meet a yellow box like this one. These are optional detours — extra context that isn’t required to follow the chapter, and sometimes touches on something worth a grown-up’s input before you try it yourself.

! Optional Aside: You Can Skip This Box

Optional Asides never contain material the rest of the chapter depends on — skip one entirely and you’ll lose nothing. When a box like this suggests exploring something involving your own accounts or personal data, talk to a parent, guardian, or teacher first.

Knowledge Checks

After major sections, you’ll find a quick self-check question. Try to answer it in your head (or on paper) *before* clicking **Reveal Answer**. If you get it wrong, that’s useful information — go back and skim the section again. Try this one:

💡 Knowledge Check

Where can you find every Definition from this book collected in one place?

Reveal Answer

In the **Glossary** appendix at the back of the book.

Try With ChatGPT

These boxes give you a ready-to-use prompt for exploring the current topic further with ChatGPT. Copy the prompt, paste it into ChatGPT, and see where the conversation takes you. Before you use your first one, read [Using AI Responsibly](#) — the very next page — for the ground rules.

💡 Try With ChatGPT

Prompt: “I’m a high school student starting a data science course. Give me three examples of questions a data scientist might ask about my school’s cafeteria menu.”

Your Turn

Near the end of each chapter, you’ll find a **Your Turn** activity. Unlike a Knowledge Check — which takes thirty seconds — Your Turn is a larger, open-ended, hands-on exercise where you apply the whole chapter’s skills to real data. There’s usually no single right answer. These are the moments where you stop reading about data science and start actually doing it.

Ready? Let's talk about using AI the right way first.

Using AI Responsibly

Throughout this book you'll find **Try With ChatGPT** boxes — ready-made prompts for digging deeper into whatever you just learned. They're there for three things: exploring a concept beyond what one chapter can cover, checking whether you actually understood something, and getting unstuck when an error message makes no sense. What they are *not* is a shortcut past the understanding itself. Before you use your first one, read this page. It's short.

The One Rule: Tutor, Not Ghostwriter

A tutor explains things until you get them. A ghostwriter does the work so you don't have to. Use AI as the first, never the second.

Here's the practical reason, beyond honesty: if you paste in an answer you don't understand, the gap doesn't disappear — it just moves. It resurfaces on the next test, in the follow-up question you can't answer, and especially in the Chapter 10 capstone project, where you'll have to explain and defend your own analysis. Borrowed understanding runs out exactly when you need it most.

Good use: your pandas filter returned zero rows and you can't see why. Paste your code into ChatGPT and ask it to explain what your condition is actually checking. You're using AI to find *your* bug and understand it.

Bad use: pasting your capstone dataset into ChatGPT and asking it to write the whole analysis. That's not your project anymore — and you'll know it the moment someone asks you a question about it.

Always Verify

AI tools can be confidently, fluently wrong — especially about specific numbers, dates, and how library functions behave. If ChatGPT tells you what a pandas method does, run the code and see. If it states a fact, cross-check it against this book or another source. “It sounded sure of itself” is not evidence.

Ask Your Teacher

Every classroom sets its own rules about AI on graded work, and those rules win. The prompts in this book are suggestions for your own studying — they don't override your teacher's policy. When in doubt, ask first.

Part I

Unit I: Foundations

1 What Is Data?

1.1 Learning Objectives

By the end of this chapter, you will be able to:

- Define data and identify where it comes from in your everyday life.
- Distinguish between structured and unstructured data, with examples of each.
- Collect a small dataset about your own day using simple data logging.
- Explain why every dataset has a “point of view” — and why bias and missing information matter.
- Describe, in one sentence each, the major skills you will build across this book.

1.2 The Big Question

Here is the question this whole book hangs on: **What is data, and where does it come from in our lives?**

You might picture a wall of glowing green numbers, or a spreadsheet the size of a football field. But data is far more ordinary — and far closer to you — than that. Data is simply **information that can be collected, stored, and analyzed**. And right now, you are surrounded by it. Your phone counted the steps you took to get to this page. A traffic sensor noticed the bus you rode this morning. Your streaming app remembers exactly where you paused last night’s episode — down to the second.

i Data

Data is information that can be collected, stored, and analyzed. It comes from our actions, our devices, and the world around us — every step counted, song skipped, and door opened can become a data point.

In other words: data isn’t something that lives only in laboratories or tech companies. It is the *exhaust* of everyday life. Every time you act — walk, type, tap, buy, wait — the world quietly takes notes. Data science is the craft of reading those notes and finding the story inside them.

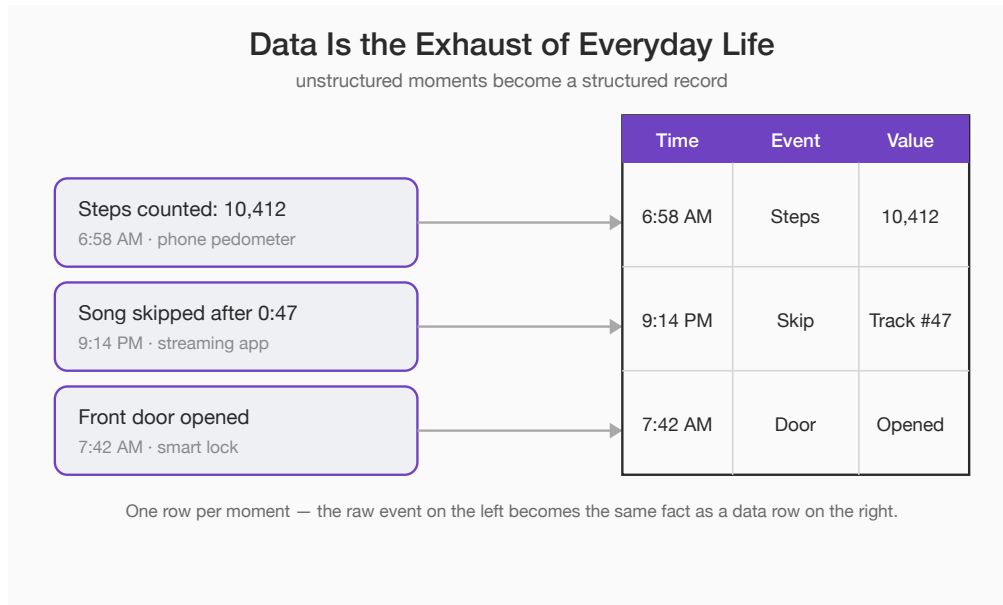


Figure 1.1: Three ordinary moments from a day, and the structured data row each one becomes.

Over the course of this book, you'll learn that craft step by step: how to clean and organize messy datasets so they're actually usable, how to build visualizations and dashboards that make patterns jump off the screen, how to train simple machine learning models that make predictions, how to spot when data is biased or misleading, and how data drives real decisions in tech, government, and business. But every one of those skills starts with the same move: recognizing data when you see it.

1.3 Structured vs. Unstructured Data

Not all data arrives in the same shape. Some of it shows up neatly filed, like books shelved in a library by call number. The rest shows up like a backpack dumped out on the floor — everything's *there*, but good luck finding your calculator.

Data scientists call these two shapes **structured** and **unstructured** data.

i Structured Data

Structured data is neatly organized information with a consistent format — usually rows and columns — which makes it easy for computers to search, sort, and analyze. Examples:

1. Your daily step counts (a number, tied to a date).

2. GPS coordinates from a phone's map app (latitude, longitude, timestamp).
3. Calendar events (a title, a date, a start time, an end time — always the same fields).

Structured data is a computer's favorite meal. Ask “What was my highest step count in March?” and a computer can answer instantly, because every entry has the same predictable shape.

Unstructured Data

Unstructured data is information with no fixed format, which makes it much harder for computers to process directly. Examples:

1. Photos in your camera roll (a grid of millions of colored pixels — nowhere does it *say* “birthday party”).
2. Videos (thousands of those photos per minute, plus sound).
3. Voice memos (sound waves, not sentences — a computer has to work hard just to figure out what words were said).

Here's a useful way to feel the difference. A calendar entry that says **Dentist - Tuesday, 3:30 PM** is structured: the computer knows exactly which part is the event and which part is the time. A photo of a sticky note that says “dentist tues 3:30!!” contains the *same information* — but to a computer, it's just pixels. Turning unstructured data into something analyzable is one of the hardest (and most interesting) jobs in data science, and it's a big part of why modern AI is such a breakthrough. We'll get there later in this book.

Knowledge Check

Your music app keeps two things about you: (a) a table of every song you've played, with the date, time, and how long you listened, and (b) the album cover images for those songs. Which is structured and which is unstructured — and why?

Reveal Answer

The play history table is **structured** — every entry has the same predictable fields (song, date, time, duration), so a computer can easily sort and analyze it. The album cover images are **unstructured** — they're just pixels with no fixed fields, so a computer can't directly “read” what's in them.

1.4 You Are a Data Source

Time to stop reading about data and start collecting some. Your subject: **you**.

Whether you realize it or not, your day already generates a steady stream of information. In data science, deliberately recording that stream is called **data logging**.

Data Logging

Data logging is the practice of recording observations or measurements over time, at regular intervals or as events happen, so the record can be analyzed later. A fitness tracker logs your steps automatically; a scientist logging tide heights every hour is doing the same thing by hand.

The activity: for the next 24 hours, be your own fitness tracker. Using paper, a notes app, or a simple table, log four things:

1. **Screen time** — roughly how many minutes you spend on a screen, and when.
2. **Steps or movement** — your phone’s built-in step counter works, or just note when you walked somewhere and for how long.
3. **What you ate** — a quick description and the time.
4. **Sleep** — when you went to bed and when you woke up.

That’s it. No apps to install, no accounts to create — just you, observing yourself the way a scientist observes the tides.

Tomorrow, look back over your log and hunt for patterns. Does your screen time spike at a particular hour? Do you snack more on days you move less? Is there a gap between when you *meant* to sleep and when you actually did? Even one day of honest data usually contains at least one small surprise.

Instructor Insight: Messy Data Is Normal Data

Students often apologize for their logs — “I forgot to write down lunch,” “I guessed on my screen time.” Don’t apologize; take notes. *Real* data is exactly like this: gaps, estimates, and honest guesses. Professional data scientists spend most of their time dealing with missing and messy values. Noticing where your own log got fuzzy is your first taste of a skill you’ll use in every chapter of this book.

1.5 A Different Kind of Data Source

Your 24-hour log took effort — you had to notice things and write them down. But some of the biggest data collections about you require no effort at all, because they’re being written *for* you.

! Optional Aside: Your Accounts Are Keeping a Diary (Read-Only — Ask First)

Services like Google keep a detailed, timestamped history of your activity: every search you’ve typed, every video you’ve watched, every event on your calendar. Google even offers a tool called **Google Takeout** that lets an account owner download that entire history as files.

This aside is **read-only and completely optional** — the exercise below uses a made-up sample, and *nothing in this book ever requires you to download or share your real account data*. If you’re curious about what your own accounts store, **talk to a parent, guardian, or teacher first** before touching any account settings or export tools. Real exports contain a lot of sensitive personal information and deserve adult guidance.

Instead of using anyone’s real data, here is a small **fictional sample** of what a few rows of an activity export might look like:

Timestamp	Service	Activity
Mar 14, 7:42 AM	Search	Searched: “how late does the number 12 bus run”
Mar 14, 3:58 PM	YouTube	Watched: “Beginner Guitar: Your First Three Chords”
Mar 14, 4:45 PM	Search	Searched: “easiest songs that only use 3 chords”
Mar 15, 6:00 PM	Calendar	Event: “Guitar practice with Sam”

Four rows. No name, no photo, no profile. And yet — read them like a detective. What can you reasonably guess about this person? They probably ride the bus. They seem to be teaching themselves guitar, and they’re just starting out. They likely have a friend (or teacher?) named Sam. Two searches and a video, and a little story assembles itself.

Now flip the question: **what’s missing, and what could be misread?** Maybe the guitar video was for a sibling. Maybe “Sam” is a cousin, a tutor, or a band. Maybe this person watched two hours of other videos on a different account we can’t see. Four rows *suggest* a story, but they don’t *prove* one — and a guess built on thin data can be confidently, completely wrong. Hold onto that thought, because it leads straight into the most important idea in this chapter.

1.6 Data Has a Point of View

It’s tempting to think of data as pure, neutral truth — after all, numbers don’t lie, right? Here’s the uncomfortable secret of data science: **every dataset has a point of view**. Data

reflects *how* it was collected, *who* it was collected from, and *what the collector decided to measure* — and every one of those choices leaves fingerprints.

Bias (in data)

Bias (in data) is a systematic tilt in a dataset — caused by how, where, or from whom the data was collected — that makes it misrepresent the real world. A biased dataset isn't “wrong numbers”; it's true numbers that tell a slanted story.

Consider a step counter. It measures activity by counting steps — a reasonable choice for many people. But for someone who uses a wheelchair, a step counter reads *zero* all day, no matter how many miles they push, how hard they train, or how active they are. The device isn't broken, and nobody typed in a false number. The **choice of what to measure** simply built a blind spot into the data. Anyone analyzing that data — a doctor, an insurance company, an app recommending fitness goals — would draw a badly wrong conclusion about a genuinely active person.

Your own 24-hour log has a point of view too. It measured what you chose to measure, when you remembered to measure it. Someone reading only your log would know about your screen time — but nothing about your mood, your conversations, or the joke that made your whole table laugh at lunch. The data isn't lying. It's just incomplete, in ways the numbers themselves will never admit.

This idea — that data can be biased, incomplete, or misleading while still being technically accurate — is one of the biggest themes of this book. In Chapter 7 (Descriptive Statistics & Data Ethics), we'll go much deeper: how averages can deceive, how biased data leads to unfair real-world decisions, and what responsible data scientists do about it. For now, just carry this habit with you: whenever you meet a dataset, ask *who collected this, how, and what might it be missing?*

Knowledge Check

A city wants to find its worst potholes, so it releases an app that uses a phone's motion sensors to auto-report bumps while residents drive. After a month, the map shows that wealthy neighborhoods have the most potholes. Is that conclusion trustworthy? Why or why not?

Reveal Answer

Not trustworthy. The data is biased by *how it was collected*: it only counts potholes felt by people who own smartphones, own cars, downloaded the app, and drive regularly. Neighborhoods with fewer app users report fewer potholes — no matter how bad their roads actually are. The map may really be showing where the *app users* are, not where the *potholes* are.

Try With ChatGPT

Prompt: “I’m a high school student learning what counts as data. List 10 everyday objects or activities in a typical student’s life that quietly generate data, and for each one, tell me whether that data is structured or unstructured. Try to surprise me with a few I wouldn’t think of.”

Compare ChatGPT’s list with your own 24-hour log — did it find data sources you missed? (This is the first Try With ChatGPT box in the book — see [Using AI Responsibly](#) for the ground rules before you start.)

1.7 Your Turn: The 72-Hour Data Log

Your 24-hour log was a warm-up. Now do the real study.

1. **Log the same four things — screen time, steps/movement, food, and sleep — for 2 to 3 days.** Keep the format consistent from day to day (that’s what makes your data *structured*).
2. **Add one measurement of your own choosing.** Money spent, minutes of homework, cups of water, songs played — anything you’re curious about.
3. **When you’re done, look across the days,** not just within one. Do weekdays look different from weekends? Does one measurement seem to rise when another falls?
4. **Write 3 sentences about what surprised you.** Not a summary — a *surprise*. The gap between what you expected and what the data showed is where data science lives.

Keep your log — you’ll want it again soon. There’s no wrong result here: even discovering “my log has huge gaps because logging is hard” is a genuine data science finding.

1.8 Roadmap: Where We Go Next

You now know what data is, where it hides in your life, and why every dataset deserves a healthy dose of skepticism. You’ve even collected a dataset of your own — which, right now, is probably scribbled in a notebook or scattered through a notes app. In Chapter 2, we’ll fix that: you’ll load data into a **spreadsheet**, the data scientist’s first power tool, and learn to sort it, filter it, and clean it up until patterns you couldn’t see on paper suddenly appear. Bring your 72-hour log — it’s about to become your first real dataset.

2 Data Wrangling with Spreadsheets

2.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain why “eyeballing” a dataset stops working once it grows past a few dozen rows.
- Use spreadsheet filters to narrow a huge dataset down to just the rows you care about.
- Write `=UNIQUE()` and `=COUNTIF()` formulas to summarize a categorical column automatically.
- Build a pivot table that groups and counts data across two categories at once.
- Recognize sorting, filtering, and grouping as the core moves of data analysis — moves you will repeat later in Python.

2.2 Why Spreadsheets First?

In [Chapter 1](#), you learned that **structured data** — information organized into consistent rows and columns — is the kind computers love. Well, there’s a tool for working with rows and columns that’s already sitting on your laptop: the spreadsheet. Before data scientists reach for code, most of them learned to think in Google Sheets or Excel, and there’s a good reason to start there.

Here’s an experiment you can do in your head. Imagine your 72-hour data log from Chapter 1 — maybe 20 rows of screen time, steps, and sleep. Want to find your highest-step day? Just look. Your eyes can scan 20 rows in a few seconds. Sorting and filtering *by eye* works fine at this scale.

Now scale up. The **Delaware Open Data Portal** (<https://data.delaware.gov/>) publishes a dataset of every reported traffic crash in the state from 2010 to 2024 — **539,947 rows**. Want to know which day of the week has the most crashes? Go ahead, start scanning. At one row per second, you’d finish in about six days — without sleeping, and assuming you could somehow keep a tally in your head the whole time.

Obviously nobody does that. The whole point of a spreadsheet is that *the software does the scanning for you*. You just have to learn to ask it the right way. In this chapter you’ll learn three ways to ask, each more powerful than the last:

1. **Filters** — “Hide everything except the rows I care about.”
2. **Formulas** — “Count these rows for me and write down the answer.”
3. **Pivot tables** — “Group everything by category and summarize each group, all at once.”

These three moves — filtering, counting, grouping — aren’t just spreadsheet tricks. They are *the* fundamental moves of data analysis, and you will meet them again, wearing Python costumes, in Unit II.

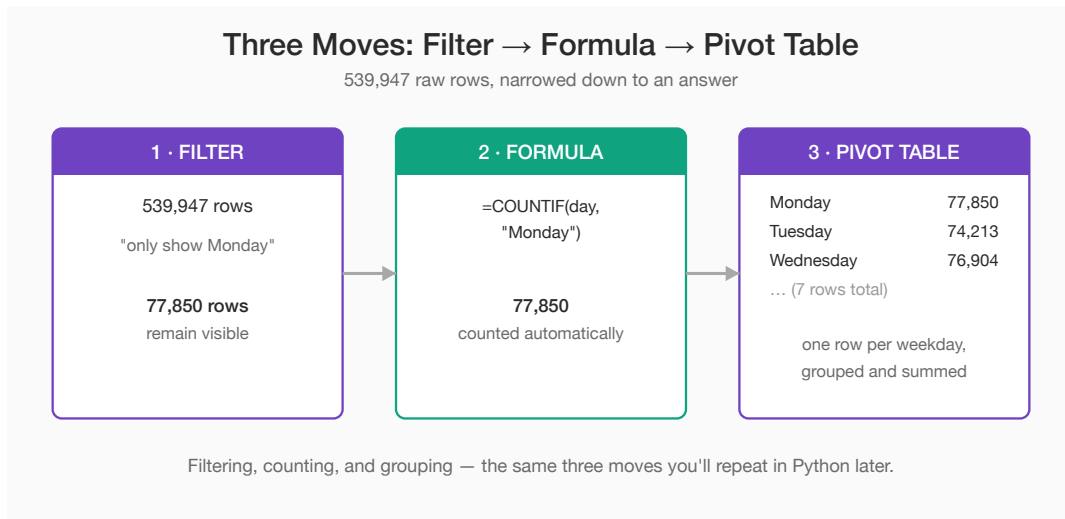


Figure 2.1: The three moves of this chapter, applied to the 539,947-row Delaware crash dataset: filter down, count with a formula, then group into a pivot table.

2.3 Sorting and Filtering

Let’s start with the biggest bang for the smallest effort. Open a dataset in Google Sheets, click the **Data** menu, and choose **Create a filter**. Small funnel-shaped icons appear on every column header — and suddenly every column is interactive.

i Filter (spreadsheet)

A **filter** is a spreadsheet feature that temporarily hides rows that don’t match conditions you choose, so you only see the subset you care about. The hidden rows aren’t deleted — turn the filter off and they all come back.

Click any column’s funnel icon and you can:

- **Sort** the whole sheet by that column (A→Z, Z→A, smallest→largest).
- **Filter by values** — check and uncheck boxes to show only certain categories.

- **Filter by condition** — show only rows where the value is greater than, less than, or between numbers you choose.

Here's how that plays out on the Delaware crash data. Suppose you only care about crashes in your own county during the last five years. Two clicks at a time:

1. On the **County** column, filter by values: uncheck everything except **New Castle**.
2. On the **Year** column, filter by condition: **is between 2020 and 2024**.

That's it. Roughly half a million rows collapse to just the tens of thousands that match — one county, one five-year window. Every formula, chart, and scroll from this point on operates on a dataset you can actually reason about.

Instructor Insight: Filters Don't Destroy

Students are often afraid to filter because it *looks* like the data disappeared. It didn't — the row numbers on the left will even skip (3, 4, 9, 10, 47...) to show you rows are merely hidden. Filtering is always reversible. Deleting is not. When in doubt, filter.

One honest warning: on a dataset this size, Google Sheets will be *slow*. Every filter click may bring a loading bar, and sometimes the dreaded spinning wheel. That sluggishness is real, it gets worse as data grows, and it's exactly the pain that will motivate us to learn Python in Unit II. For now, patience — the filters do work.

Knowledge Check

You filter the crash dataset to show only crashes where **County = Sussex** and **Year is between 2020 and 2024**. Your friend panics: “You deleted 500,000 rows of state data!” Are the other rows gone? And what two things could you do to see them again?

Reveal Answer

The rows are not gone — a filter only **hides** rows that don't match; the skipping row numbers prove they're still there. To see them again you could (1) turn the filter off entirely (Data menu → Remove filter), or (2) edit the filter conditions — for example, re-check the other counties or widen the year range.

2.4 Formulas That Do the Counting For You

Filters let you *see* a subset. But data science usually needs a number: *how many?* You could filter to Mondays, squint at the row count, write it down, then repeat six more times for the other weekdays... or you could make the spreadsheet do all of it with formulas.

Formula

A **formula** is an instruction you type into a spreadsheet cell, always starting with =, that tells the spreadsheet to calculate a value for you. When the underlying data changes, the formula automatically recalculates — you never have to redo the work by hand.

Formulas need a way to point at data, and that's what cell references are for.

Cell Reference

A **cell reference** is the address of a cell or range of cells, written as a column letter plus a row number. B2 means “column B, row 2.” A range like B2:B540000 means “every cell in column B from row 2 down to row 540000.” Formulas use references so they can work on data wherever it lives.

Let's answer a real question with the crash data: **which day of the week has the most crashes?** Suppose column B holds the weekday of each crash (Monday, Tuesday, ...), with the header in row 1 and data running from row 2 down.

Step 1: Find the distinct values with =UNIQUE(). Before counting anything, you need to know what categories even exist in the column. Click an empty cell (say, D2) and type:

```
=UNIQUE(B2:B540000)
```

The spreadsheet scans the entire weekday column and spills out each distinct value exactly once — seven cells, **Monday** through **Sunday**. On your own 72-hour log this would be overkill; on 539,947 rows, it's the only sane way to discover what's in a column. (It's also a great data-quality check: if **UNIQUE** returns eight values because someone typed **monday** in lowercase somewhere, you've just caught a problem you'd never have spotted by scrolling.)

Step 2: Count each category with =COUNTIF(). Now, next to the first unique value in D2, click cell E2 and type:

```
=COUNTIF(B2:B540000, D2)
```

Read it like a sentence: “Look through the range B2:B540000, and count every cell that matches whatever is in D2.” If D2 says **Monday**, E2 becomes the total number of Monday crashes in fifteen years of Delaware data — computed in about a second.

Step 3: Fill down. Drag the small square at the corner of E2 down six rows. The reference to D2 slides along with it (D3, D4, ...), so each row counts its own weekday. Seven formulas, and the entire question is answered:

D	E
Monday	78,412
Tuesday	79,905
Wednesday	81,338
Thursday	82,010
Friday	91,773
Saturday	72,441
Sunday	54,068

(Counts shown are illustrative — yours will depend on the exact dataset you download.) Notice what just happened: a question that would take *days* by eye took three formulas and a drag. And because formulas recalculate automatically, if the state publishes next year’s crashes tomorrow, your summary updates itself.



Try With ChatGPT

Prompt: “I’m a high school student learning Google Sheets. Explain, piece by piece, what the formula =COUNTIF(B2:B540000, D2) does. Then suggest a formula that would count only the crashes where column B says ‘Friday’ AND column C says ‘2023’ — and explain why plain COUNTIF can’t do that alone.”

Try the suggested formula on a real sheet before you trust it — does it return the same number you get by filtering to Friday + 2023 and reading the row count? Verifying an AI’s formula against a method you already trust is a habit worth keeping.

2.5 Pivot Tables: Grouping Made Visual

COUNTIF answered “how many crashes per weekday?” — one category, seven formulas. But real questions are often two-dimensional: *how many crashes happened on each weekday, in each year?* That’s 7 weekdays $\times$ 15 years = 105 combinations. Nobody wants to write 105 COUNTIF formulas.

This is the job of the **pivot table**: everything COUNTIF does, for every category combination at once, with zero formulas typed.



Pivot Table

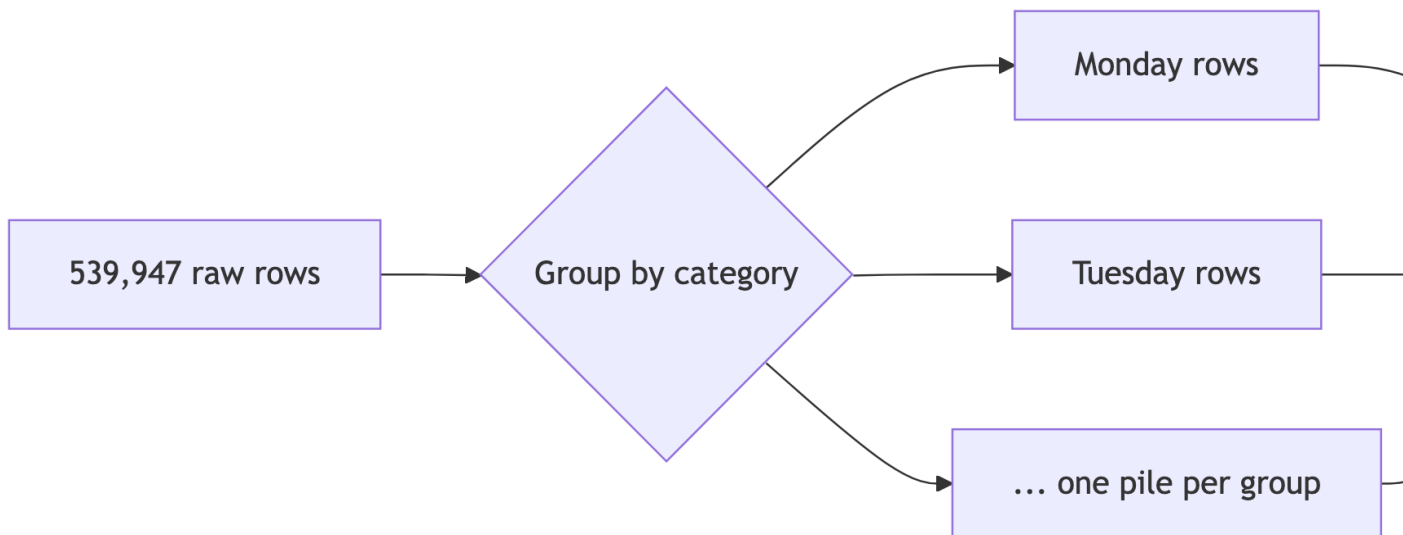
A **pivot table** is a spreadsheet tool that automatically groups rows by the categories in one or more columns and calculates a summary (count, sum, average...) for each group. It turns a long list of raw rows into a compact grid of totals — no formulas required.

In Google Sheets: select your data, then **Insert** → **Pivot table**. A blank grid appears with a panel offering four slots, and building the weekday-by-year summary is pure drag-and-drop:

- **Rows:** add the **Weekday** column → each weekday becomes a row label.
- **Columns:** add the **Year** column → each year becomes a column header.
- **Values:** add any column, summarized by **COUNTA** → each cell of the grid becomes the number of crashes for that weekday-year pair.

Ten seconds of dragging, and you're looking at a 7×15 grid: every Friday of 2020, every Sunday of 2024, all counted. Add **County** to the Filters slot and you can restrict the whole table to New Castle with one dropdown — the pivot equivalent of the filter you built earlier. Want averages instead of counts, or a different pair of categories? Drag the fields around and the table rebuilds itself instantly. That's why it's called a *pivot* table: you can spin the same data around to face any question.

The mental model to keep is **group, then summarize**:



This exact idea — group rows by a category and summarize them — comes back later as Python's `groupby()` in Chapter 5.

💡 Knowledge Check

You want to know the number of crashes for each **county** in each **month of the year** (January through December). Would you reach for `COUNTIF` or a pivot table — and if you chose the pivot table, what goes in the Rows, Columns, and Values slots?

Reveal Answer

A pivot table. With 3 counties × 12 months = 36 combinations, writing 36

COUNTIF formulas would be slow and error-prone. In the pivot table: put **County** in Rows, **Month** in Columns, and any column in **Values** summarized by COUNTA (count). Each cell of the resulting 3×12 grid is the crash count for one county in one month. (Swapping Rows and Columns also works — it’s the same table, pivoted.)

2.6 Your Turn: Summarize a Real Dataset

Time to run the full playbook yourself, start to finish.

1. **Get a dataset.** Download a small-to-medium dataset (a few hundred to a few thousand rows) as a CSV from the [Delaware Open Data Portal](#) or [Kaggle](#), and open it in Google Sheets. Pick one that has at least one **categorical column** — a column with repeating labels like a type, day, county, or category.
2. **Explore with a filter.** Turn on **Data** → **Create a filter** and narrow the data to one category or one range of values. Then bring everything back.
3. **Summarize with formulas.** In an empty column, use =UNIQUE() to list the distinct values of your categorical column. Next to it, use =COUNTIF() (filled down) to count how many rows match each value.
4. **Build one pivot table** that groups by your categorical column — and, if your data has a second interesting column, put it on the other axis. Check that the pivot table’s counts agree with your COUNTIF results. (They should. If they don’t, you’ve found either a typo in a formula or a data-quality surprise — both worth investigating.)

Bonus Challenge

Look at your finished summary and write down **one sentence** stating the most interesting pattern you see — for example, “Crashes peak on Fridays.” Then write a second sentence guessing *why*. Keep both sentences; you’ll want them in the next chapter.

2.7 Roadmap: From Tables to Pictures

You can now take a raw pile of thousands of rows and wrangle it into a small, meaningful summary table — distinct categories, counts per category, even a two-dimensional grid of totals. But a table of numbers still makes your reader do the work of finding the pattern. In **Chapter 3**, you’ll turn exactly these summary tables into charts — bar charts, line charts, and more — so the pattern jumps off the screen before anyone reads a single number. The wrangling you practiced here is step one of every chart you’ll ever make: first summarize, then visualize.

3 Data Visualization Basics

3.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain the difference between a **dataset** (the raw material) and a **dashboard** (the visual display built from it).
- Match a question to the right chart type — bar, line, pie, or scatter — and explain why it fits.
- Import a dataset into **CODAP**, a free browser-based tool, and build a chart with zero code.
- Compare two variables on one graph using CODAP’s drag-and-drop attributes and legends.
- Spot the three most common ways a chart can mislead its reader — and avoid building one yourself.

3.2 From Tables to Pictures

In [Chapter 2](#), you wrangled 539,947 rows of Delaware crash data down to a tidy seven-row summary: one row per weekday, one count per row. That table *contains* the answer to “which day has the most crashes?” — but it doesn’t *show* it. A reader still has to scan the numbers, compare them in their head, and hunt for the winner.

Now imagine the same seven numbers as seven bars. The tallest bar hits you before you’ve read a single label. That’s the entire promise of this chapter: a good chart moves the pattern from the reader’s head onto the page.

Data Visualization

Data visualization is the practice of representing data as pictures — charts, graphs, maps — so that patterns, trends, and outliers can be seen at a glance instead of computed in your head. It is the last step of the summarize-then-show pipeline you started in Chapter 2.

One of the earliest and most famous examples predates computers entirely. In 1858, nurse and statistician Florence Nightingale wanted to convince the British Army that unsanitary conditions — not battle wounds — were killing most of their soldiers. A table of numbers hadn't moved anyone. So she built this instead:

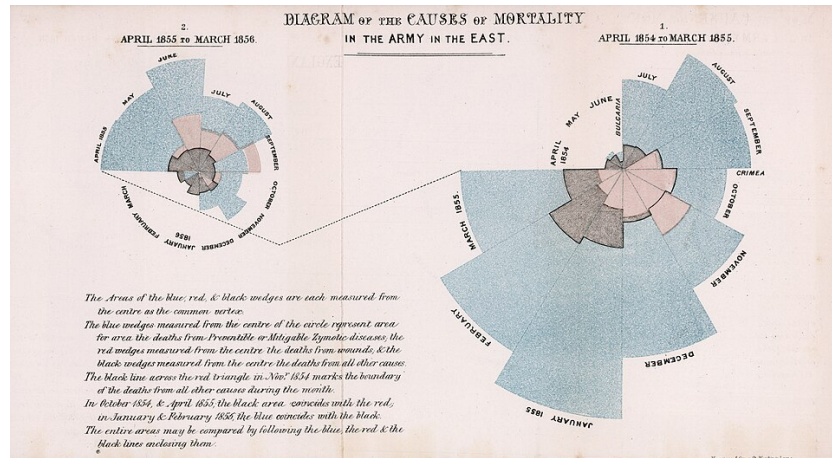


Figure 3.1: Florence Nightingale’s 1858 “Diagram of the Causes of Mortality in the Army in the East” — each wedge is a month, and the blue area shows deaths from preventable disease dwarfing the red area of deaths from battle wounds.

The Army reformed its hospitals soon after. That’s the whole promise of this chapter in one picture: the numbers didn’t change, but making them *visible* changed what people did about them.

3.3 Dataset vs. Dashboard

Two words you’ll hear constantly in data science are *dataset* and *dashboard*, and it’s worth being precise about the difference.

A **dataset** is the raw material: the full table of rows and columns, every crash, every timestamp, every county. It’s complete, it’s detailed, and it’s unreadable at a glance — like the engine of a car.

A **dashboard** is what you build *on top of* the dataset: a small set of visual displays that surface just the results that matter. Think of an actual car dashboard. Your car is tracking thousands of readings per second — fuel injection timing, oxygen levels, wheel rotation — but the dashboard shows you exactly three or four things: speed, fuel, temperature, and a warning light. Or think of your phone’s lock screen: the phone knows *everything* (every message, every app’s state, your battery’s exact voltage), but the lock screen curates it down to the time, the battery percent, and a couple of notifications.

i Dashboard

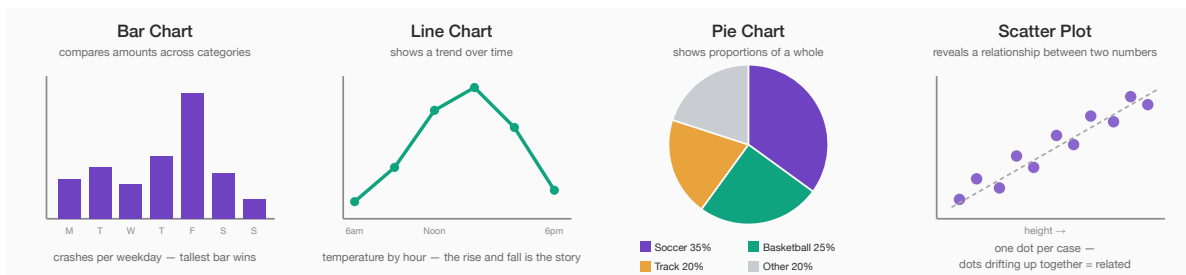
A **dashboard** is a visual display that gathers one or more charts (and sometimes tables or maps) in a single view, so a reader can absorb the most important results of a dataset at a glance — without ever touching the raw rows.

The skill of this chapter is exactly that curation: deciding *which* picture, built from *which* summary, tells the story best.

3.4 Choosing the Right Chart

Charts are not interchangeable. Each type answers a different *shape* of question, and picking the wrong one can bury your pattern. Here are the four you'll use most:

- **Bar chart** — compares amounts across **categories**. *Example:* number of crashes per weekday. Seven categories, seven bars, tallest bar wins.
- **Line chart** — shows a **trend over time** (or any ordered sequence). *Example:* temperature throughout the day, hour by hour — the line's rise and fall *is* the story.
- **Pie chart** — shows **proportions of a whole**, when the slices genuinely add up to 100%. *Example:* the percent of students who prefer each sport in your school. (Use sparingly: past five or six slices, humans are bad at comparing wedge sizes.)
- **Scatter plot** — reveals a **relationship between two numeric variables**, one dot per case. *Example:* does height predict weight? If the dots drift upward together, the two variables move together.



A quick self-test before you build anything: *Am I comparing categories? Bar. Tracking change over time? Line. Splitting a whole into parts? Pie. Looking for a relationship between two numbers? Scatter.*

Knowledge Check

For each question below, pick the best chart type (bar, line, pie, or scatter) and say why in one phrase:

1. “Has the number of crashes in Delaware gone up or down since 2010?”
2. “Do crashes happen more when it rains more? (Compare each day’s rainfall to that day’s crash count.)”
3. “What fraction of all 2024 crashes happened in each of Delaware’s three counties?”

Reveal Answer

1. **Line chart** — it’s a trend over time (years in order on the x-axis, crashes on the y-axis).
2. **Scatter plot** — it’s a relationship between two numeric variables (rainfall vs. crash count, one dot per day).
3. **Pie chart** (or a three-bar bar chart) — three parts of a whole that add to 100%. With only three slices, a pie is readable; a bar chart works too and makes the exact comparison easier.

3.5 Meet CODAP: Charts Without Code

To build charts, this book uses **CODAP** (Common Online Data Analysis Platform), a free tool from the Concord Consortium that runs entirely in your browser at <https://codap.concord.org/> — no download, no account, no code. Click **Launch CODAP** on that page, then **Create a New Document**, and you have a blank workspace.

CODAP is built around one idea: *drag and drop*. Data comes in as a **case table** (rows and columns, just like your spreadsheet), and you build graphs by physically dragging column names onto them. Whenever you get stuck, CODAP’s official [How-to Guides](#) cover every move in this chapter with short animations.

3.6 Hands-On: Building Your First Chart in CODAP

Let’s rebuild Chapter 2’s headline finding — crashes per weekday — as a real chart. You’ll need your summarized data from Chapter 2’s Your Turn activity (or make a new one the same way: `=UNIQUE()` for the weekdays, `=COUNTIF()` for the counts). In Google Sheets, select your two summary columns and download them as a CSV (**File** → **Download** → **Comma Separated Values**), so you have a small file with a **Weekday** column and a **Crashes** column — seven rows plus a header.

Step 1: Import the data. With a new CODAP document open, simply **drag your CSV file into the CODAP window** — that’s the whole import. If it worked, your data appears as a case table. (Prefer menus? Click the **File** menu, select **Import**, click the **Local File** tab, click inside the dashed box, choose your file, and select **Open**.)

Step 2: Make an empty graph. Click the **Graph** button on the top toolbar. A graph appears with your seven cases as points scattered randomly — CODAP is showing you the cases exist but doesn’t yet know what you want on the axes.

Step 3: Drag attributes onto the axes. In CODAP, columns are called **attributes**. Drag the **Weekday** attribute *from the column header of the case table* onto the graph’s horizontal (x) axis, and drag **Crashes** onto the vertical (y) axis. The random scatter snaps into order: one point per weekday, at the height of that day’s crash count. The pattern from your Chapter 2 table — Friday’s spike, Sunday’s dip — is now visible from across the room. (Changed your mind about an axis? Drag a new attribute onto it to replace the old one, or click the axis title to pick from a menu of attributes.)

Step 4: Turn dots into bars. Points work, but a bar chart makes comparisons even more legible. Make a second graph with just **Weekday** on the x-axis. To its right you’ll see the **inspector palette** — a slim strip of icons. Click the **Configuration** icon and select **Fuse Dots into Bars**. CODAP draws bars whose lengths are proportional to the *number of cases* in each category — which, in a one-row-per-weekday summary table, is 1 for every day! To make bar length come from your **Crashes** column instead, click the **Configuration** icon again, set the scale to **Formula**, then click the numeric axis label, choose **Edit Formula**, and enter **Crashes**. Now each bar’s length is that weekday’s crash count.

Step 5: Add a title. Every CODAP component has a title bar across its top. Click the title text on your graph and type something a stranger could understand, like *Delaware Crashes per Weekday, 2010–2024*. A chart without a title is a picture; a chart with one is an argument.

Instructor Insight: Bars Count Cases

Notice what Step 4 revealed: CODAP’s bar charts naturally count *rows*. If you imported the raw crash data — one row per crash — then **Weekday** on the x-axis plus **Fuse Dots into Bars** would give you correct bar heights automatically, no formula needed, because each category really would contain thousands of cases. Our summary table pre-counted the rows, so we had to tell CODAP where the counts live. Knowing *what your rows represent* is half of visualization.

3.7 Comparing Multiple Variables

One variable answers “how much?” Two variables answer the more interesting question: “compared to what?” Suppose your Chapter 2 pivot table gave you crashes per weekday for two

different years, so your CSV now has three columns: `Weekday`, `Crashes2023`, and `Crashes2024`. CODAP gives you two drag-and-drop ways to put both on screen.

Way 1: Add a second attribute to the y-axis. Build the Step 3 graph with `Weekday` on x and `Crashes2023` on y. Now drag `Crashes2024` onto the graph and drop it **directly on top of the plus sign** that appears near the top of the y-axis. When you're over the right spot, highlighting appears around the plus sign along with the text "*Add attribute Crashes2024.*" Release, and both years plot on the same graph in different colors — every weekday now shows two points, one per year, and you can see instantly whether 2024 ran higher or lower than 2023. One warning: if you drop the new attribute on the *axis itself* instead of the plus sign, CODAP will **replace** the first attribute rather than add a second one. Aim for the plus.

Each of those color-coded sets of points is what data scientists call a **trace**.

i Trace

A **trace** is one layer of data drawn on a chart — one line on a line chart, one set of same-colored points on a graph. A chart comparing two variables has two traces; the legend tells you which is which.

i Dual-Axis Chart

A **dual-axis chart** is a chart-literacy term for a graph with *two different* y-axis scales — one on the left, one on the right — so two variables with very different ranges (say, crash counts in the thousands and rainfall in inches) can share one picture. CODAP supports adding a second, right-side y-axis as a separate option, but handle the idea with care wherever you meet it: as you'll see in "How Charts Can Mislead," two independently-scaled axes are one of the easiest ways to accidentally lie with a chart.

Way 2: Color by a category (the legend). If your second variable is a *category* rather than a number — say each row also has a `County` column — drag that attribute name from the case table to the **center of the graph**. When the graph region turns yellow, release the mouse. CODAP colors every point by its category and adds a color-coded **legend** at the bottom of the graph, turning one cloud of points into several comparable groups.

And here's CODAP's quiet superpower, called **dynamic linking**: every table and graph in a document is connected. Click a bar, a point, or a row in *any* one of them, and the same cases light up in *all* of them. Select the Friday bar and watch Friday's row highlight in your table — the picture and the data are never out of sync.

3.8 Build Your Own Dashboard

Time to assemble the pieces. A CODAP document that holds a case table and two graphs, arranged so you can see them together, *is* a small dashboard — a curated view of your dataset, just like the car dashboard from earlier. Build one:

1. Using your crash summary data, make **two different charts of the same data** in one CODAP document — for example, the fused-bar chart of crashes per weekday from Step 4, and the two-year comparison graph from “Comparing Multiple Variables” (or a points-by-weekday graph with a legend).
2. Drag the components around the workspace so the table and both graphs are visible side by side, each with a clear title.
3. Click a weekday in one graph and watch dynamic linking highlight it everywhere.

Then answer, in writing: **which of your two charts tells the crash story better, and why?** There is a defensible answer for either choice — what matters is that your reason talks about the *reader* (“the bar chart makes Friday’s spike obvious without reading any numbers”) and not just the *builder* (“it was easier to make”).



Try With ChatGPT

Prompt: “I made a bar chart showing the number of traffic crashes for each day of the week, and Friday’s bar is clearly the tallest. My teacher asked whether a different chart type would tell this story more clearly. Describe the strengths and weaknesses of showing this data as a bar chart vs. a line chart vs. a pie chart, and tell me which you’d choose and why.”

Before you accept the answer, test it against what you learned in “Choosing the Right Chart”: weekdays are *categories*, not a continuous timeline — does ChatGPT’s recommendation respect that? If it suggests a line chart, ask it a follow-up: “What does a line between Tuesday and Wednesday imply that might not be true?”

3.9 How Charts Can Mislead

Charts are persuasive — that’s their job. But the same power that makes a real pattern obvious can make a fake pattern look real. Here are the three failure modes you’ll meet most often in the wild. Learn them as a reader first, and vow never to ship them as a builder.

1. The truncated axis. A bar chart’s y-axis starts at 4,980 instead of 0. Company A’s bar (5,000 sales) towers at *twice the height* of Company B’s bar (4,990 sales) — for a difference of ten sales, about 0.2%. Starting the axis just below the smallest value spends the entire vertical space on the *difference*, exaggerating it wildly. Bars encode quantity by length, so the length

has to start at zero; a bar chart with a non-zero start is showing you a zoomed-in sliver and letting your eyes believe it's the whole picture.

2. Mismatched scales. A graph shows two lines: your school's fundraiser (drawn against a left axis that runs 0–\$500) and a rival school's (drawn against a right axis running 0–\$50,000). The two lines cross and climb together, looking neck-and-neck — but the rival raised a hundred times more money. Whenever two variables are drawn on two *independently chosen* scales, the picture's "they look equal" or "they look correlated" is an artifact of whoever picked the scales. This is exactly why the dual-axis chart defined above demands suspicion: always read both axes before you believe the shapes.

3. The cherry-picked window. A headline chart shows crashes *falling* impressively from January to April — but the full fifteen-year dataset shows crashes *rising* steadily, with a dip every winter. By cropping the time window to one convenient downhill stretch, the chart-maker showed a true fact that tells a false story. The data isn't fake; the *frame* is. When a trend chart covers a suspiciously short or oddly specific period, ask what the years outside the frame were doing.

Knowledge Check

A news graphic shows a bar chart titled "Teen screen time EXPLODES!" comparing last year (6.2 hours/day) to this year (6.5 hours/day). The y-axis runs from 6.0 to 6.6, and this year's bar is three times the height of last year's. Which failure mode is this, what is the honest size of the change, and what one fix would make the chart truthful?

Reveal Answer

This is a **truncated axis**. The honest change is 0.3 hours on a base of 6.2 — roughly a 5% increase, not an explosion. The fix: start the y-axis at 0, so the two bars' lengths are proportional to the actual quantities. (Drawn honestly, the bars would look nearly identical — which is the true story.)

3.10 Your Turn: Two Charts, One Dataset

Run the full pipeline on data of your own choosing.

1. **Pick a dataset** from the [Delaware Open Data Portal](#) or [Kaggle](#) — ideally one with at least one categorical column and one or two numeric columns. If it's big, use your Chapter 2 skills to filter or summarize it in Google Sheets first, then download the result as a CSV.
2. **Import it into CODAP** (drag the CSV into a new document) and build **at least two different chart types** — for example, a fused-bar chart of counts per category and a scatter-style graph of two numeric attributes, or a graph with a legend comparing groups.

3. **For each chart, write 2–3 sentences** on what pattern it reveals — or hides. Be specific: “The bar chart reveals that one category is nearly double all the others, but it hides how that category changed over time” is the level of thinking to aim for.
4. **Bonus:** apply the “How Charts Can Mislead” checklist to your own charts. Does any axis start somewhere other than zero? Is any time window doing sneaky work? Fix anything you catch — being your own toughest chart critic is the habit that makes the rest of this book work.

3.11 Roadmap: When Even the Dashboard Isn’t Enough

You can now take raw data, wrangle it into a summary (Chapter 2), and turn that summary into charts that make the pattern undeniable (this chapter) — all without a line of code. But remember the honest warning from Chapter 2: at 539,947 rows, the spreadsheet itself was wheezing. Some datasets are too big even to *open* in Google Sheets, let alone filter and summarize there. In **Chapter 4**, you’ll meet Python and its data library, pandas — tools that do the same filtering, counting, and grouping you’ve been doing by hand, but on millions of rows in seconds. The moves won’t change; only the muscle behind them will.

Part II

Unit II: Programming for Data Science

4 Python & Pandas: Working with Big Data

Part 1: Foundations and the Power of Python

4.1 Learning Objectives

By the end of this tutorial, you will be able to:

- Identify the limitations of traditional spreadsheet software when handling large-scale datasets.
- Define Python’s core characteristics and its modular “LEGO-like” architecture.
- Recognize the diverse industries that utilize Python for data-driven decision-making.
- Understand the fundamental workflow of a data scientist: questioning, filtering, and reshaping data.

4.2 The Challenge: When Spreadsheets Fail

In our previous exploration of the **Delaware Open Data Portal**, we encountered a dataset documenting traffic crashes from 2010 to 2024. This dataset contains over **539,000 rows**.

When attempting to filter or analyze half a million records in tools like Google Sheets, the experience often becomes “slow and clunky,” frequently resulting in application crashes or the dreaded “spinning wheel of doom.” This highlights a critical question for any data professional: *Is there a better, easier way to work with such a big dataset?*

Big Data

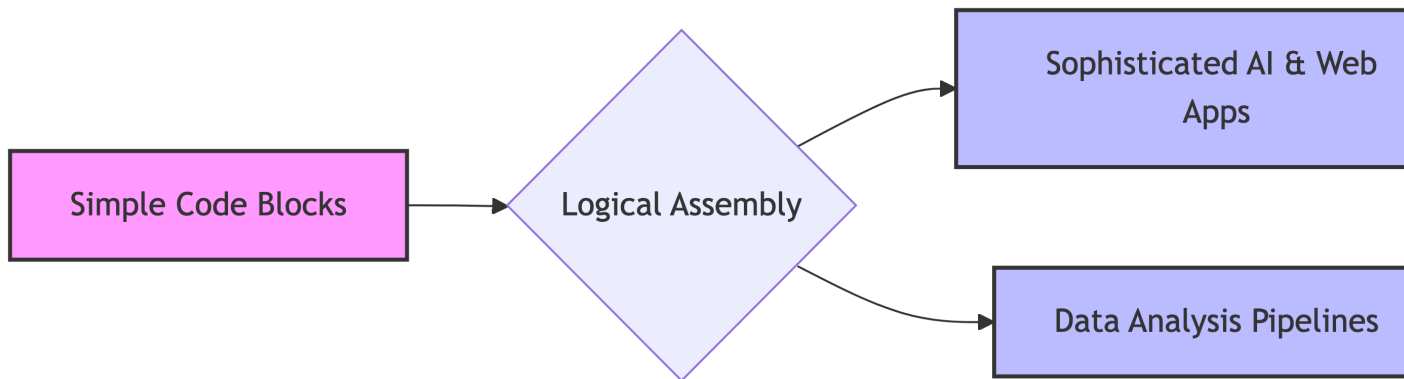
In this context, **Big Data** refers to datasets that are too large or complex to be dealt with efficiently by traditional data-processing software, necessitating more robust tools like Python and Pandas.

4.3 What is Python?

Python is a high-level, beginner-friendly programming language used across global industries. Its design philosophy emphasizes readability, often described as feeling almost like “plain English.”

4.3.1 The LEGO Metaphor for Logic

To understand how Python works, think of it as **LEGO for coding**. Just as simple, individual blocks snap together to build complex structures, Python uses simple, modular lines of code to build incredibly powerful applications.



4.3.2 The “iPhone” of Coding

Python is often called the **iPhone of coding** because it is sleek, approachable, and incredibly powerful. It abstracts away the complex, messy parts of computer science, allowing you to focus on solving problems rather than fighting with the tool.

💡 Instructor Insight: The Reward of Curiosity

Many beginners feel intimidated by coding, but remember: Python rewards curiosity. You don’t need to be a math genius to start; you just need to be willing to ask the right questions of your data.

4.4 Why Learn Python?

Python isn’t just a trend; it is the engine behind modern innovation. Its versatility allows it to be used for:

- **Healthcare:** Analyzing patient data and detecting diseases with AI.
- **Self-Driving Cars:** Writing the logic that helps vehicles “see” and navigate the road.
- **Finance:** Spotting market trends and making accurate predictions.
- **Gaming & Apps:** Building mobile applications and complex gaming environments.
- **Data Tasks:** Cleaning messy datasets—exactly what we will do with the Delaware crash data.

4.5 Python as a Superpower

With Python, you move from being a passive reader of data to an active problem solver. You don’t just “look” at nearly 540,000 rows; you **question, filter, and reshape** them into insights that matter. Even as a beginner, writing just a few lines of code can accomplish what would take hours to do manually in a spreadsheet.

4.5.1 Your First Step into the Console

The journey begins with the simplest command: telling the computer to speak back to you.

```
# The classic starting point for every programmer  
print('hello Python')
```

Output:

hello Python

The “Exit Code 0”

When you run a script in a professional environment, seeing **Process finished with exit code 0** is the computer’s way of giving you a “thumbs up,” meaning the code ran perfectly without errors.

Knowledge Check

Why does Google Sheets struggle with a dataset of nearly 540,000 rows, while Python and Pandas handle it easily?

Reveal Answer

Spreadsheet programs like Google Sheets try to keep the entire dataset in view and recalculate formulas across all visible cells, which gets slower and more

memory-intensive as rows grow. Pandas processes data programmatically in memory without needing to render every row, so it stays fast even at very large scales.

4.6 Roadmap for Part 1

Today we have established the “why.” Our next steps will take us into the “how”:

1. **Setting up PyCharm:** Preparing our professional digital workshop.
2. **Introduction to Pandas:** Loading and taming our first half-million rows of data.

Part 2: Getting Started with PyCharm

Learning Objectives

By the end of this tutorial, you will be able to:

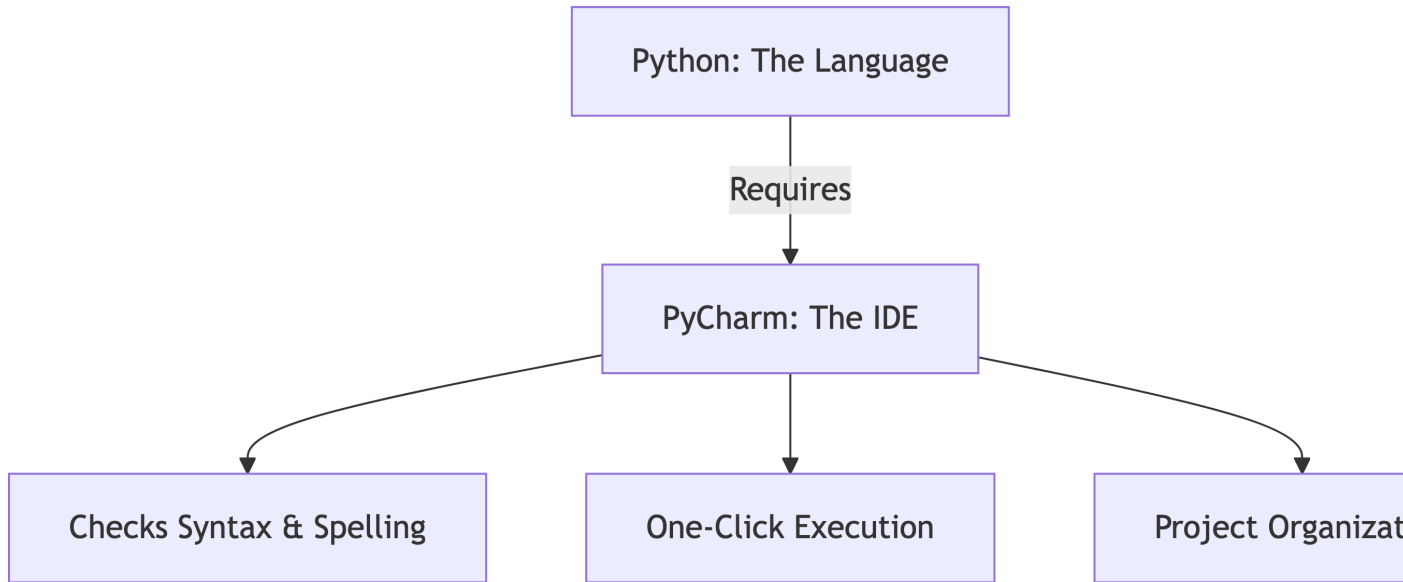
- Define the role of an Integrated Development Environment (IDE) in software development.
- Successfully install and configure PyCharm Community Edition on your operating system.
- Create a properly named Python project following industry naming conventions.
- Navigate the primary components of the PyCharm interface.
- Write and execute your first Python program and interpret console output.

Python is the Language; PyCharm is the Tool

Now that you understand what Python is, it is time to start writing it. While you can write code in a simple text editor, professional data scientists use a specialized tool called an **IDE**. Think of it like this: if Python is the language you are using to write an essay, PyCharm is the powerful word processor that helps you write faster and with fewer mistakes.

i Integrated Development Environment (IDE)

An **IDE** is a super-powered digital notebook designed specifically for writing, running, and managing code in one centralized location.



Setting Up Your Digital Workshop

To begin, you will need to download PyCharm from the official JetBrains website. PyCharm is a versatile tool built to support both beginners and experts alike.

Installation Steps

1. **Choose Your OS:** Select the version compatible with Windows, macOS, or Linux.
2. **Pick the Right Version:** Choose the **Community Edition**. It is completely free and perfect for those just starting their journey.
3. **Download and Setup:** Follow the installer prompts to complete the setup on your device.

Creating Your First Project

When you launch PyCharm for the first time, your first task is to create a new project. This is essentially creating the folder where all your work will live.

Naming and Saving Conventions

You have the freedom to choose your project name, but there are professional standards you should follow. You generally do not need to change the default save location path, but you should be intentional with the name.

💡 Instructor Insight: The “No Spaces” Rule

Computers can sometimes get confused by spaces in file names. When naming your project, **never use spaces**. Instead, use **underscores** (e.g., `my_first_project`) or **CamelCase** (e.g., `MyFirstProject`) to keep your files organized and error-free.

Navigating the Interface

Once your project is open, you will spend most of your time in two primary areas of the workspace.

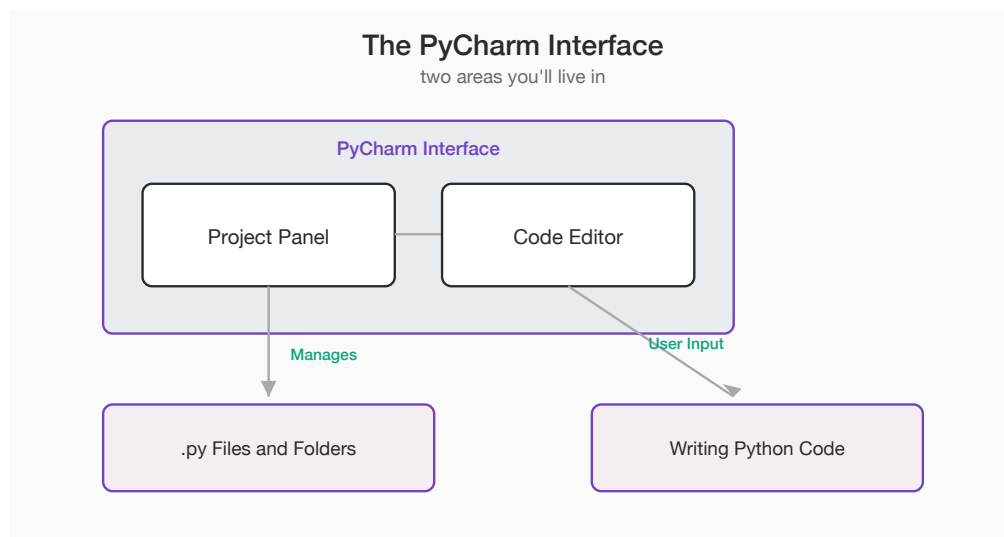


Figure 4.1: The two areas of the PyCharm workspace: the Project Panel manages your files and folders, while the Code Editor is where you write Python code.

- **Project Panel:** Located on the left, this shows all your folders and `.py` files.
- **Code Editor:** This is the main central area where you type, edit, and refine your Python code.

Writing and Running Your First Code

To verify your environment is working, we will use the `print()` function to display a message on the screen.

```
# Use the print function to display text in the console
print('hello Python')
```

Output:

```
hello Python
```

```
Process finished with exit code 0
```

Understanding the Syntax

- **The Function:** `print()` tells the computer to “show this message”.
- **The String:** Text placed inside quotes (either `'single'` or `"double"`) is called a **string**.
- **The Brackets:** The text you want to display must always go inside the parentheses of the function.

String

A **String** is a data type used in programming to represent text. In Python, strings are always wrapped in quotation marks.

Interpreting the Result

After clicking the **green run button**, PyCharm executes your file and shows the output in the **Run Console** at the bottom of the screen.

If your code runs successfully, you will see your message followed by the phrase “**Process finished with exit code 0**”. This is the computer’s way of confirming that the program ran perfectly with no errors.

Knowledge Check

You named a project “my first project” with spaces in the name. What’s likely to go wrong, and what should you do instead?

Reveal Answer

Spaces in file and folder names can confuse some tools and command-line operations. Use underscores (`my_first_project`) or CamelCase (`MyFirstProject`) instead.

 Instructor Insight: The “Thumbs Up”

Think of “Exit Code 0” as a digital “thumbs up” from your computer. If you see any other number, it means something went wrong, and it’s time to double-check your spelling and syntax!

Part 3: Meet Your New Best Friend—Pandas

Learning Objectives

By the end of this tutorial, you will be able to:

- Explain the role of the Pandas library in a typical data science workflow.
- Utilize the Terminal and `pip` to install external Python packages.
- Import libraries using aliases to streamline your code.
- Load external CSV files into a Python environment as DataFrames.
- Execute fundamental exploration functions (`head`, `shape`, `info`, `describe`) to audit a new dataset.

Meet Your New Best Friend: Pandas

In data science, **getting data into a usable shape is 80% of the work**. While Python provides the logic, **Pandas** is the library that turns raw, messy data into something you can actually understand, explore, and analyze. Whether you are working with crash records, sales sheets, or scientific measurements, Pandas is a data scientist's best friend.

Library

A **Library** is like a box of LEGO bricks—ready-made code you snap together to add functionality to your programs rather than building every tool from scratch.

Installing and Importing Pandas

Before you can use Pandas, you must fetch it from the internet and bring it into your project environment.

Step 1: Installation via Terminal

We use the **Terminal** in PyCharm—a text-based window for direct computer instructions—to talk to **pip**.

i pip

pip stands for “Package Installer for Python.” Think of it as Python’s own App Store. You tell it what you need, and it fetches and installs everything for you automatically.



Step 2: Importing into Your Code

Once installed, you bring Pandas into every Python file with a single line of code using an **alias**.

```
# The standard way to load Pandas
import pandas as pd
```

💡 Instructor Insight: Why “pd”?

The word **import** is the keyword that loads the library, and **pandas** is the name of the tool you installed. We add **as pd** as a **shortcut alias** so you can simply write **pd** instead of “pandas” every time you call a function.

Loading Real-World Data

The first step in any analysis is reading your file into Python’s memory. We do this by creating a **variable** to store a **DataFrame**.

i DataFrame

A **DataFrame** is the table Pandas stores your data in—rows and columns, like a spreadsheet—but handled entirely through code. `pd.read_csv()` is the function that reads a CSV file *into* a DataFrame.

```
# Load the Delaware crash data
df = pd.read_csv("delaware_crash_data.csv")
```

i Code Logic

1. **df**: A variable that stores your entire dataset — you can name it anything (e.g., `records`, `crash_data`).
2. `pd.read_csv()`: The specific tool that opens the CSV.
3. **File Path**: The text in quotes tells Python exactly where the file lives on your computer.

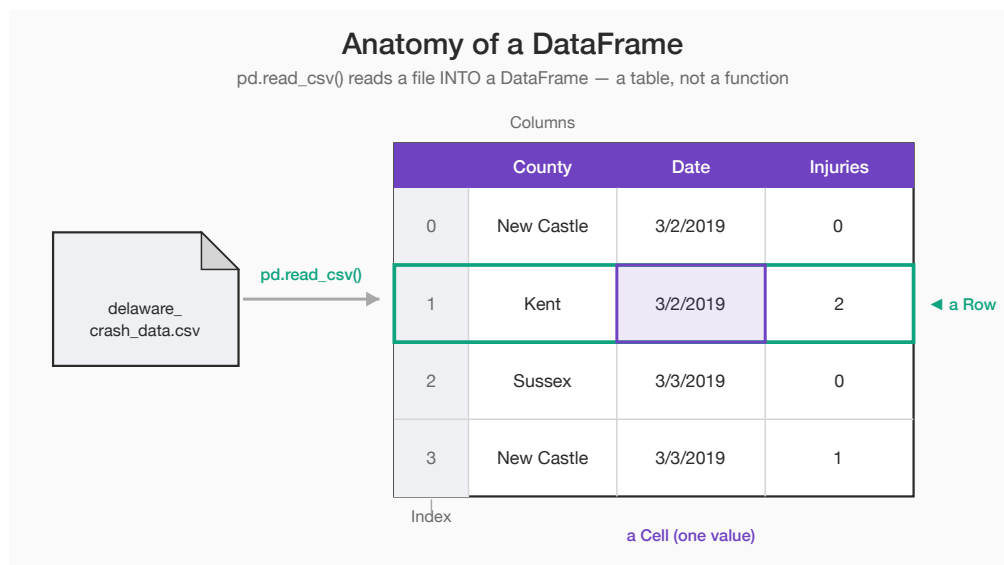


Figure 4.2: The parts of a DataFrame: `pd.read_csv()` reads a file into a table with an Index (row labels), Columns (field names), Rows, and individual Cells.

The Exploration Toolkit: First Steps

Once your data is loaded, a data analyst must immediately **peek inside**. These two commands are the foundation of every exploration workflow.

1. The Quick Preview: `df.head()`

This shows the **first 5 rows** of your dataset. Use it to quickly verify the file loaded correctly and to understand what columns are present before writing analysis code.

2. Checking the Scale: `df.shape`

This returns the **dimensions** of your DataFrame as (`rows`, `columns`). It is crucial for knowing if you are working with hundreds or hundreds of thousands of records.

💡 Instructor Insight: The 10-Second Habit

Develop the “Good Habit” of always running `df.head()` and `df.shape` right after loading a new dataset. It takes 10 seconds but saves hours of debugging later.

Understanding Your Data: Report Cards and Snapshots

To dive deeper, we use functions that audit the quality and types of data we have.

3. The Data Report Card: `df.info()`

This provides a concise summary of your entire DataFrame. Use it to spot missing data and understand whether columns are **numeric** or **text (object)** before you start cleaning. It displays:

- Total number of rows and columns.
- Column names and their **data types** (int, float, object).
- Count of **non-null** (non-missing) values per column.

4. The Statistics Snapshot: `df.describe()`

This generates quick summary statistics for **numeric columns**, including the count, mean, standard deviation, min/max values, and percentiles. It is perfect for spotting outliers and understanding data distribution at a glance.

Case Study: The Delaware Crash Data

When we applied these tools to the Delaware Crash dataset, the results provided a critical data science lesson.

- **Total Rows:** 539,947.
- **True Numeric Columns:** ~0.

Key Insight: Even though some columns *look* like numbers (such as weather codes or crash types), they are actually **categorical labels** disguised as digits.

Instructor Insight: Why `.info()` comes first

Running `.describe()` on categorical data produces misleading or meaningless results. This is why you must use `.info()` first to verify a column's true data type. Because most Delaware crash data columns are categorical, `.describe()` has limited usefulness here.

Try With ChatGPT

Copy the text output of `df.info()` from your own dataset and paste it into ChatGPT. Ask: *“Which of these columns are actually categorical, even though they might look numeric? How can you tell from this output?”*

Knowledge Check

You run `df.describe()` on a column called `weather_code` and get a mean, min, and max back — but the results don't make sense. What probably went wrong?

Reveal Answer

`weather_code` is probably categorical data (a coded label) stored as numbers, not truly numeric data. `.describe()` treats it as numeric anyway, producing meaningless statistics. Running `.info()` first would have revealed this.

Your Turn: Explore a Dataset

Now it is time to put these five functions to work on your own data. The goal is to discover something real and share your findings.

1. **Find a Dataset:** Use the Delaware crash data or find a new one with both categorical and numeric columns for richer results.

2. **Run the Functions:** Execute `read_csv()`, `head()`, `shape`, `info()`, and `describe()` in order.
3. **Share Your Discovery:** What surprised you? What patterns did you notice?

 Pro Tip

Datasets with true numeric columns (like temperature readings, prices, or test scores) will make `df.describe()` far more informative! You can find these on sites like Kaggle or data.gov.

Part 4: Filtering Like a Pro

Learning Objectives

By the end of this tutorial, you will be able to:

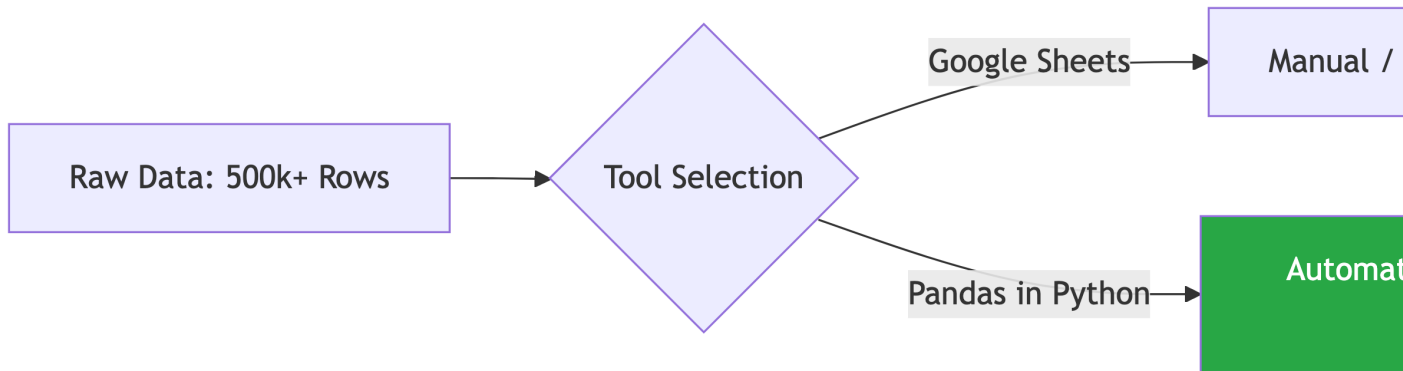
- Explain why programmatic filtering is superior to spreadsheet-based filtering for large datasets.
- Execute a multi-step data cleaning sequence to prepare columns for numeric comparison.
- Implement data filtering using three professional methods: `.loc`, `.query()`, and bracket notation.
- Apply complex conditional logic (AND, NOT EQUAL TO) to pinpoint specific data subsets.

From Sheets to Code: Why It Matters

Every data scientist’s “superpower” is the ability to ask precise questions of messy, massive datasets. In previous sessions, we utilized Google Sheets for basic filtering, but we quickly discovered its limitations. While a spreadsheet often breaks or slows down when handling millions of rows, Pandas handles them effortlessly.

Once you transition your filters into code, you unlock three major advantages:

1. **Scalability:** Process millions of rows without crashing.
2. **Automation:** Run the same analysis on new data instantly.
3. **Reusability:** Share your exact logic with others so they can replicate your results.



Step 1: Cleaning the Data First

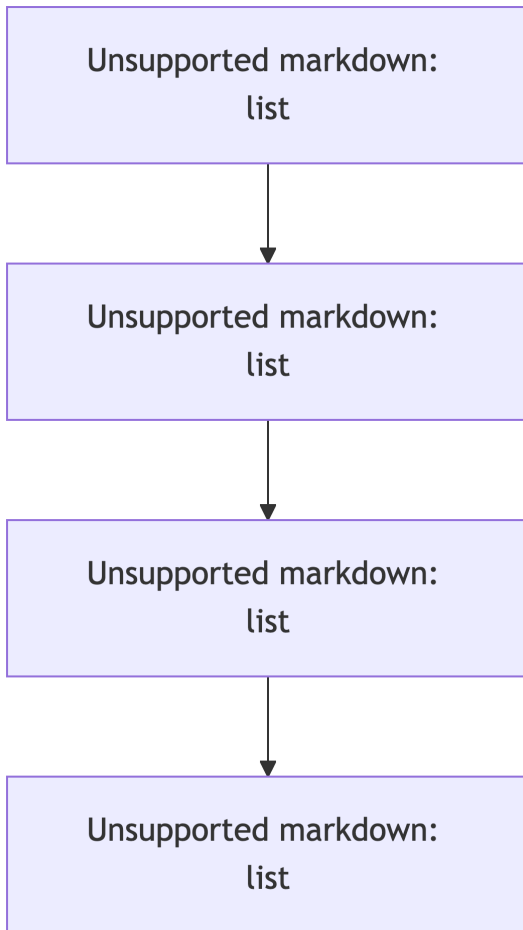
Before we can filter by specific years or values, our data must be “clean.” Raw data often contains formatting quirks—like commas in numbers—that prevent Python from performing mathematical comparisons.

i Data Preparation

Data Preparation is the process of cleaning and transforming raw data prior to processing and analysis. It often involves reformatting types and handling missing values.

To prepare our `YEAR` column for filtering, we follow this specific sequence:

1. **Remove Commas:** Strip characters that prevent numeric interpretation (e.g., “2,021” becomes “2021”).
2. **Convert to Numbers:** Use `pd.to_numeric()` with `errors='coerce'`. This turns text into numbers and marks unconvertible values as `NaN` (Not a Number).
3. **Drop Missing Values:** Delete rows where the `YEAR` is missing to ensure a continuous dataset.
4. **Cast to Integer:** Convert the cleaned numbers into whole integers (`int`) so they are ready for comparison.



Step 2: Three Professional Filtering Methods

Pandas provides multiple ways to extract the exact rows you need. While they achieve the same result, each has its own strength.

Method 1: The `.loc` Approach

The `.loc` method selects rows and columns based on labels or conditions. It is the “heavy-duty” tool of choice when you need to perform complex selections and specify which columns to keep simultaneously.

```
# Filtering for New Castle county and a specific year range
filtered_df = df.loc[
    (df["COUNTY NAME"].str.strip() == "New Castle") &
    (df["YEAR"] >= 2020) & (df["YEAR"] <= 2024)
]
```

💡 Instructor Insight: Beware Invisible Spaces

A common “aha!” moment occurs when a filter fails even though the spelling looks correct. Use `.str.strip()` on your column names or values to remove invisible leading or trailing spaces that often hide in raw CSV data!

Method 2: The `.query()` Method

If you prefer code that reads like a plain English sentence, `.query()` is your best friend. It allows you to write filter conditions as a concise string.

```
# Using a sentence-like format for readability
filtered_df = df.query("`COUNTY NAME` == 'New Castle' and YEAR >= 2020 and YEAR <= 2024")
```

i Why `.query()` Reads Like a Sentence

The `.query()` method is ideal for large datasets because it is concise and highly readable for collaborators.

Method 3: Direct Bracket Notation (Conditionals)

This is the most direct approach. You write a rule inside square brackets directly on the DataFrame. It is excellent for quick, one-off filters.

```
# Matching a specific value
df[df["COUNTY NAME"] == "New Castle"]

# Excluding specific values using != (Not Equal To)
df[df["YEAR"] != 2022]
```

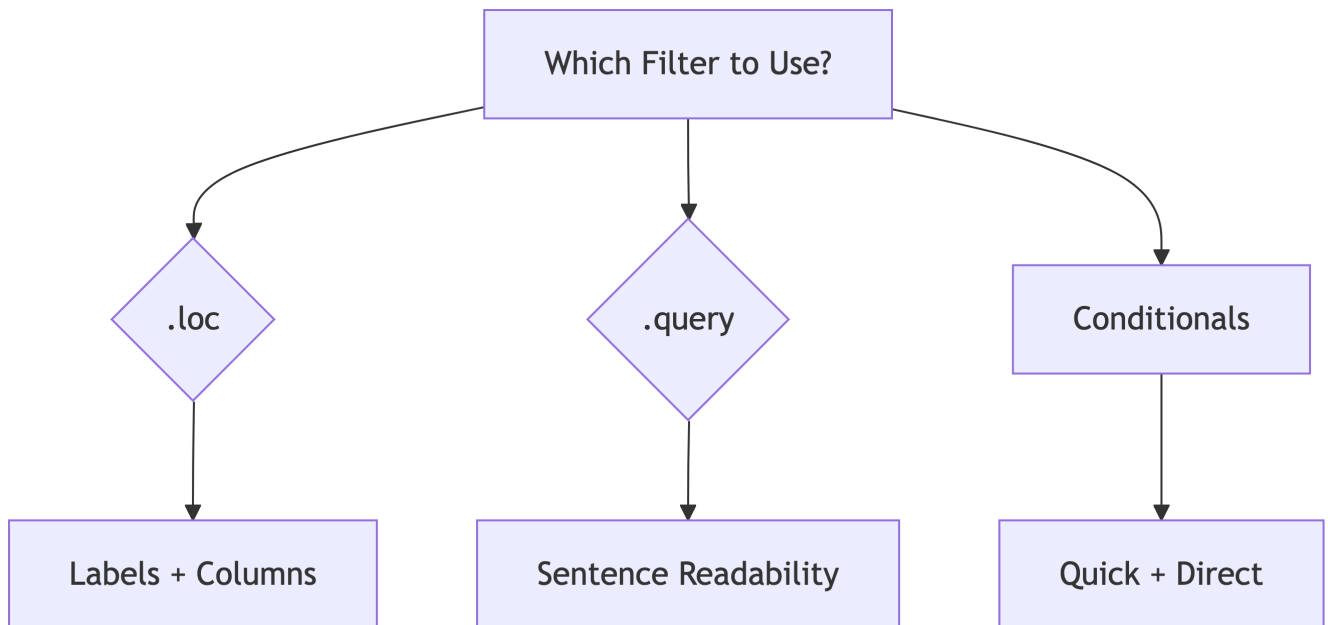
Boolean Masking

Bracket notation works by creating a **Boolean Mask**—a series of True/False values for every row. Python then only displays the rows marked “True.”

Comparison and Summary

All three methods return the same filtered result. Your choice depends on your specific needs:

- Use `.loc` when you need both row and column selection together.
- Use `.query()` when readability and clean syntax are your top priorities.
- Use **Conditionals** for quick, direct filtering of rows.



Knowledge Check

You filter for `df["COUNTY NAME"] == "New Castle"` and get zero rows back, even though you can see “New Castle” in the data. What’s the most likely cause?

Reveal Answer

Invisible leading or trailing whitespace in the county name values. Running `.str.strip()` on the column before filtering usually fixes this.

Your Mission: Filter Like a Pro

It is time to test your skills on the Delaware crash dataset or a dataset of your choice.

1. **Preparation:** Run the cleaning sequence on a numeric-style column (like Year or Cost).
2. **Implementation:** Apply the same filter using all three methods: `.loc`, `.query()`, and bracket notation.
3. **Validation:** Compare the `.shape` of your original DataFrame versus your filtered one.

Bonus Challenge

How many rows did your filter remove? Use `.shape` to calculate the difference and see if you can spot any interesting patterns in the data that remained!

5 ETL in Action: Cleaning & Preprocessing

Raw data is ubiquitous—it flows from your wrist, your phone, and across the web. However, before a data scientist can extract any meaningful value, that data must be cleaned, organized, and prepared. This tutorial introduces the **ETL process**, the essential backbone of every real-world data pipeline.

5.1 Learning Objectives

By the end of this tutorial, you will be able to:

- Define the three stages of the ETL process.
- Explain how ETL functions as a bridge within a standard data pipeline.
- Identify the risks of “messy data” and the advantages of standardized data.
- Recognize real-world applications of ETL in fitness, healthcare, and e-commerce.

5.2 What is ETL?

i ETL

ETL stands for **Extract, Transform, and Load**. It is a three-stage process designed to turn raw, messy data into something clean, structured, and ready for analysis or machine learning.

The process follows a logical progression:

1. **Extract:** Pulling raw data from disparate sources like apps, sensors, databases, and websites.
2. **Transform:** Cleaning, converting, and organizing that data into a usable format.
3. **Load:** Storing the processed data where it can be analyzed or used to train models.

5.2.1 Visualizing the ETL Flow

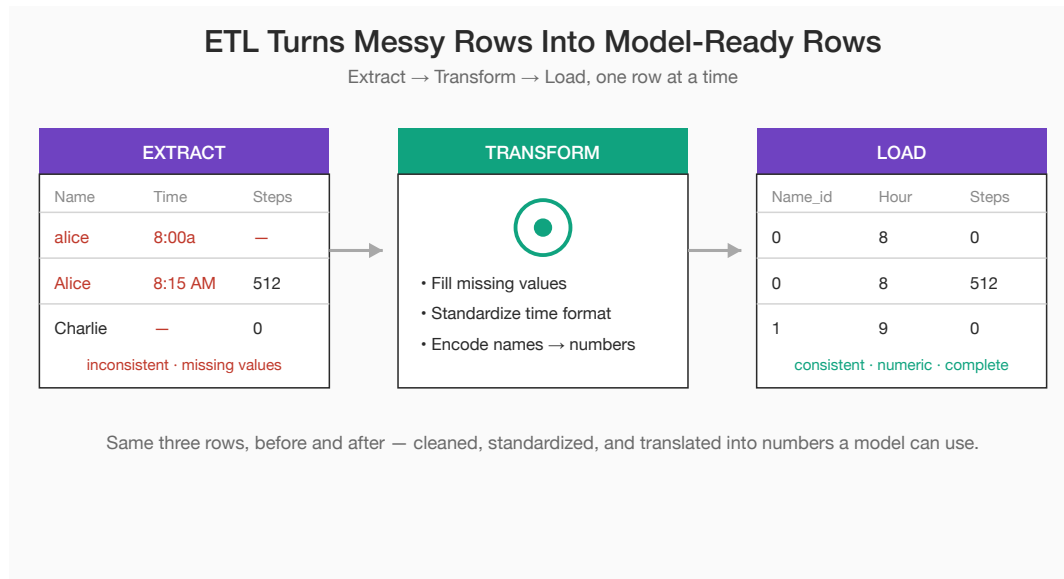
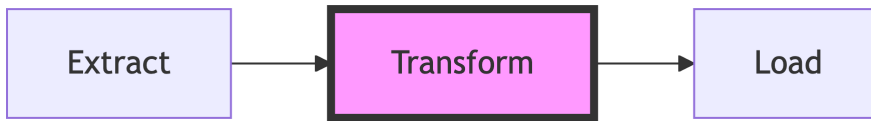


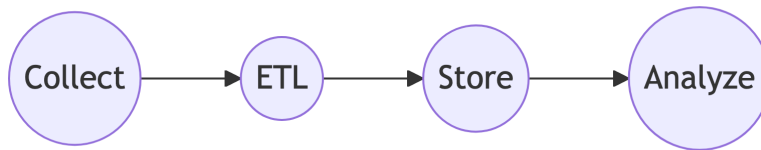
Figure 5.1: The same three rows before and after ETL: messy, inconsistent values on the left become clean, standardized, numeric values on the right.

5.3 How ETL Fits Into the Data Pipeline

ETL serves as the **bridge between raw data and real insights**. Without it, data remains scattered across systems and impossible to use effectively. Every chart, recommendation engine, or machine learning model you encounter depends on data that has traversed this pipeline.

5.3.1 The Data Journey

The full journey of data can be visualized as a continuous flow:



5.4 Why ETL Matters: Solving the Messy Data Problem

Working without a dedicated ETL process leads to the “Messy Data Problem”. Inconsistent fields, mislabeled data, and missing values can break reports and lead to unreliable insights.

Without ETL	With ETL
Data is scattered across multiple sources	Data is unified into a single clean source
Fields are inconsistent or mislabeled	Formats are standardized and consistent
Missing values break models and reports	Gaps are identified and handled properly
Insights are unreliable or impossible to reach	Analysis and ML models can run with confidence

5.5 ETL in the Real World

ETL isn’t just a theoretical concept; it powers the technology we use every day.

- **Fitness Trackers:** Your smartwatch extracts raw sensor data, transforms it into readable statistics (like steps or heart rate), and loads it into an app dashboard.
- **Healthcare Records:** Patient data from multiple hospitals is cleaned and merged via ETL to support medical research and diagnostic tools.
- **E-Commerce:** Purchase histories from millions of users are processed through ETL pipelines to power the recommendation engines that suggest what you might buy next.

5.6 Reflective Code: Programmatic Extraction

As discussed in earlier sessions of this series, we often use libraries like **pandas** to perform the initial “Extract” step from complex sources like multi-sheet Excel files.

```
import pandas as pd

# Extracting all sheets from a raw data file
df_dict = pd.read_excel('activity_data.xlsx', sheet_name=None)

# Transforming by concatenating separate sheets into a master table
df_all = pd.concat(df_dict.values())

print(df_all.head())
```

What You’ll See:

```
Checking data alignment...
[OK] 11 columns identified
[OK] 3 daily sheets unified
```

Knowledge Check

A hospital wants to combine patient records from three different clinics, each using a different field name for “date of birth.” Which stage of ETL handles renaming these fields so they match?

Reveal Answer

Transform. Extract just pulls the raw data in from each source; standardizing field names happens during Transform, before Load.

5.7 Putting It All Together: The ETL Mindset

💡 Instructor Insight: ETL as a Mindset

ETL isn't just a technical process—it's a **mindset**. Every data scientist must constantly ask: *Where does my data come from? Is it trustworthy? Is it ready to use?*

Raw sensor readings or database entries mean nothing until they are extracted, cleaned of errors, and loaded into a system where patterns can emerge.

Key Takeaway: Clean data is not a starting point—it's an achievement. ETL is how you get there.

Part 2: The Extraction Frontier

Raw data is almost never ready for immediate use. Before a single insight can be drawn or a model trained, data must travel through a pipeline that separates amateur analysis from professional data engineering. This tutorial focuses on **Extraction**, the first and most critical mile of the data science journey.

Learning Objectives

By the end of this tutorial, you will be able to:

- Explain why ETL is the foundational backbone of data-driven decision-making.
- Identify key sources for real-world datasets, including open data portals and Kaggle.
- Understand the multi-dimensional structure of activity-based datasets.
- Use Python and the Pandas library to programmatically extract and unify data from multi-sheet workbooks.

Why ETL is the Foundation of Data Science

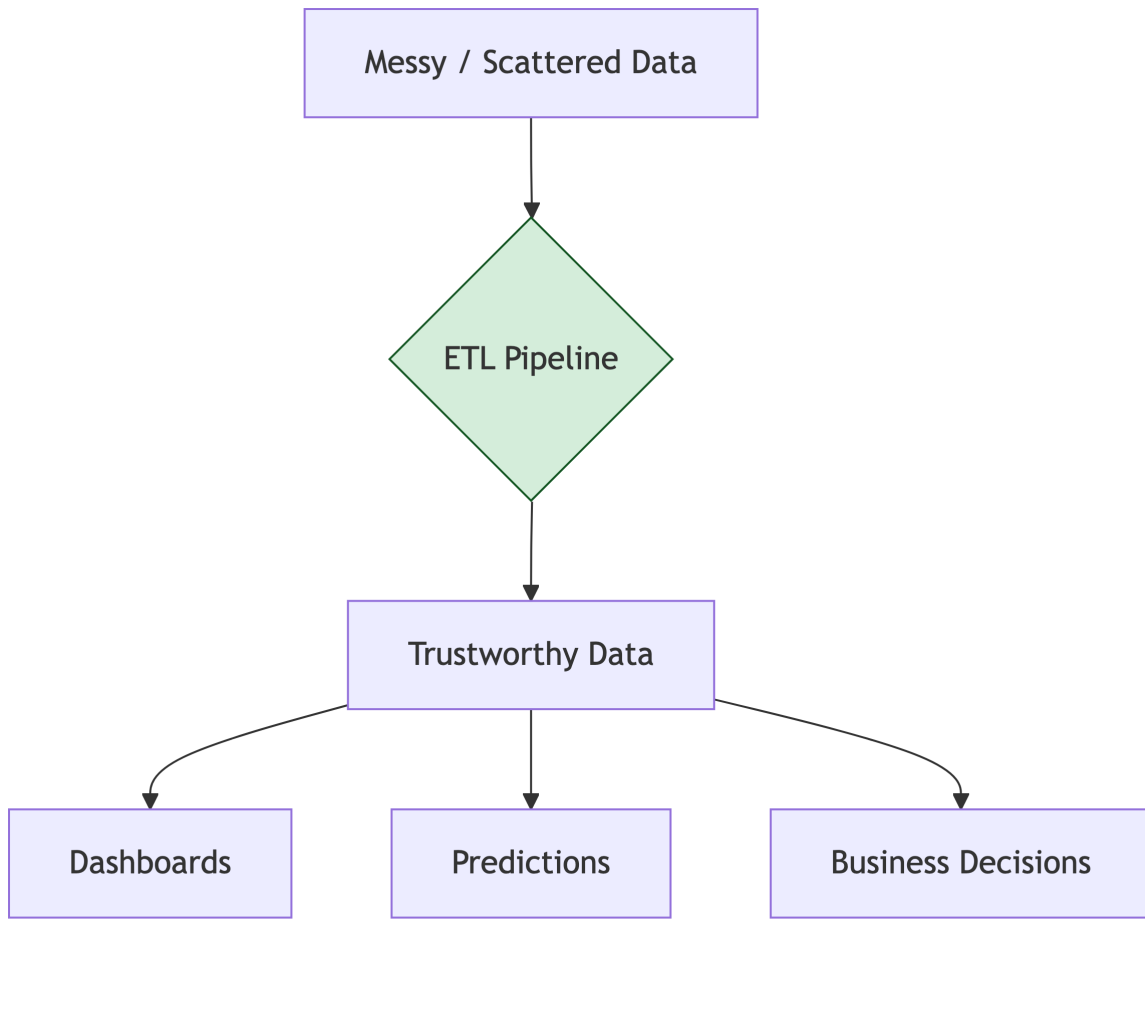
Every great data science project begins with messy, scattered, and incomplete data. Without a structured process to handle this “noise,” even the most advanced machine learning algorithms will produce unreliable results.

i ETL

ETL stands for **Extract, Transform, and Load**. It is the invisible backbone of every dashboard and prediction model, ensuring that data is trustworthy, consistent, and analysis-ready.

The golden rule of data engineering is “Garbage In, Garbage Out.” Clean data is not an optional luxury; it is foundational to the integrity of your work.

Visualizing the Pipeline Logic



Phase 1: Extract — Where It All Begins

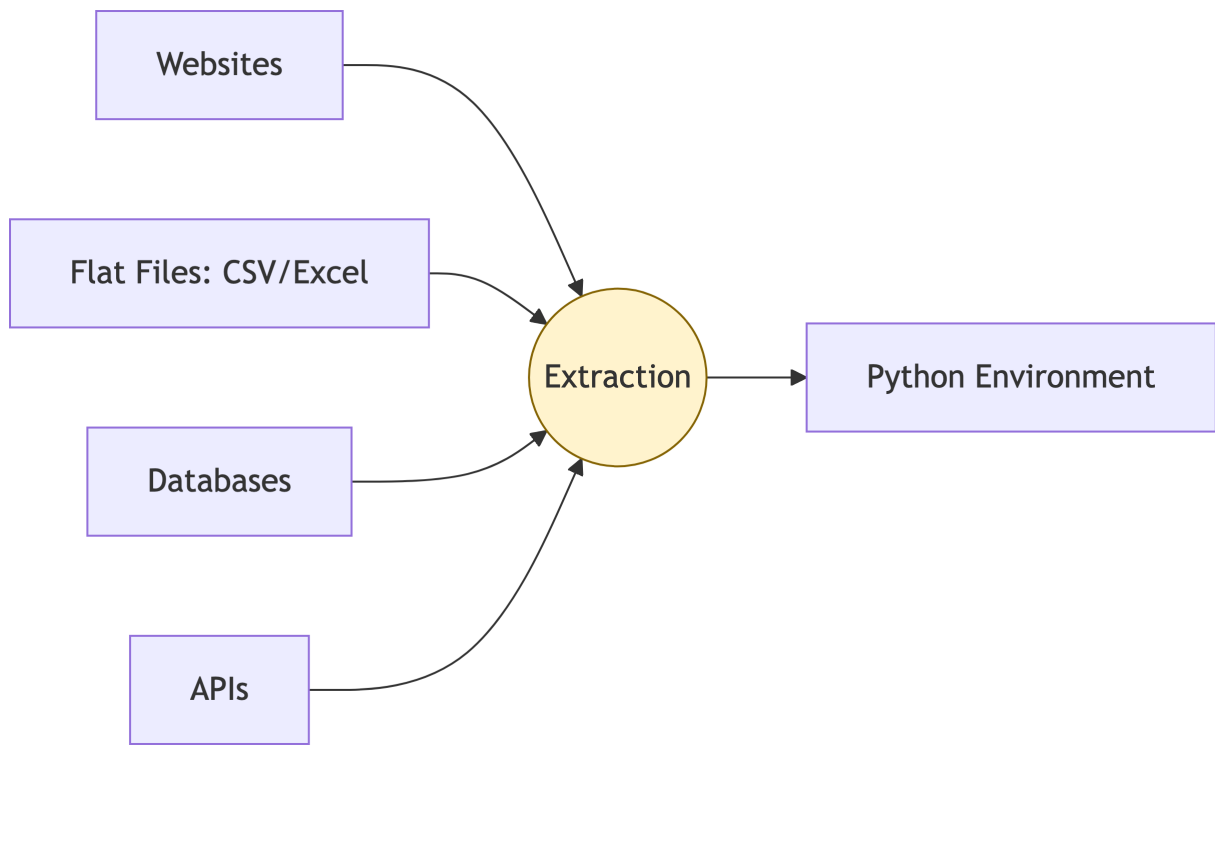
Extraction is the act of pulling raw data from any source—be it websites, flat files (CSV/Excel), databases, or APIs. Getting the right data from the right source is a skill in itself.

Finding Your Playground

To practice extraction, you need reliable data sources. Two primary hubs for data scientists include:

1. **Delaware Open Data Portal:** A government-run repository for real-world datasets covering everything from education to public health.
2. **Kaggle:** The premier platform for data science learners. Kaggle hosts thousands of datasets, from Titanic passenger records to YouTube comment trends, and serves as both a learning lab and a proving ground.

Abstract Concept: The Extraction Web



Anatomy of an Activity Dataset

In this tutorial, we work with a **Simulated Daily Activity Record**. This dataset is structured to mimic the complexity you will find in the real world:

- **Timeline:** Tracks three individuals over three days (July 28–30, 2025).
- **Format:** A multi-sheet Excel workbook where each day lives in its own sheet.
- **Granularity:** One record per hour per person.

- **Dimensions:** 11 columns total, blending personal identity, biometrics (Steps/HeartRate), location (GPS), and behavioral spend data.

💡 Instructor Insight: Multi-Dimensionality

Real-world data is rarely “flat.” The most valuable insights often come from datasets that blend different types of information—like spatial GPS data and physiological heart rate data—simultaneously.

Hands-On: Loading Your Dataset with Pandas

We use the **Pandas** library as our primary data manipulation toolkit. The following code demonstrates how to programmatically extract data from a multi-sheet Excel file and unify it into a single table.

```
import pandas as pd

# Step 1: Load all sheets into a dictionary of DataFrames
# Setting sheet_name=None ensures every tab is captured
df = pd.read_excel("activity_data.xlsx", sheet_name=None)

# Step 2: Combine all sheets into one unified master table
# ignore_index=True ensures the row numbering is continuous
df_all = pd.concat(df.values(), ignore_index=True)

# Step 3: Confirm everything loaded correctly
print(df_all.head())
```

What You'll See:

	Date	Hour	Name	Steps	HeartRate
0	2025-07-28	09:00	Alice	1200	72
1	2025-07-28	10:00	Alice	3500	85
2	2025-07-28	11:00	Alice	4200	90
3	2025-07-28	09:00	Charlie	800	68
4	2025-07-28	10:00	Charlie	5600	110

That's the first 5 of all 216 rows — `head()` always shows just the top of the table, and only 5 of the dataset's 11 columns fit on one line here; the rest (**Gender**, **ActivityType**, GPS coordinates, and spending) come into play in the next few sections.

Knowledge Check

Why does the code use `sheet_name=None` when loading the Excel file, and `ignore_index=True` when combining the sheets?

Reveal Answer

`sheet_name=None` tells pandas to load every sheet in the workbook — not just the first one — into a dictionary of DataFrames. `ignore_index=True` renumbers the combined rows continuously instead of restarting the row index at 0 for each sheet.

Transformation Roadmap

Once the data is extracted, it enters the **Transform** phase. This is where raw data becomes analytically useful through five primary techniques:

1. **Normalization:** Scaling numeric values to a common range.
 2. **Sequential Transformation:** Handling time-ordered data to find trends.
 3. **Encoding:** Converting categories (like Activity Type) into numbers.
 4. **Merging:** Combining multiple sources into one cohesive DataFrame.
 5. **Filtering:** Isolating rows that meet specific, actionable conditions.
-

Summary: The First Mile

Every step you have taken—from browsing data portals to writing your first lines of Pandas code—is part of a continuous data engineering workflow. By mastering the **Extract** phase, you have mastered the first mile of every data science project. You are now ready to move into the “Engine Room” of transformation.

Part 3: The Transform Engine Room

Raw data is rarely ready for use. In this tutorial, we explore the Transform stage—the critical engine room where messy, inconsistent records are reshaped into structured, machine-ready insights through normalization, sequential sorting, and geospatial engineering.

Transform: The Bridge Between Raw and Ready

Imagine trying to build a high-performance engine using parts from three different countries, each using different measurement systems. Some parts are in inches, others in millimeters, and some aren't labeled at all. This is exactly what a data scientist faces with raw data.

The **Transform** step is where data gains its meaning. It is the process of converting raw records into a cleaner, more useful format. Without it, even the best datasets remain unusable—full of incompatible units, unreadable categories, and missing context.

Learning Objectives

By the end of this tutorial, you will be able to:

- Explain the core purpose of data transformation in the ETL pipeline.
 - Apply **Normalization** to rescale disparate numeric columns.
 - Utilize **Sequential Transformation** to uncover behavioral patterns over time.
 - Implement **Geospatial Transformation** to convert raw coordinates into human-readable features.
-

The Purpose of Transformation

Transform

Transform is the process of converting raw data into a structured format by applying operations like filtering, encoding, normalization, or feature engineering.

Every transformation step solves a specific real-world problem, making data comparable, interpretable, and ready for mathematical models.

Visualizing the Transformation Pipeline

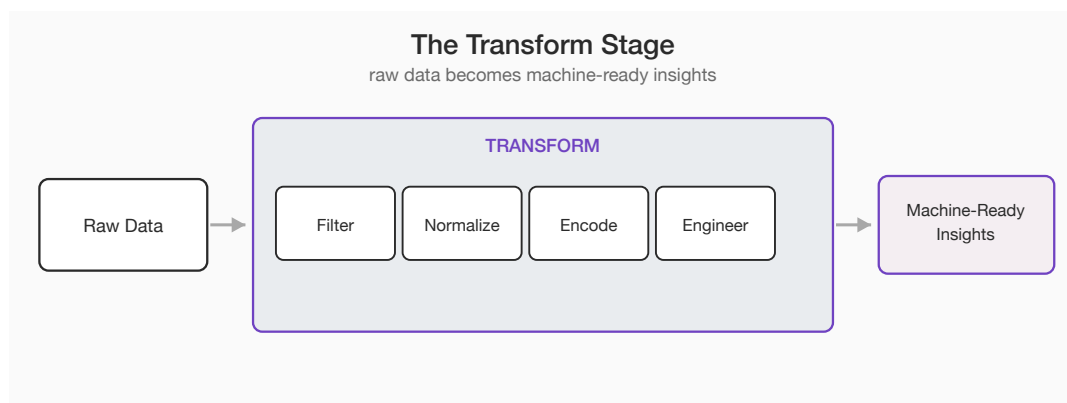


Figure 5.2: Raw data enters the Transform stage — filtered, normalized, encoded, and engineered — and comes out the other side as machine-ready insights.

Technique 1: Normalization

In datasets involving physical activity, columns like **Steps** (range: 0–15,000) and **HeartRate** (range: 60–200) exist on completely different scales. Without normalization, the “Steps” column would dominate any analysis simply because its numbers are larger, not because it is more important.

Normalization levels the playing field. It rescales numbers to a similar range—typically **0 to 1**.

The Formula

$$\text{Normalized Value} = \frac{\text{value} - \text{min}}{\text{max} - \text{min}}$$

This maps the smallest value to 0, the largest to 1, and everything else in between.

Reflective Code: Normalizing Activity Data

```
# Normalizing a sample step count
steps_min, steps_max = 0, 15000
alice_steps = 10500

steps_norm = (alice_steps - steps_min) / (steps_max - steps_min)
print(f"Normalized Steps: {steps_norm}")
```

Output:

Normalized Steps: 0.7

💡 Instructor Insight: Outlier Visibility

Once you normalize data, outliers become immediately visible. If Diana's heart rate of 108 maps to **1.0**, while the average sits at **0.2**, you instantly see the peak in her activity without needing to know the technical range of heart rates.

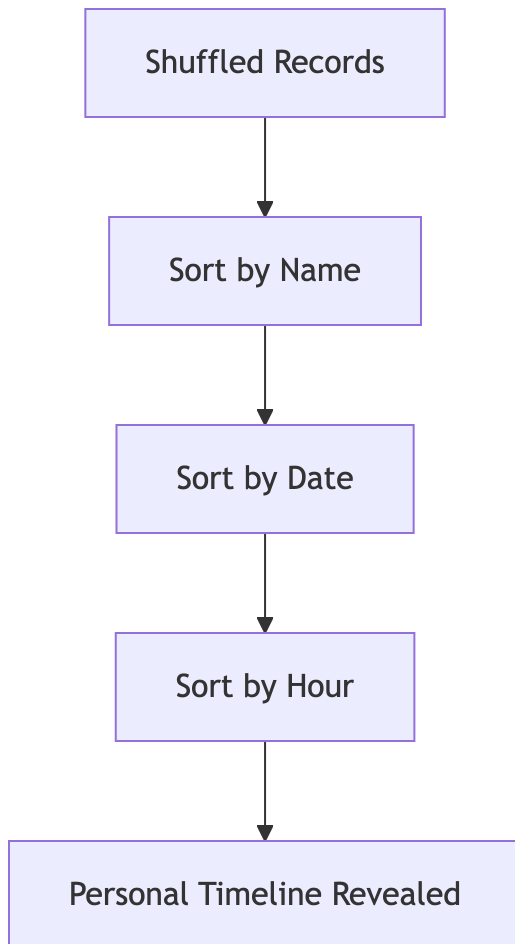
💡 Try With ChatGPT

Describe a column from your own dataset to ChatGPT (its name, minimum, and maximum values) and ask: *“Would normalizing this column to a 0-1 range make sense here? Why or why not?”*

Technique 2: Sequential Data Transformation

Raw datasets have no inherent order. To see a “story,” we must sort the data by time. By sorting by **Name**, **Date**, and **Hour**, we create a personal timeline for every individual.

From Chaos to Chronology



Reflective Code: Sorting the Timeline

```
import pandas as pd

# Creating a personal timeline from raw data
# Sorting reveals behavioral trends like "active in the evening"
df_sorted = df.sort_values(by=['Name', 'Date', 'Hour'])
print(df_sorted[['Name', 'Hour', 'steps_norm']].head())
```

What You'll See:

Data Discovery: Alice's steps jumped from 0.06 at 9 AM to 1.00 at 8 PM on July 30.

Technique 3: Geospatial Data Transformation

Raw latitude and longitude coordinates (e.g., 39.74, -75.54) are meaningless to most analysts and machine learning models. **Geospatial Transformation** is a classic example of **Feature Engineering**—creating new, meaningful columns from raw data.

The Tool: Haversine Formula

We use the Haversine formula to calculate the shortest distance between two points on the Earth's surface. This allows us to convert abstract coordinates into a human-readable feature: “Distance from School.”

Reflective Code: Feature Engineering (Distance)

```
# Converting coordinates to distance in km from a reference point
# Reference: School Campus (39.74, -75.54)
distance = calculate_haversine(lat, lon, school_lat, school_lon)
print(f"Feature Created: {distance} km from school")
```

What You'll See:

Individual Variation Found:

- Diana: 0.07 km (On Campus)
 - Charlie: 0.58 km (Off Campus)
-

Knowledge Check

Alice's heart rate normalizes to 1.0, while the group average normalizes to about 0.2. What does that tell you, and what doesn't it tell you?

Reveal Answer

It tells you Alice's heart rate reading was the highest in the dataset relative to everyone else's range — an outlier on the high end. It doesn't tell you her actual heart rate in beats per minute, since normalization strips out the original units and scale.

Summary: Shaping the Story

Normalization, sequential sorting, and geospatial transformation each solve a different technical problem, but together they represent a single core principle: **Data must be shaped to tell its story.**

1. **Normalize** to level the scale for fair comparison.
2. **Sequence** to order by time and reveal patterns.
3. **Geolocate** to convert raw coordinates into human meaning.
4. **Analyze** by feeding this high-quality, clean data into your models.

Part 4: Encoding, Merging, and the ETL Endgame

In this final stretch of the ETL journey, we transition from cleaning raw data to preparing it for machine learning. We explore how to translate human language into mathematics through encoding and how to distill high-frequency noise into actionable insights through data aggregation.

Teaching Machines to Speak Data

Raw data rarely speaks the language that machine learning algorithms understand. While humans see categories like “Resting” or “Cycling,” an algorithm sees only mathematical vectors. This tutorial covers the “Endgame” of the ETL process: **Encoding** categorical text into numeric features and **Merging** granular records into meaningful summaries. By mastering these final steps, you turn a collection of individual data points into a cohesive, analysis-ready information pipeline.

Learning Objectives

By the end of this tutorial, you will be able to:

- Distinguish between **Label Encoding** and **One-Hot Encoding**.
- Implement categorical translation using Python’s **pandas** library.
- Apply **Aggregation** logic to collapse redundant hourly records into daily signals.
- Synthesize the four pillars of the ETL workflow: Normalization, Sequential/Geospatial, Encoding, and Merging.

1. Encoding: The Language Bridge

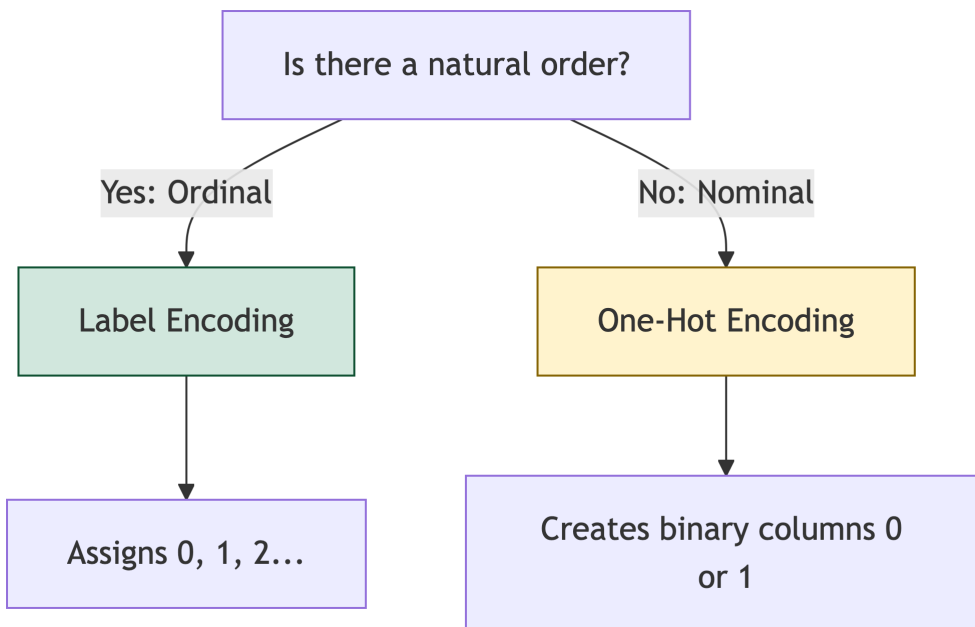
Algorithms do not speak English; they speak math. Encoding is the bridge that converts text labels into numbers a model can process. Without this step, even the richest categorical data remains invisible to a machine learning model.

i Encoding

Encoding is the process of converting categorical data (text-based labels) into numerical values. This ensures that non-numeric features can be used as inputs for mathematical models.

Choosing the Right Encoding Logic

The choice between encoding methods depends on whether your categories have a natural “ranking” or order.



2. Label Encoding: Simple and Efficient

Label Encoding assigns each unique category a unique integer (e.g., 0, 1, 2). It is fast and memory-efficient, making it ideal for datasets with a natural order, such as “Low,” “Medium,” and “High.”

Reflective Code: Label Encoding Gender

In our activity dataset, we can translate “Gender” into a numeric format. By default, this often follows alphabetical order.

```
import pandas as pd

# Simulating the creation of a 'Gender_code' column
# "Female" becomes 0, "Male" becomes 1
df['Gender_code'] = df['Gender'].astype('category').cat.codes

print(df[['Name', 'Gender', 'Gender_code']].head())
```

Output:

Encoding Verification:

- Alice (Female) -> 0
- Diana (Female) -> 0
- Charlie (Male) -> 1

[Status] Encoding consistent across all 216 rows.

3. One-Hot Encoding: Eliminating Bias

When categories carry no natural order—like “Cycling,” “Running,” and “Resting”—Label Encoding can be dangerous. If “Running” is assigned 2 and “Resting” is assigned 1, a model might incorrectly assume that Running is “greater than” or “twice as much” as Resting.

One-Hot Encoding solves this by expanding one column into several binary columns.

💡 Instructor Insight: Fair Predictions

One-Hot Encoding is the “safety first” choice for nominal data. It guarantees that no activity outranks another in the model’s eyes, which is critical for making fair and unbiased predictions.

Reflective Code: Applying `get_dummies()`

```
# Expanding 'ActivityType' into binary flags
df_encoded = pd.get_dummies(df, columns=['ActivityType'])

print(df_encoded.columns)
```

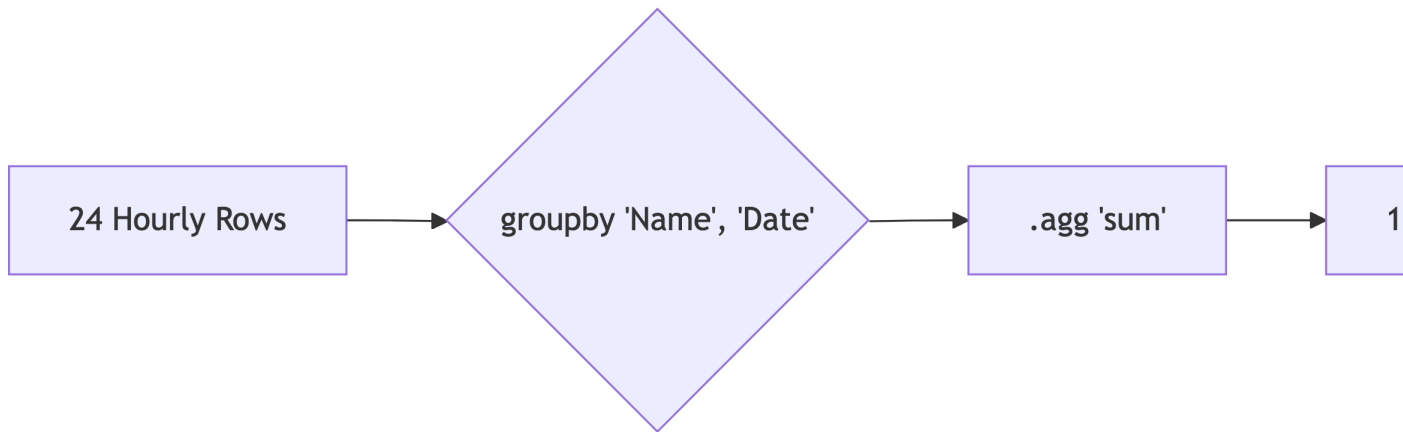
Output:

```
New Feature Columns Created:
- ActivityType_Cycling [0 or 1]
- ActivityType_Resting [0 or 1]
- ActivityType_Running [0 or 1]
In each row, exactly one column equals 1.
```

4. Merging: From Hourly Noise to Daily Signal

In our raw dataset, we have 24 entries per person per day. While granular, this “hourly noise” can obscure the bigger picture. **Merging (Aggregation)** collapses these redundant records into a single daily summary.

The Aggregation Workflow



Reflective Code: Extracting the Daily Signal

By grouping and summing, we reveal behavioral trends that raw records obscure.

```
# Aggregating hourly steps into daily totals
daily_summary = df.groupby(['Name', 'Date']).agg({'Steps': 'sum'}).reset_index()

print(daily_summary)
```

What You'll See:

Insights Revealed:

- Alice: Most Active (Peak: 21,025 steps on July 28)
- Diana: Rising Trend (7,900 -> 10,877 -> 18,608 steps)
- Charlie: Lower activity detected (Only 2 days recorded)

💡 Knowledge Check

Why is one-hot encoding safer than label encoding for a column like `ActivityType` (Cycling, Running, Resting)?

Reveal Answer

Label encoding assigns arbitrary numbers (e.g., Cycling=0, Resting=1, Run-

ning=2), which implies a false order or magnitude a model might learn from. One-hot encoding creates separate binary columns so no category outranks another.

The ETL Endgame: One Pipeline

You have now explored the complete ETL workflow. Mastering the logic behind each step is what separates a skilled data engineer from someone who just runs code.

1. **Normalization:** Scale numeric values to a common range.
2. **Sequential & Geospatial:** Transform time and location into context.
3. **Encoding:** Convert categories into numeric features for the machine.
4. **Merging:** Aggregate records into actionable summaries.

Instructor Insight: Experiment Boldly

The real power of ETL comes from knowing **when** and **why** to apply each technique. Don't just run the code—audit your dataset. Ask yourself: *Will this transformation reveal a hidden signal?*

Part 5: The Load Phase

Every data science project lives or dies by the quality of its pipeline. In this final session, we close the loop on the ETL process—Extract, Transform, and Load—by learning how to save your processed results so they can be shared, reused, and built upon. This is the stage where raw data finally becomes a permanent asset.

Learning Objectives

By the end of this tutorial, you will be able to:

- Define the **Load** phase and its role in the data pipeline.
 - Distinguish between saving to a CSV and writing to a database.
 - Implement the `.to_csv()` method in Pandas to export cleaned data.
 - Understand the importance of the `index=False` argument for professional output.
 - Synthesize the full ETL journey from raw input to production-ready export.
-

What is the Load Phase?

After the intensive work of extracting raw data and transforming it into something meaningful, the **Load** step serves as your finish line. It is the act of publishing your work—taking the data out of the temporary memory of a notebook and making it accessible to other people and systems.

The Load Phase

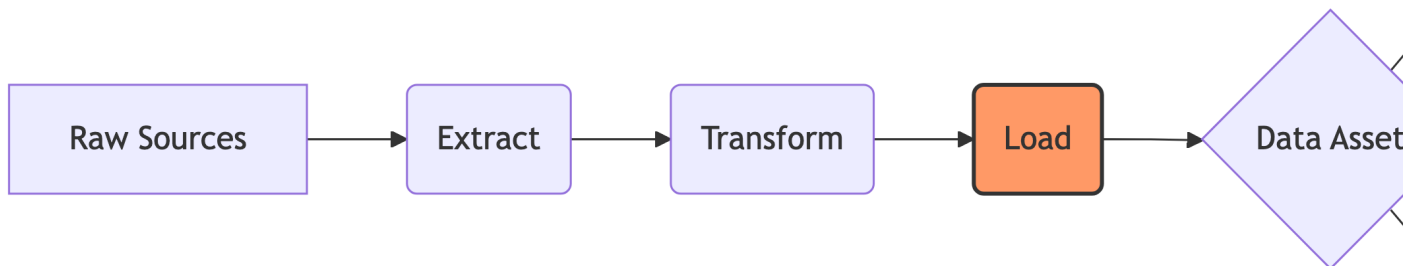
The **Load** phase is the final stage of ETL where transformed data is saved to a permanent destination, ensuring it is preserved for future analysis or application.

Common Storage Destinations

Depending on the scale and portable needs of your project, you typically choose between two main destinations:

- **Save to CSV:** A portable, lightweight file format that anyone can open with basic spreadsheet software.
- **Write to a Database:** Structured storage designed for querying data at scale across large organizations.

The Data Life Cycle



Hands-On: Saving to CSV with Pandas

The `.to_csv()` method is one of the most frequently used tools in a data scientist's toolkit. It allows you to move your final `DataFrame` into a shared environment with just a few parameters.

The Saving Workflow

To save your work professionally, follow these three steps:

1. **Invoke the method:** Call `.to_csv()` on your final transformed DataFrame.
2. **Name your file:** Provide a descriptive string (e.g., `"aggregated_daily_activity.csv"`).
3. **Set `index=False`:** Ensure your output is clean by removing unnecessary row numbers.

Reflective Code: The Final Export

```
# Assuming aggregated_df is your final, transformed dataset
# We name the file and strip the index for a clean output
aggregated_df.to_csv("aggregated_daily_activity.csv", index=False)
```

What You'll See:

Success: 'aggregated_daily_activity.csv' generated in local directory.

 **Instructor Insight: Don't Forget the Index!**

The `index=False` argument is a small but critical detail. Without it, Pandas adds an extra numbered column (0, 1, 2...) to your CSV. This clutters your output file and can cause errors when other systems try to read your data later.

 **Knowledge Check**

What does `index=False` do in `aggregated_df.to_csv("file.csv", index=False)`, and why does it matter?

Reveal Answer

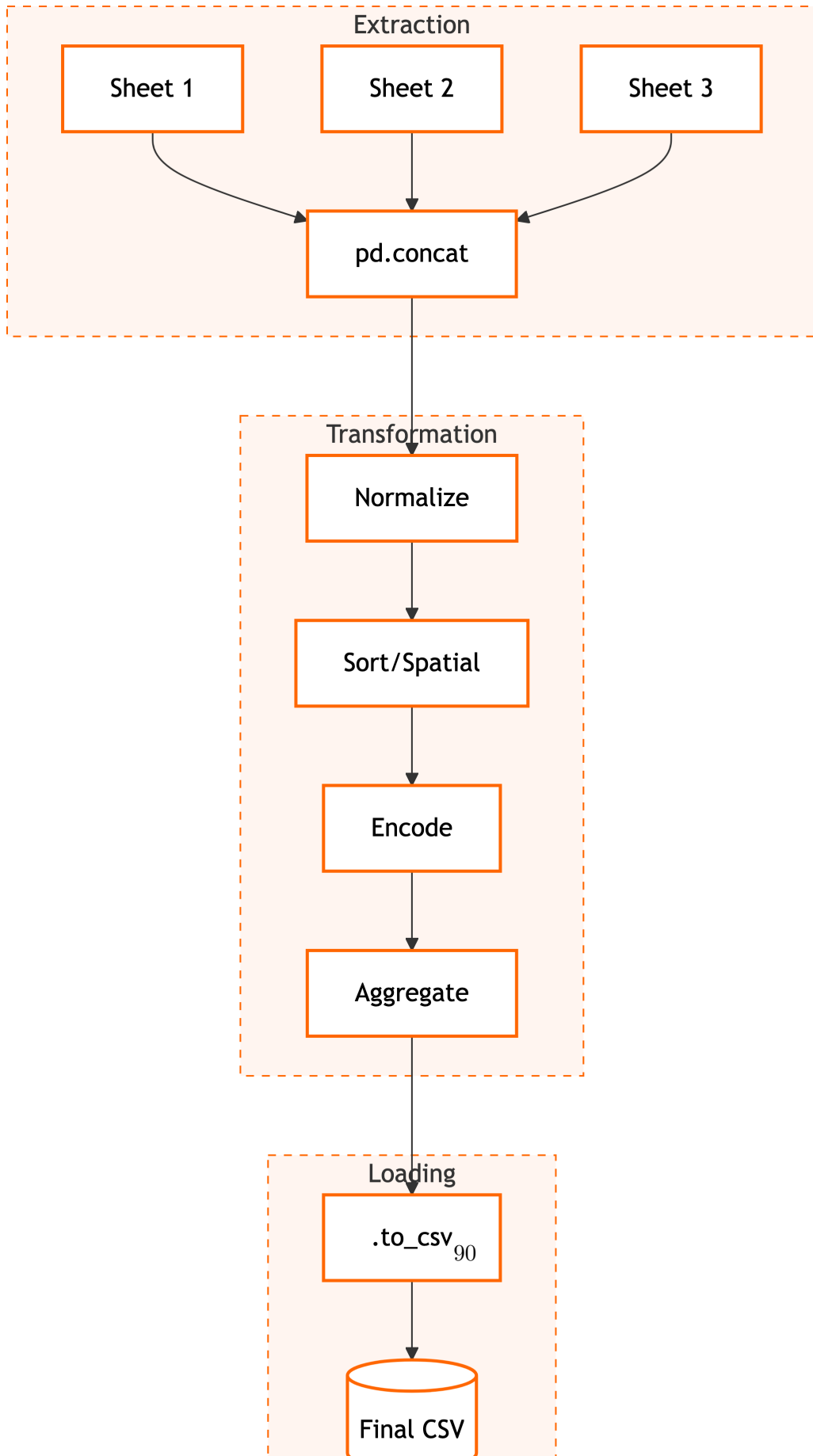
It tells pandas not to write the DataFrame's row-number index as an extra column in the CSV. Without it, the exported file gets a meaningless extra column that can confuse other programs or people reading the file.

The Full Journey: ETL from Start to Finish

Saving your file is a major milestone—it signifies that your data pipeline works end-to-end. Let's zoom out and look at the cohesive pipeline we built using real-world activity data:

1. **Extract:** We loaded three separate Excel sheets and unified them into a single master DataFrame.
2. **Transform:** We applied five key techniques to make the data meaningful:
 - **Normalization:** Scaled numbers for fair comparison across different units.
 - **Sequential:** Ordered data chronologically to reveal behavioral patterns over time.
 - **Geospatial:** Converted abstract coordinates into human-readable distances.
 - **Encoding:** Translated text categories (Gender, Activity) into numbers (Label & One-Hot).
 - **Merge:** Aggregated hourly noise into clean daily summaries.
3. **Load:** We exported the final dataset to a `.csv` file for future use.

Pipeline Architecture



Key Takeaways

- **Data is never ready out of the box:** Real-world data is messy; ETL is the intentional process that makes it usable for every model and dashboard.
- **Every step has a purpose:** Extraction brings data in, Transformation makes it meaningful, and Loading makes it permanent and shareable.
- **Production-Ready Thinking:** Moving from raw Excel sheets to a structured CSV is the hallmark of professional data science thinking.

6 Where Data Comes From: APIs & Real-World Sources

In [Chapter 5](#), you mastered the **Extract** phase of ETL by pulling data out of flat files — a multi-sheet Excel workbook that was already sitting on your computer. But professional data scientists rarely get their data hand-delivered as a tidy file. More often, the data lives on *someone else's computer*, and you have to go ask for it. This chapter teaches you how to do exactly that.

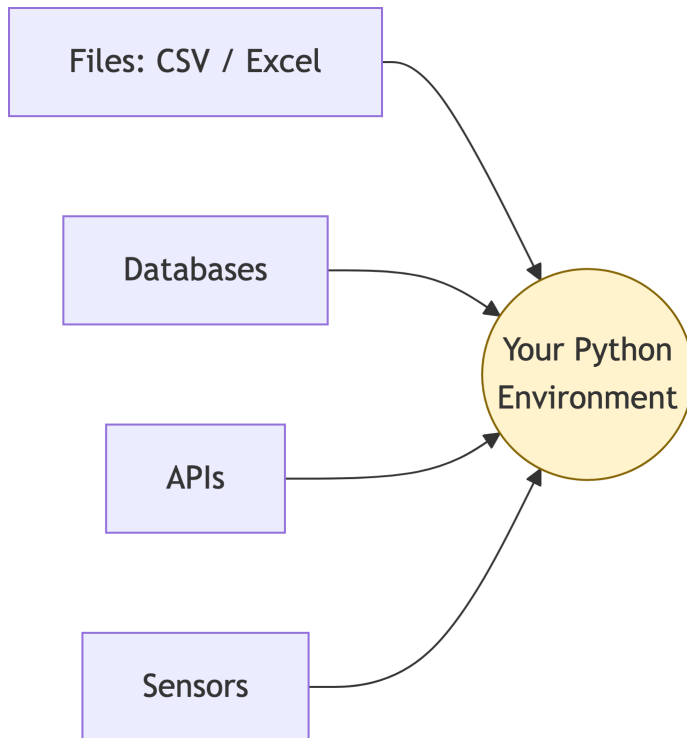
6.1 Learning Objectives

By the end of this chapter, you will be able to:

- Describe the four major channels through which data reaches a data scientist: files, databases, APIs, and sensors.
 - Explain what an API is and why it is the standard way programs share live data.
 - Read a JSON response and map its structure onto Python dictionaries and lists.
 - Use the Python `requests` library to call a real, free public API and extract specific fields from the response.
 - Judge when it is appropriate — ethically and legally — to extract data from a web source.
-

6.2 Four Ways Data Reaches You

Think back to the “Extraction Web” diagram from Chapter 5: websites, flat files, databases, and APIs all feeding into your Python environment. Let’s slow down and look at each channel properly, plus one more that is quietly becoming the biggest data source on Earth.



Files. This is what Chapter 5 covered: CSVs, Excel workbooks, and other flat files that you download once and load with `pd.read_csv()` or `pd.read_excel()`. Files are simple and portable, but they are frozen snapshots — the moment a file is saved, it starts going stale.

Databases. When an organization has too much data to fit in a spreadsheet — millions of customer orders, years of hospital records — it stores that data in a **database**: structured, indexed, and queryable with a language called SQL. Databases are the workhorses of industry, but this book does not go deep on them. For now, just know they exist, and that pandas can talk to them when the time comes.

APIs. An API lets your program request *live, on-demand* data from another computer over the internet. Want the temperature in Dover right now, the current position of the International Space Station, or the latest exchange rates? You don't download a file — you ask an API, and it answers in real time. This is the star of this chapter.

Sensors. Fitness trackers, weather stations, traffic cameras, smart thermostats — physical devices that generate a *constant stream* of measurements. The simulated activity dataset you unified in Chapter 5 (steps, heart rate, GPS) was designed to mimic exactly this kind of sensor output. Sensor data usually reaches you *through* one of the other three channels: it gets logged to files, stored in databases, or served up by an API.

6.3 What Is an API?

i API

An **API** (Application Programming Interface) is a way for one program to ask another program for data — usually over the internet — and get back a structured answer. The requesting program sends a specially formatted URL; the serving program replies with machine-readable data instead of a human-readable web page.

The classic analogy: an API is the **waiter** in a restaurant. You (the customer) don't walk into the kitchen and start opening refrigerators — the kitchen would (rightly) throw you out. Instead, you look at the **menu** (the API's documentation, which lists exactly what you're allowed to ask for), give your order to the waiter in a standard format ("one forecast for latitude 39.16, please"), and the waiter brings back exactly what you ordered, plated in a predictable way.

This arrangement is good for everyone. The kitchen keeps control of its ingredients and processes; you get your meal without needing to know how the stove works. Likewise, a weather service keeps control of its supercomputers and forecast models, while you get clean, structured data with three lines of Python.

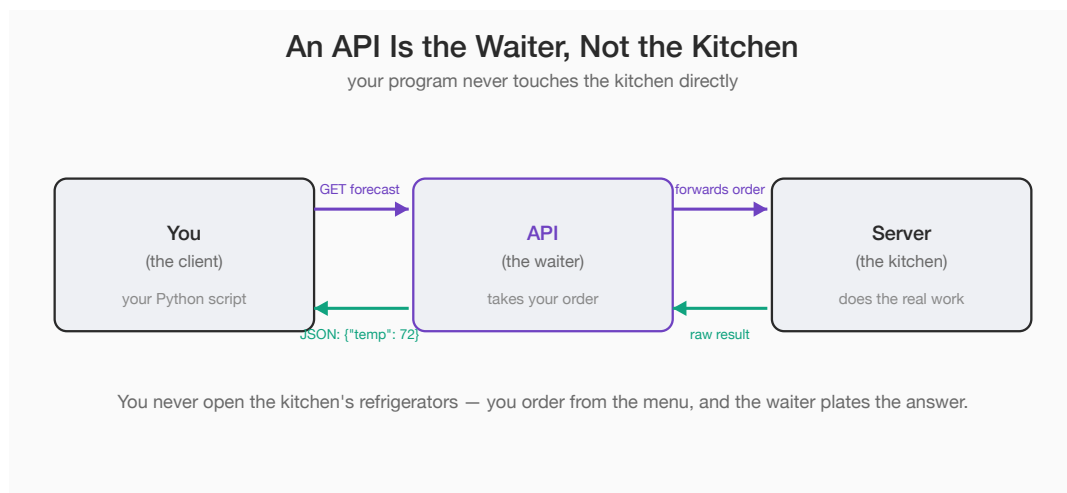


Figure 6.1: The client sends a request through the API and gets a response back — it never touches the server's internals directly.

For this chapter's hands-on work we will use **Open-Meteo** (<https://open-meteo.com>), a genuinely free weather API: no account, no sign-up, no API key. It serves real forecast data for any point on the planet.

6.4 JSON: The Language APIs Speak

When a website answers your *browser*, it sends HTML — text designed for humans to read. When an API answers your *program*, it almost always sends JSON.

JSON

JSON (JavaScript Object Notation) is a text format for structured data. Despite the “JavaScript” in its name, every programming language can read it — and to a Python programmer, JSON looks almost exactly like nested **dictionaries** (curly braces, “**key**”: **value** pairs) and **lists** (square brackets).

Here is the actual JSON that Open-Meteo returns when asked for the current weather in Dover, Delaware:

```
{
  "latitude": 39.169113,
  "longitude": -75.51602,
  "elevation": 6.0,
  "timezone": "GMT",
  "current_weather_units": {
    "temperature": "°C",
    "windspeed": "km/h",
    "winddirection": "°"
  },
  "current_weather": {
    "time": "2026-08-28T02:15",
    "temperature": 23.6,
    "windspeed": 9.6,
    "winddirection": 146,
    "is_day": 0,
    "weathercode": 3
  }
}
```

Let’s decode a few of these fields in plain English:

- **latitude** / **longitude** — the coordinates the forecast is *actually* for. Notice they aren’t exactly what we asked for (39.16, −75.52): the API snapped our request to the nearest point on its weather-model grid. Real data sources make small, honest adjustments like this all the time.

- **current_weather** — a nested object (an inner dictionary!) holding the actual measurements: 23.6 degrees, wind at 9.6 km/h. The `is_day: 0` field means it was nighttime when this response was captured.
- **current_weather_units** — a matching dictionary that tells you the *units* for each measurement. Never assume units; a temperature of 23.6 means something very different in °C than in °F.

Notice the nesting: a dictionary inside a dictionary. That is JSON's superpower — it can represent hierarchies that a flat CSV cannot.

💡 Knowledge Check

In the JSON above, is **current_weather** a single value, a list, or something else? And how would you describe the overall structure of this response to a classmate who only knows Python?

Reveal Answer

current_weather is a **nested object** — a dictionary inside the outer dictionary. The whole response is one big Python-style dictionary whose values are a mix of numbers (**latitude**), strings (**timezone**), and inner dictionaries (**current_weather**, **current_weather_units**).

6.5 Hands-On: Calling Your First API

In Chapter 5, extraction meant `pd.read_excel()`. Extracting from an API takes a different library — **requests** — but the spirit is identical: pull raw data into Python so you can work with it.

```
import requests

response = requests.get(
    "https://api.open-meteo.com/v1/forecast"
    "?latitude=39.16&longitude=-75.52&current_weather=true"
)
data = response.json()
print(data)
```

Output:

```
{'latitude': 39.169113, 'longitude': -75.51602,
 'generationtime_ms': 0.259, 'utc_offset_seconds': 0,
 'timezone': 'GMT', 'timezone_abbreviation': 'GMT', 'elevation': 6.0,
 'current_weather_units': {'time': 'iso8601', 'interval': 'seconds',
 'temperature': '°C', 'windspeed': 'km/h', 'winddirection': '°',
 'is_day': '', 'weathercode': 'wmo code'},
 'current_weather': {'time': '2026-08-28T02:15', 'interval': 900,
 'temperature': 23.6, 'windspeed': 9.6, 'winddirection': 146,
 'is_day': 0, 'weathercode': 3}}
```

(Your numbers will differ — this is *live* weather. That’s the whole point.)

Three lines did all the work:

1. `requests.get(url)` sends the request across the internet — the waiter carries your order to the kitchen.
2. `response.json()` parses the JSON reply into a regular **Python dictionary**.
3. From there, it’s just Python. No new syntax needed:

```
weather = data["current_weather"]
print("Temperature:", weather["temperature"], "°C")
print("Wind speed:", weather["windspeed"], "km/h")
```

Output:

```
Temperature: 23.6 °C
Wind speed: 9.6 km/h
```

The `["key"]` lookups chain naturally: `data["current_weather"]["temperature"]` reaches two levels deep in one line, exactly like any nested dictionary you’ve used before.

6.6 Two Words Every API User Needs

i Endpoint

An **endpoint** is one specific URL (or URL pattern) that an API exposes for one specific kind of data. Open-Meteo's `/v1/forecast` endpoint serves weather forecasts; its `/v1/air-quality` endpoint serves air-quality data. One API, many endpoints — like one restaurant menu with separate sections for entrées and desserts.

The part of the URL after the `?` is the **query string**: `latitude=39.16&longitude=-75.52¤t_weather=true` is how you fill in the details of your order. Change the coordinates, and the same endpoint serves you weather for Tokyo or São Paulo.

i Rate Limit

A **rate limit** is a cap on how many requests you may send an API in a given time window (for example, Open-Meteo's free tier allows up to 10,000 requests per day). Exceed it and the API starts refusing you.

Rate limits matter for two reasons. First, **politeness**: a free service is donating computing power to you; a runaway `while True: loop` hammering it a thousand times a second is the programming equivalent of ordering everything on the menu and eating none of it. Second, **self-preservation**: services block abusers, sometimes permanently. If you ever need many requests, add a pause (`time.sleep(1)`) between them and cache what you've already fetched.

6.7 Is It OK to Take This Data?

Just because your code *can* fetch something doesn't mean you *should*. Here is the practical dividing line:

- **Documented public APIs — yes.** When a service like Open-Meteo publishes documentation, examples, and usage limits, it is explicitly inviting programs to request its data. Using it as documented, within its rate limits, is exactly what it's for.
- **Scraping a site with no API — slow down.** “Scraping” means writing code that downloads a site's human-facing web pages and picks data out of the HTML. Some sites tolerate it; many explicitly forbid it in their **terms of service**, and doing it anyway can violate those terms, overload the site, or misuse people's personal information.

Before you ever write a scraper, do two concrete checks:

1. **Read the site’s terms of service** (usually linked in the page footer) and search for words like “scrape,” “crawl,” “automated,” or “bot.”
2. **Check the site’s robots.txt** — a public file at the site’s root (e.g., <https://example.com/robots.txt>) where the site declares which pages automated programs may and may not visit. A **Disallow:** line is the site telling bots to stay out.

And the golden rule: **if an official API exists, use it instead of scraping.** It’s more reliable, more respectful, and the data arrives as clean JSON instead of tangled HTML.

Knowledge Check

Your friend wants hourly prices from an online store for a class project. The store has no public API, and its terms of service say “automated access is prohibited.” Your friend says, “It’s just for school, so it’s fine.” What would you tell them, and what alternatives could you suggest?

Reveal Answer

“Just for school” doesn’t override the terms of service — the prohibition applies to *any* automated access, and the scraper could get their IP blocked or worse. Better alternatives: look for an official API from the store or a comparable retailer, search Kaggle or an open data portal for an existing price dataset, or email the site to ask permission. Checking the terms of service and **robots.txt** *first* is exactly the habit that separates responsible data scientists from reckless ones.

Try With ChatGPT

Call an API (or find any raw JSON online), paste a confusing chunk of it into ChatGPT, and ask: “*Here is a JSON response from an API. Explain what each field probably means and what units it might use.*” For example, try the Open-Meteo response above and ask what **weathercode: 3** and **interval: 900** mean. Then verify ChatGPT’s guesses against the official documentation at <https://open-meteo.com/en/docs> — did it get everything right?

6.8 Your Turn

1. Call Open-Meteo from Python for a location you care about — your town, a city you'd love to visit, or the middle of the Pacific Ocean. Change the `latitude` and `longitude` in the query string. (Or pick another free, no-key API, such as the ISS position tracker at <http://api.open-notify.org/iss-now.json>.)
 2. Print **three specific fields** from the response — not the whole dictionary. For example: temperature, wind speed, and whether it's currently daytime there.
 3. Write **two sentences** about what this data could be used for. Who would want a live feed of it, and what decision could it help them make?
-

6.9 Roadmap: From Raw Data to Understanding

You now command the full extraction toolkit: files (Chapter 5), and live APIs (this chapter), with databases and sensors on your radar. That completes Unit II — you can *get* data and *clean* it. But a dictionary full of temperatures is not yet knowledge. In Unit III, we turn to what the numbers actually *mean*: summarizing data with statistics, questioning it with an ethical eye, and finally teaching machines to find patterns in it and make predictions. The data you learned to fetch today is the fuel for everything that comes next.

Part III

Unit III: Understanding and Modeling Data

7 Descriptive Statistics & Data Ethics

7.1 Learning Objectives

By the end of this chapter, you will be able to:

- Compute the mean, median, and mode of a small dataset by hand, and explain when each one is the best “middle” to report.
- Show, with actual numbers, how a single outlier can drag the mean while barely touching the median.
- Explain why two datasets with the same average can still be wildly different, and why *spread* matters as much as the middle.
- Tell the difference between correlation and causation — and spot news headlines that confuse the two.
- Describe how algorithmic bias happens (“bias in, bias out”) and list practical steps for protecting your own data privacy.

7.2 The Number That Lies While Telling the Truth

Back in Chapter 1, we ended with an uncomfortable idea: [every dataset has a point of view](#). We promised that in Chapter 7 we’d go deeper — into how averages can deceive, how biased data leads to unfair decisions, and what responsible data scientists do about it. Welcome to Chapter 7. Time to pay up.

Here’s the twist that makes this chapter fun: a statistic can be **completely accurate and still totally misleading**. Nobody has to fake a number. Nobody has to lie. You just pick the *technically true* number that tells the story you want. By the end of this chapter, you’ll be able to spot that trick from across the room — and, just as important, avoid pulling it by accident yourself.

7.3 Three Ways to Describe the Middle

Imagine your teacher hands back a quiz and someone asks the eternal question: “*What did the class get?*” Seven students took the quiz. Here are their scores:

Student	Score
Ava	82
Ben	85
Carlos	88
Dana	88
Elena	91
Finn	94
Grace	32

Six students scored in the 80s and low 90s. Grace was out sick all week, took the quiz cold, and got a 32. Keep an eye on that 32 — it’s about to cause trouble.

There are three standard ways to answer “what’s the middle of this data?” — and, as you’re about to see, they can disagree with each other *while all being correct*.

7.3.1 The Mean: Everybody Into the Pool

i Mean

The **mean** (what most people call the “average”) is the sum of all the values divided by how many values there are. It uses every single data point — which is both its superpower and its weakness.

Let’s compute it for our quiz:

$$\text{Sum: } 82 + 85 + 88 + 88 + 91 + 94 + 32 = 560$$

$$\text{Mean: } 560 \div 7 = 80$$

The mean is **80**. Technically true. But look back at the table — does 80 *feel* like a fair summary? Six of the seven students beat 80. Not one student actually scored an 80. Something is off.

Here’s what’s off: the 32. Watch what happens if we compute the mean *without* Grace’s outlier score:

$$\text{Sum: } 82 + 85 + 88 + 88 + 91 + 94 = 528$$

$$\text{Mean: } 528 \div 6 = 88$$

One single score dragged the class mean from 88 down to 80 — a full 8 points. That’s the mean’s weakness: because it uses every value, one extreme value (an **outlier**) can yank it around like a dog on a leash. The mean got pulled 8 points *toward* the outlier, away from where most of the class actually landed.

7.3.2 The Median: The One in the Middle

i Median

The **median** is the middle value when the data is sorted from smallest to largest. Half the values sit below it, half above. (If there's an even number of values, the median is the mean of the two middle ones.) Because it only cares about *position*, the median barely notices outliers.

Sort the seven scores and find the one in the middle:

32, 82, 85, [88], 88, 91, 94
 ^^
 3 below · 3 above

The median is **88** — right in the heart of where most students scored. And here's the beautiful part: it doesn't matter whether Grace scored 32, 52, or 2. As long as her score is the lowest, the middle value of the sorted list stays exactly where it is. The outlier that dragged the mean down 8 points **moved the median not at all**.

Side by side:

Measure	With the 32	Without the 32
Mean	80	88
Median	88	88

This is why you'll often hear about *median* household income or *median* home prices in the news. In any dataset where a few extreme values exist — one billionaire in a neighborhood, one mansion on a street of ordinary houses — the median tells you about the *typical* case, while the mean gets dragged toward the extremes.

7.3.3 The Mode: The Crowd Favorite

i Mode

The **mode** is the value that appears most often in a dataset. It's the only measure of the middle that works on *categorical* data — data made of labels and categories instead of numbers.

In our quiz scores, 88 appears twice (Carlos and Dana) and every other score appears once, so the mode is **88**.

For quiz scores, the mode is honestly the least interesting of the three. But now try this dataset — the same seven students’ favorite subjects:

Student	Favorite Subject
Ava	Art
Ben	Science
Carlos	Math
Dana	Science
Elena	English
Finn	Science
Grace	Math

What’s the *mean* favorite subject? Go ahead, add up “Art + Science + Math” and divide by 7. You can’t — the question doesn’t even make sense. You can’t sort subjects from smallest to largest either, so the median is out too. But the mode works perfectly: **Science** appears three times, more than any other subject, so Science is the mode. When your data is categories — favorite subjects, most common shoe size sold, most-streamed artist — the mode is the only middle you’ve got.

Knowledge Check

A real-estate website says the “average” home price on Maple Street is \$500,000. You visit and find five houses: four nearly identical homes that each cost about \$200,000... and one enormous mansion that cost \$1,700,000. Check the website’s math — is the \$500,000 figure accurate? Is it *honest*? Which measure would better describe a typical Maple Street home, and what is its value?

Reveal Answer

The math checks out: $(200,000 \times 4 + 1,700,000) \div 5 = 2,500,000 \div 5 = \textbf{\$500,000}$. The mean is perfectly *accurate* — and badly misleading, because not a single house on the street costs anywhere near \$500,000. The mansion is an outlier dragging the mean up, just like Grace’s 32 dragged the quiz mean down. The **median** is the better summary here: sorted, the prices are 200,000, 200,000, [200,000], 200,000, 1,700,000, so the median is **\\$200,000** — exactly what a typical house actually costs.

7.4 Spread: How Different Are the Numbers From Each Other?

Here’s a puzzle. Two classes take the same test:

- **Class A:** 78, 79, 80, 81, 82
- **Class B:** 60, 70, 80, 90, 100

Compute the mean of each (go on, they're quick): Class A is $400 \div 5 = \mathbf{80}$. Class B is $400 \div 5 = \mathbf{80}$. Identical means. Same median too — 80 in both.

But would you say these classes had the same experience? Not even close. In Class A, everyone landed within two points of the mean — the class moved as a pack. In Class B, one student aced it, one student is having a very bad day, and the mean of 80 describes a “typical student” who barely exists. The middle is the same; the **spread** is completely different.

Spread — how far the values scatter from the center — is the other half of describing data, and it's the half people forget. A summary that gives you only the middle is like a weather forecast that gives you only the average temperature for the year: technically informative, useless for deciding what to wear. Any time someone hands you an average, your next question should be: “*Okay — but how spread out is it?*”

Statistician Francis Anscombe made this point unforgettable in 1973 with four datasets now called **Anscombe's Quartet**. Each of the four has the *exact same* mean, the *exact same* spread, and the *exact same* correlation — but plot them, and you see four completely different pictures:

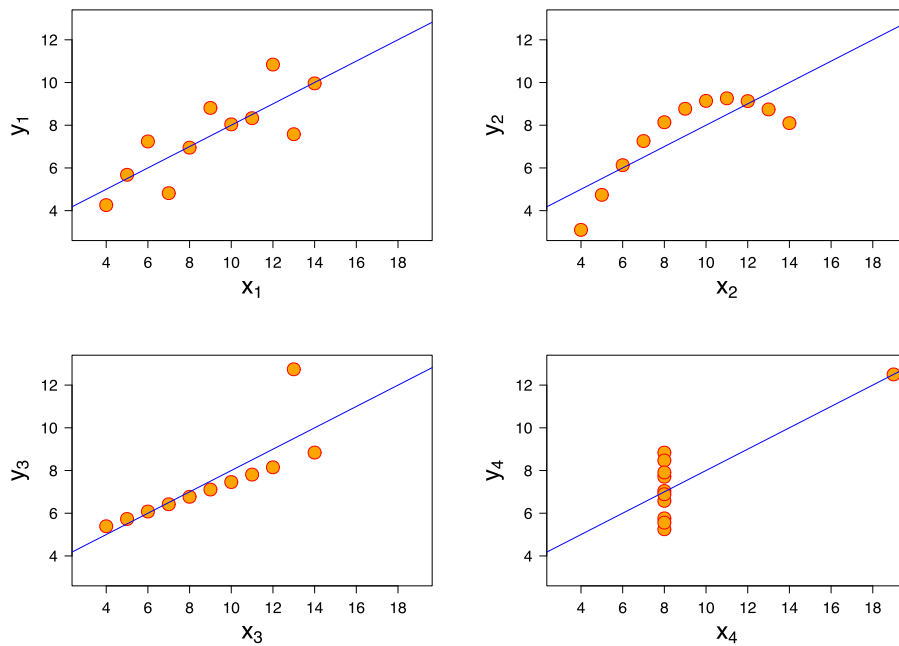


Figure 7.1: Anscombe's Quartet: four datasets with identical summary statistics (mean, spread, and correlation), but visibly different shapes when plotted.

Same numbers on paper, four different stories in the data. That's the whole lesson of this section, made visible: **never trust a summary statistic you haven't plotted.**

Statisticians have precise tools for measuring spread (with names like *standard deviation* — you'll meet them in a future stats course), but the intuition is what matters here: **two datasets can share the same middle and still tell completely different stories.** Tuck this idea away somewhere safe, because in Chapter 8 we'll use a closely related measure — called **Mean Squared Error** — to describe how spread out a *model's mistakes* are, which turns out to be exactly how data scientists decide whether a prediction machine is any good.

7.5 Correlation Is Not Causation

So far we've described one column of numbers at a time. But the questions that get data scientists out of bed in the morning involve *two* things at once: Does more studying lead to better grades? Does more screen time mean worse sleep? Questions like these are about **relationships** — and there's a famous trap waiting inside them.

i Correlation

Correlation means two things tend to move together: when one goes up, the other tends to go up (or reliably down). Correlation describes a *pattern* you can see in the data. It says nothing about *why* the pattern exists.

i Causation

Causation means one thing actually *makes* the other happen — changing the first genuinely changes the second. Causation is a claim about how the world *works*, and it's much, much harder to prove than correlation.

The trap: our brains are pattern-hungry, and the instant we spot a correlation, we itch to declare a cause. Here's the classic example of why that itch must be resisted.

Ice cream and drowning. If you chart monthly ice cream sales against monthly drowning deaths, you'll find they rise and fall *together*, year after year. A real correlation, visible in real data. So — does ice cream cause drowning? Should lifeguards confiscate cones at the beach? Of course not. Both numbers are being pushed up by a third thing: **summer**. Hot weather sends people to buy ice cream, and hot weather sends people swimming — and more swimming means, tragically, more drownings. Ice cream and drowning are correlated, but neither causes the other. Data scientists call that hidden third factor a *lurking variable*, and it's behind a huge share of the world's misleading statistics.

Now one for you to evaluate yourself:

Fire trucks and damage. Analyze a city’s fire department records and you’ll find a strong correlation: the more fire trucks dispatched to a fire, the more damage that fire causes. The pattern is real and strong. So, by the logic of “correlation = causation,” the city could reduce fire damage by... sending fewer fire trucks?

Stop and work it out before reading on. What’s really going on here?

You probably got it: **big fires** cause both. A serious fire makes the department send more trucks, *and* a serious fire causes more damage. The trucks don’t cause the damage — the size of the fire causes both numbers to rise together. (In fact, the causation runs *toward* the trucks, not from them.) Cutting trucks wouldn’t reduce damage; it would just mean big fires get fought by too few firefighters.

From now on, every time a headline says “Study finds people who do X have more Y,” run the checklist: Could Y be causing X instead of the other way around? Could some third thing — a summer, a big fire — be causing both? A correlation is a *clue*, never a verdict.



Try With ChatGPT

Prompt: “I’m a high school student learning the difference between correlation and causation. Give me three real-world examples where two things are correlated but neither causes the other, and for each one, tell me what the hidden third factor (lurking variable) is. Don’t use the ice cream and drowning example — I already know that one.”

Then comes the important part: **pick ChatGPT’s most interesting example and verify it.** Search for the claim online — is it a documented real correlation, or something that just *sounds* plausible? Does the lurking variable ChatGPT named actually explain it? AI tools sometimes produce examples that are made up or subtly wrong, which is exactly why [Using AI Responsibly](#) tells you to always verify. Write two sentences on whether the example held up.

7.6 Data Has a Point of View — Revisited

In [Chapter 1](#), we met a step counter that read *zero* all day for a wheelchair user — an honest device faithfully collecting biased data, because of a choice about *what to measure*. Back then, the stakes were one misleading fitness log. Now raise the stakes: what happens when biased data gets fed into an **algorithm** that makes decisions about people automatically, thousands of times a day?



Algorithmic Bias

Algorithmic bias happens when a computer system produces systematically unfair results because the data it learned from contained bias. The algorithm isn’t malicious —

it's *obedient*. It faithfully learns whatever patterns are in its training data, including the unfair ones. Data scientists summarize it in four words: **bias in, bias out**.

Here's a concrete, realistic example of how this happens — no villains required.

Imagine a company gets 10,000 job applications a year, far too many to read. So it builds a program to help: feed the program the last ten years of the company's hiring decisions — which résumés got interviews, which got rejected — and let it learn the patterns of a “successful applicant.” Sensible, right? The program is even designed carefully: it is never told any applicant's gender, race, or age.

But suppose that over those ten years, the humans doing the hiring had a bias — say, they consistently favored applicants from a handful of familiar neighborhoods and schools, and overlooked equally qualified people from elsewhere. That bias is now *baked into the training data*. The program studies the data and learns, with perfect accuracy, the pattern it was shown: “résumés like these get hired; résumés like those don't.” It starts scoring new applicants the same way. The old bias didn't disappear when the humans handed the job to a machine — it got **automated**, applied faster, at larger scale, and with a new disguise: “*it's not a person being unfair, it's just math.*”

And here's the detail that should make the hair on your neck stand up, because it connects everything in this chapter: hiding the sensitive information didn't help. The program never saw race, gender, or age — but other fields it *did* see, like zip code, school name, or even hobbies, can be **correlated** with them. The algorithm doesn't need to be told what it isn't supposed to know; correlation smuggles it in through the side door. The correlation trap from the last section isn't just a way to misread headlines — it's the exact mechanism by which “neutral” algorithms end up biased.

The fix is not “never use algorithms.” Algorithms grade essays, recommend songs, and spot diseases in medical scans — used well, they're genuinely great. The fix is what responsible data scientists actually do: **audit the training data** for the same blind spots you learned to hunt in Chapter 1 (*who* was included, *what* was measured, *what's* missing), **test the system's results** across different groups of people to catch unfairness before launch, and **keep humans responsible** for decisions that seriously affect people's lives. An algorithm can process a million rows; it cannot care about a single one of them. Caring remains your job.

Knowledge Check

A hospital builds a program to predict which patients will get the flu, so it can offer them vaccines early. It trains the program on ten years of the hospital's own patient records. A data scientist warns: “This model will underestimate flu risk for people who rarely visit the hospital.” The hospital replies, “Impossible — our records are accurate. Every diagnosis in the data really happened.” Who is right, and why?

Reveal Answer

The data scientist is right — and notice that *both* statements can be true at once. The records can be perfectly accurate and still biased, because they only contain people who *came to the hospital*. Someone who gets the flu but toughs it out at home (maybe they lack insurance, transportation, or time off work) appears in the data as a person who “never gets the flu.” The model learns that pattern and scores similar people as low-risk — so the very people least connected to healthcare get the fewest early vaccine offers. That’s algorithmic bias from a blind spot in data collection: bias in, bias out. No inaccurate numbers required.

7.7 Your Data, Your Privacy

There’s one more person in the data ethics story we haven’t talked about yet: **you** — not as the data scientist this time, but as the *data*.

Data Privacy

Data privacy is your ability to control information about yourself — what gets collected, who can see it, how it’s used, and how long it’s kept. It rests on **consent**: the idea that companies should tell you what they collect and get your agreement before collecting it.

Think back to Chapter 1, when you logged your own day for 24 hours. It took real effort — noticing, remembering, writing things down. Meanwhile, the apps on a typical phone were logging away effortlessly, automatically, and in far more detail: not just *which* apps were opened, but for how long, at what hour, in what location, after which notification, scrolling past which posts at which speed. You logged four things about yourself and found it hard. Your devices log hundreds of things and never get tired. And remember the fictional activity export from Chapter 1 — four rows of search history were enough to sketch a little story about a stranger’s life. Now imagine forty thousand rows.

How is any of this allowed? The official answer is **consent and the terms of service** — that wall of text you scrolled past and clicked “I Agree” on. Legally, that document is where a company tells you what it collects and what it does with the data (including, often, sharing it with other companies). Practically, almost nobody reads them, and companies know it. That gap — between consent in theory and “I Agree” in practice — is one of the biggest open problems in data ethics, and smart people are actively fighting about how to fix it.

You don’t have to solve it. You just have to stop being on autopilot. Three habits worth building now:

1. **Pause before you share.** The internet has a long memory; data, once collected, is very hard to un-collect. Before posting or signing up, ask: *would I be comfortable with this in a dataset someone else controls?*
2. **Ask the exchange question.** Free apps aren't free — you usually pay in data. That trade can be worth it! But it's only a real *choice* if you notice you're making it: what am I giving, and what am I getting?
3. **Use the controls that exist.** Privacy settings, location permissions, “ask app not to track” — they're imperfect, but they're yours. Checking them occasionally (with a parent or guardian for accounts you share) beats never checking at all.

Notice how the chapter has come full circle. Chapter 1's question about any dataset was *who collected this, how, and what might it be missing?* Data privacy is the same question pointed at yourself: *who is collecting me, how, and did I actually agree to it?*

7.8 Your Turn: The Homework Dataset

Time to run the whole playbook yourself. Ten students were asked how many minutes of homework they did last night:

Student	Minutes of Homework
Jordan	30
Priya	45
Marcus	45
Sofia	60
Tyler	40
Amara	50
Diego	45
Lily	35
Noah	240
Zoe	55

(Noah has a giant project due tomorrow. It's not going great.)

Your tasks — by hand or in a spreadsheet, your choice:

1. **Compute the mean.** Add all ten values, divide by 10.
2. **Find the median.** Sort the values smallest to largest; with ten values, the median is the mean of the 5th and 6th.
3. **Find the mode.** Which value appears most often?
4. **Compare.** Which measures landed close together, and which one got pulled away? By whom?

5. **Write 2–3 sentences** answering: *If you had to report one number as “how much homework a typical student in this group does,” which measure would you choose, and why?* There’s a strong best answer here, but the sentences explaining your reasoning matter more than the pick itself.

Bonus: recompute the mean without Noah. How much did one busy night move the “average” for the whole group? Now imagine a news headline written from each version of the number.

7.9 Roadmap: From Describing to Predicting

Look at the toolkit you just assembled. You can find the middle of a dataset three different ways — and choose the honest one. You know that spread matters as much as the middle. You can spot a correlation and refuse to call it causation until it earns the title. And you know that data, algorithms, and the people they affect are connected by a simple rule: bias in, bias out.

Everything in that toolkit points forward. In Chapter 8, we take the leap from *describing* data to **predicting** with it: building a simple machine learning model that learns a pattern from data and makes predictions about cases it has never seen. Every idea will feel familiar. A model’s predictions are built on the patterns of correlation; judging whether the model is any good comes down to measuring the spread of its mistakes (that Mean Squared Error idea from the spread section); and deciding whether to *trust* the model comes down to the ethics questions you just learned to ask. You already think like a data scientist. Next, you’ll build like one.

8 Introduction to Machine Learning

8.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain what machine learning is — and how “learning patterns from examples” differs from ordinary programming with hand-written rules.
- Tell regression problems apart from classification problems, and sort real-world prediction tasks into the right bucket.
- Explain why data scientists split data into a training set and a test set, and how that split guards against overfitting.
- Train a Linear Regression model and a Decision Tree model in Python with scikit-learn, using the activity dataset you cleaned in Chapter 5.
- Judge a model’s predictions using Mean Squared Error (MSE) and the R^2 score — and explain, in plain English, what those numbers actually say.

8.2 From Describing to Predicting

At the end of [Chapter 7](#), we made you a promise: you’d stop *describing* data and start *predicting* with it. This is that chapter.

And here’s the good news — you’ve already done the hard part. Back in [Chapter 5](#), you spent four whole tutorials cleaning, normalizing, and encoding the activity dataset: Alice, Charlie, and Diana’s steps, heart rates, locations, and spending, all scrubbed and translated into numbers a machine can read. At the time, we kept saying that all this work was “preparing the data for machine learning.” Today the machine finally shows up to learn. You did the cooking; now we eat.

One warning before we start: our first model is going to fail. Not a little — spectacularly. That’s on purpose. Watching a model fail, *measuring* how badly it failed, figuring out why, and then trying something better is not the embarrassing part of machine learning. It **is** machine learning. Let’s go.

8.3 What Is Machine Learning?

i Machine Learning

Machine learning is teaching a computer to find patterns in data and make predictions from them — by showing it many examples, instead of writing exact rules by hand.

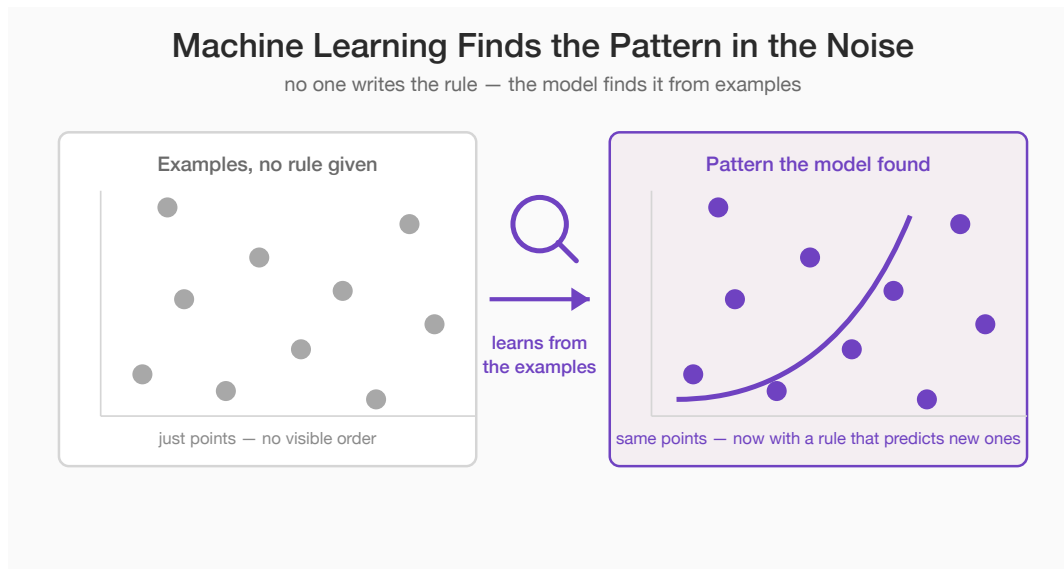


Figure 8.1: Machine learning in one picture: the same scattered examples, before and after a model finds the pattern connecting them — that pattern is what lets it predict new, unseen cases.

To see why that’s a big deal, compare it to the programming you’ve done so far. Everything you wrote in Chapters 4 and 5 was **rule-based**: you told the computer *exactly* what to do, step by step. “If the value is missing, fill it with the median. Multiply this column by that number. Sort by date.” The computer contributed zero ideas of its own; it just followed your recipe very, very fast.

Now try to write exact rules for this: *look at a photo and decide whether there’s a pedestrian in it*. Go ahead — where do you even begin? “If some pixels are... person-shaped”? People come in every size, pose, outfit, and lighting condition. Nobody can write that rule book by hand. But a self-driving car’s obstacle-detection system does this many times per second — because instead of being given rules, it was shown **millions of labeled examples** (“this photo contains a pedestrian; this one doesn’t”) and it *learned* the pattern itself.

You already interact with machine learning daily, probably before breakfast:

- **Your recommendation feed.** TikTok, YouTube, and Spotify don't have a hand-written rule that says "this specific user likes this specific video." They learned your taste from thousands of examples of what you watched, skipped, and replayed — and they predict what you'll want next.
- **A self-driving car's obstacle detection.** As above: trained on enormous piles of labeled road images, so it can predict "pedestrian" or "clear road" for images it has never seen before.
- **Your email's spam filter.** Trained on millions of messages that humans marked as spam or not-spam, it predicts which pile each new message belongs in.

Notice the shape all three share, because it's the shape of this whole chapter: **examples in, pattern learned, predictions out on new, never-before-seen cases.** And notice something else, from Chapter 7: a model learns whatever patterns are in its examples — including the biased ones. *Bias in, bias out* was never just a slogan about the past; it's a warning about every model you're about to build.

8.4 Regression vs. Classification

Prediction problems come in two flavors, and the very first question a data scientist asks about any new problem is: *which flavor is this?*

i Regression

Regression is predicting a *number* — a continuous value that can land anywhere on a scale, like 72.4 beats per minute or \$312,500.

i Classification

Classification is predicting a *category* — a label from a fixed set of choices, like "spam / not spam" or "cat / dog / hamster."

The test is simple: look at the **answer** you want the model to produce. Is it a number you could put on a number line, or a label you could put on a name tag?

Prediction task	Answer looks like...	Flavor
Tomorrow's high temperature	74°F	Regression
A house's selling price	\$312,500	Regression
Is this email spam?	spam / not spam	Classification
Is this wild plant safe or toxic?	safe / toxic	Classification

Now you try. In a few pages we’re going to predict a person’s **HeartRate** from their Steps, Age, distance from school, and so on. Which flavor is that?

Think about the answer the model gives: a heart rate like 78 or 103 — a number on a scale, not a label from a list. That makes it **regression**, which is why both models in this chapter have “Regressor” or “Regression” in their names. (If instead we asked the model to predict each person’s *ActivityType* — Cycling, Running, or Resting — *that* would be classification.)

8.5 Train/Test Split and Overfitting

Here’s a question that sounds dumb and turns out to be one of the most important ideas in all of machine learning: once you’ve trained a model, how do you find out if it’s any good?

The obvious answer: ask it to predict the data it trained on and see if it gets it right. And the obvious answer is a trap.

Imagine a classmate who gets tomorrow’s exact test questions in advance, memorizes the answer key overnight, and scores 100%. Impressive? No — meaningless. The 100% tells you they can *memorize*, not that they *understand*. To find out what they actually learned, you’d have to hand them questions they’ve **never seen**.

Models can pull the same trick, and there’s a name for it:

i Overfitting

Overfitting is when a model learns its training data *too* specifically — memorizing its quirks and noise instead of the real underlying pattern — so it scores brilliantly on data it has seen and falls apart on data it hasn’t. It’s the model that memorized the practice questions but fails the real test.

The defense is beautifully simple:

i Train/Test Split

A **train/test split** divides your dataset into two piles before any learning happens: **training data** (usually around 80%), which the model learns its patterns from, and **test data** (the remaining ~20%), which is kept hidden from the model and used afterward to check its predictions on data it has never seen. The test set is the model’s real exam.

The model never gets to peek at the test set during training — that’s the whole point. If it truly learned the pattern, it will do well on the hidden pile. If it merely memorized, the test set will expose it.

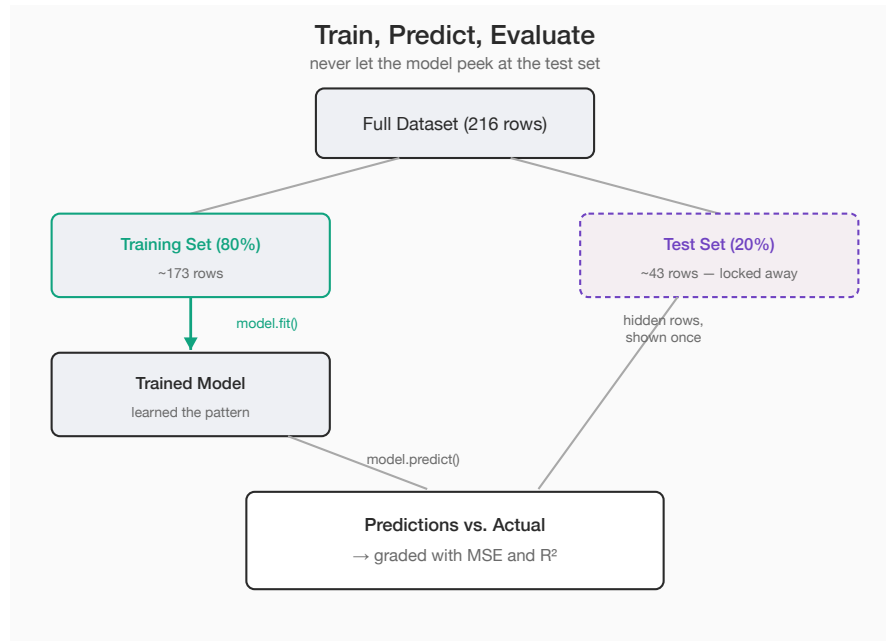


Figure 8.2: The three steps every model in this chapter follows: split the data, fit the model on the training set only, then predict on the test set and grade the result.

The train/test split is your first tool against overfitting, but not the only one. Data scientists also:

1. **Keep the test set locked away** and judge every model on data it never trained on — the split itself.
2. **Prefer simpler models** when possible; a model with fewer knobs to turn has less capacity to memorize noise.
3. **Collect more data** — the more examples a model sees, the harder it becomes to memorize them all, and the more it's forced to learn the actual pattern.

💡 Knowledge Check

A student builds a model to predict quiz scores and proudly reports: “It’s incredible — I tested it on the same 50 rows I trained it on, and it predicted every score almost perfectly!” What would you warn them about, and what one change to their process would give a trustworthy measure of the model’s quality?

Reveal Answer

Warn them that testing on the training data proves nothing — the model may simply have *memorized* those 50 rows (overfitting), like a student who saw the answer key before the test. Near-perfect scores on seen data are exactly what

a memorizer produces. The fix: **split the data first** — train on one portion (say 80%) and evaluate only on the held-out test portion the model has never seen. Its performance on that unseen 20% is the real measure of quality.

8.6 Hands-On: Predicting Heart Rate with Linear Regression

Time to build. Our question: **can we predict a person's heart rate from the other things we know about them?** Our data: the fully processed activity dataset from Chapter 5 — the one where you encoded `Gender` into `Gender_code` and engineered `DistanceFromSchool` out of raw coordinates. Every feature below exists because of work you already did.

💡 Which Chapter 5 Dataset Are We Using?

This chapter picks up right *before* Chapter 5's final Merging/Aggregation step (Part 4, Section 4). At that point, every hourly row has already been cleaned, normalized, and encoded — `Gender` into `Gender_code`, `ActivityType` into one-hot columns, raw coordinates into `DistanceFromSchool` — but not yet collapsed into daily summaries. We work with that per-hour encoded table (`encoded_activity_data.csv`), not the `aggregated_daily_activity.csv` file Chapter 5's Load phase produces — collapsing to one row per day, as that final step does, would erase `Hour` and most of the row-level detail our model needs.

We'll use **scikit-learn** (imported as `sklearn`), the standard Python library for machine learning. First, load the data and choose what the model gets to look at (the **features**, `X`) and what it must predict (the **target**, `y`):

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("encoded_activity_data.csv")

X = df[['Steps', 'Gender_code', 'Age', 'DistanceFromSchool', 'Hour', 'MoneySpent($)']]
y = df['HeartRate']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

That last line is the train/test split from the previous section, made real: `test_size=0.2` hides 20% of the rows in a test set, and `random_state=42` just fixes the random shuffle so you get the same split every time you run it.

Our first model is **Linear Regression** — the simplest prediction machine there is. It assumes the target is a straight-line combination of the features: heart rate equals *some number* times Steps, plus *some number* times Age, and so on. “Training” means finding the numbers that make that line fit the training data as closely as possible.

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Three lines: create the model, fit it to the training pile (this is the learning), then **predict** heart rates for the hidden test pile. Now — the moment of truth. How good are those predictions? We’ll grade them with two standard measures:

```
from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

Output:

```
Mean Squared Error: 484
R2 Score: -1.8
```

(These numbers are reproducible exactly, not illustrative — the dataset is fixed and `random_state=42` fixes the split, so running this code as written gets you the same 484 and -1.8 every time. If you swap in your own dataset or a different `random_state`, expect different numbers.)

Before you can react to those numbers, you need to know what they mean.

i Mean Squared Error (MSE)

Mean Squared Error is the average size of the model’s mistakes, *squared* — for each test row, take the gap between the predicted value and the actual value, square it, then average all those squared gaps. Squaring means big mistakes count extra: missing by 20 hurts far more than four times as much as missing by 5. **Lower is better**, and an MSE of 0 would mean every prediction was perfect. This is exactly the “spread of a model’s mistakes” idea we promised you at the end of [Chapter 7’s section on spread](#).

i R² Score

The **R² score** (“R-squared”) says what fraction of the pattern in the data the model actually explains. An R² of 1 is a perfect model; an R² of 0 means the model is no better than ignoring the features entirely and just guessing the average heart rate for everyone. And yes — R² can go **negative**, which means the model’s predictions are actively *worse* than guessing the average every time.

Now look at our results again. An MSE of 484 sounds abstract until you un-square it: the square root of 484 is 22, so this model’s typical miss is on the order of **22 beats per minute**. Predicting 95 when the truth is 73 — for a resting teenager, that’s not a small miss; that’s the difference between “sitting in class” and “just ran up the stairs.”

And the R² of **-1.8** is the real gut punch. Negative. Our carefully trained model, fed six features of lovingly cleaned data, performs *worse than a model with no features at all* — worse than shrugging and guessing the average heart rate for every single person. All that ETL work, and the machine came back with something a wild guess would beat.

There’s one more way to see the failure, and it’s the most vivid. The source script plots the results with matplotlib:

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Actual HeartRate")
plt.ylabel("Predicted HeartRate")
plt.title("Actual vs Predicted HeartRate (Linear Regression)")
plt.show()
```

Picture what this draws. Each blue dot is one test row: its position left-to-right is the person’s *actual* heart rate, and its height is what the model *predicted*. The red dashed diagonal is the “perfect prediction” line — a dot lands exactly on it only when predicted equals actual. For a good model, the dots hug that diagonal like cars in a lane. For ours? The blue dots are **scattered far from the line** — some high above it (predictions way too high), some far below (way too low), with barely any visible relationship between where a dot *should* be and where it is. It looks less like a lane of traffic and more like the aftermath of a paintball match.

So the verdict is in, delivered three ways — a big MSE, a negative R², and a scatter plot in open rebellion. Our model is bad. The next question is the one that separates data scientists from button-pushers: *why*?

8.7 Why Did the Model Do So Poorly? And What Can We Try Instead?

Resist the urge to conclude “machine learning doesn’t work” or “the data is garbage.” Diagnose instead. There are three honest suspects here:

1. **The test set is small — and possibly noisy.** Our dataset covers three people over a few days; 20% of that is not many rows. With a tiny test set, a couple of unlucky or unusual rows (remember Diana’s outlier heart-rate spike from Chapter 5?) can wreck the scores all by themselves. Small samples make noisy verdicts — the same lesson, you’ll notice, as judging a restaurant by its first two reviews.
2. **Linear regression assumes a straight line — and heart rate may not follow one.** Our model can only express “as X goes up, HeartRate goes up (or down) at a steady rate.” But think about how heart rate actually behaves across a day: low through a resting morning, spiking during a workout, dropping again after — rising and falling in waves. If the true relationship bends and the model can only draw a straight line, it will fail no matter how much data you give it. You can’t trace a curve with a ruler.
3. **We may have the wrong features — or be missing the right ones.** Does MoneySpent(\$) really influence heart rate? Probably not much — it may be adding noise, not signal. Meanwhile, features that *would* matter, like what activity the person was doing at that moment, aren’t in our feature list at all. A model can only learn from what you show it.

Suspect #2 suggests a concrete experiment: try a model that *isn’t* forced to draw straight lines.

Decision Tree

A **decision tree** is a model that makes predictions by asking a sequence of yes/no questions about the features — like a flowchart it learns from the data. For example: *Is Steps greater than 800? If yes, is Hour of Day past 6 PM?* — and each path of answers ends in a prediction. Because each question splits the data into pieces that get their own predictions, a tree can capture patterns that bend, jump, and zigzag — no straight-line assumption anywhere.

If you’ve ever played twenty questions, you already understand decision trees. Let’s see if asking good questions beats drawing a straight line.

8.8 Hands-On: Trying a Decision Tree

Here’s the quiet superpower of scikit-learn: every model speaks the same language — `fit`, then `predict`. Swapping in an entirely different algorithm takes three changed lines. Same training

data, same hidden test set, same grading:

```
from sklearn.tree import DecisionTreeRegressor
tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)

mse_tree = mean_squared_error(y_test, y_pred_tree)
r2_tree = r2_score(y_test, y_pred_tree)
print("Mean Squared Error:", mse_tree)
print("R2 Score:", r2_tree)
```

Output:

```
Mean Squared Error: 82
R2 Score: 0.52
```

Read those against the linear model's report card. MSE fell from 484 to **82** — un-square it and the typical miss shrank from about 22 beats per minute to about **9**. And R^2 climbed from -1.8 all the way to **0.52**: instead of doing worse than guessing the average, the tree now explains about **52% of the variation** in heart rate. Not a perfect model — nearly half the pattern still escapes it — but a genuinely useful one, produced from the *exact same data* that made linear regression look hopeless.

The scatter plot tells the same story visually. Draw it just like before — actual heart rate across, predicted heart rate up, red dashed “perfect prediction” diagonal:

```
plt.scatter(y_test, y_pred_tree, alpha=0.6, color='orange')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Actual HeartRate")
plt.ylabel("Predicted HeartRate")
plt.title("Actual vs Predicted HeartRate (Decision Tree)")
plt.show()
```

This time the orange dots land **noticeably closer to the diagonal** than the blue ones did. They don't sit *on* the line — an R^2 of 0.52 still leaves real scatter, dots drifting above and below — but now there's an unmistakable trend hugging the diagonal: when the actual heart rate is high, the prediction tends to be high; when it's low, the prediction tends to be low. The paintball aftermath has organized itself into something that actually resembles a lane.

8.9 Comparing the Two Models

Put the two contenders side by side:

	Linear Regression	Decision Tree
Task type	Regression (predicts a number)	Regression (predicts a number)
How it works	Finds the best straight-line formula combining the features	Learns a flowchart of if/else questions that split the data into groups
Strengths	Simple, fast, easy to interpret; excellent when the relationship really is close to a straight line	Captures non-linear, bendy relationships; no straight-line assumption
Weaknesses	Can <i>only</i> draw straight lines — hopeless when the true pattern curves	Can overfit by growing very deep and memorizing training data
Result on our data	MSE = 484, $R^2 = -1.8$	MSE = 82, $R^2 = 0.52$

So *why* did the Decision Tree win — and win this decisively? Not because trees are simply “better.” It won because of the mismatch we diagnosed as suspect #2: **the real relationship between our features and heart rate isn’t a straight line, and only one of these models was allowed to bend.** Heart rate rises and falls in waves across a day — resting, active, resting again. Linear regression was structurally forbidden from expressing that shape; forced to summarize a wavy pattern with one flat formula, it produced predictions worse than guessing the average. The tree, free to carve the data with questions — *this* hour range and *that* step count get one prediction, a different combination gets another — could follow the curves the ruler couldn’t.

The lesson to carry with you: **when a model fails, the model’s assumptions are always a suspect.** The right response to a bad score isn’t despair, and it isn’t blind trust in a fancier tool — it’s diagnosis, then a targeted experiment. That’s what you just did.

Knowledge Check

A friend sees your results and says: “Decision Trees beat Linear Regression, 82 to 484. Got it — I’ll never use Linear Regression again.” Using this chapter’s diagnosis of *why* the tree won here, explain what’s wrong with that conclusion, and describe a situation where Linear Regression would be the better choice.

Reveal Answer

The tree didn’t win because trees are universally better — it won because

this particular relationship (features vs. heart rate) is non-linear, and linear regression is structurally limited to straight lines. On a problem where the true relationship really is close to a straight line — say, predicting a taxi fare from trip distance — Linear Regression can be the better tool: simple, fast, easy to explain, and less prone to overfitting than a deep tree that might memorize training quirks. The model should match the shape of the problem; there is no single “best” model for everything.

Try With ChatGPT

Prompt: “I’m a high school student who just trained my first machine learning models. My model’s Mean Squared Error was _____ and its R^2 score was , ***predicting*** (fill in your own numbers and target from the Your Turn activity below — or use this chapter’s results). In plain English: is that a good result? What does each number tell me, and what would you try next to improve it?”

Then do what you practiced in [Chapter 7](#): **verify before you trust**. Check ChatGPT’s explanation of MSE and R^2 against this chapter’s definitions. Did it mention that R^2 can be negative, and what that means? Did its “what to try next” suggestions match any of our three suspects (small data, wrong model shape, wrong features)? Write two sentences on where its explanation matched this chapter — and where, if anywhere, it went off track.

8.10 Your Turn: Two Models Enter

Run the full experiment yourself, end to end. Use your own dataset if you have one from earlier chapters — or the processed activity dataset from [Chapter 5](#).

1. **Pick a target.** Choose one *numeric* column to predict (that makes it regression). If you’re using the activity dataset, try predicting **Steps** instead of **HeartRate** — a fresh question with the same data.
2. **Pick your features.** Choose 3–6 other columns as **X**. For each one, ask Chapter 7’s favorite question: *do I have a reason to believe this is related to the target, or am I just throwing it in?*
3. **Split.** Use `train_test_split` with `test_size=0.2`, exactly as in this chapter.
4. **Train both models.** Fit a `LinearRegression()` and a `DecisionTreeRegressor()` on the same training data.
5. **Grade both.** Compute MSE and R^2 for each model on the test set. Un-square each MSE (take its square root) to get the typical miss in the target’s real units.
6. **Write the verdict.** In 2–3 sentences: which model did better, and *why do you think so?* Don’t just report the numbers — make a claim about the shape of the relationship, the size of your data, or your choice of features, the way this chapter did.

Bonus: describe (or actually plot) the actual-vs-predicted scatter for your better model. How close do your dots come to the red dashed diagonal?

8.11 Roadmap: The Same Idea, a Billion Times Bigger

Take stock of what just happened. A computer looked at examples, found a pattern no one explicitly programmed into it, and made predictions about data it had never seen — and you measured exactly how good those predictions were, diagnosed a failure, and fixed it by choosing a model whose shape matched the problem. That loop — *learn from examples, predict, evaluate, improve* — is the beating heart of every machine learning system on Earth.

Including one you’ve been using all along. In Chapter 9, we turn to the AI tools that have been at your elbow since the start of this book and ask how they actually work — because ChatGPT and its cousins are, at their core, a much, much bigger version of exactly what you did today: a model trained to find patterns in examples and predict what comes next. The differences are scale (trillions of words instead of a few hundred rows of heart rates) and ambition (predicting language itself instead of a number) — but the fundamentals are ones you now own: training data, learned patterns, predictions on unseen input, and models that can be confidently, measurably wrong. You’ve built a model. Next, you’ll understand the biggest ones ever built.

9 How AI Learns From Data

9.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain what a Large Language Model (LLM) is, and how it uses the exact same “learn a pattern from examples, predict on unseen input” idea you practiced in Chapter 8 — just at a vastly larger scale.
- Describe where an LLM’s training data comes from, and why biases or gaps in that data show up in the model’s answers — for the same reason a biased hiring algorithm makes biased decisions.
- Write specific, well-structured prompts that get noticeably better answers than vague ones, and explain *why* they work better.
- Define hallucination, explain in plain language why models produce them, and catch one in the wild.
- Apply the “always verify” rule from [Using AI Responsibly](#) to any factual claim an AI tool makes.

9.2 The Other End of the Prompt

Since the very first chapters of this book, a little box labeled **Try With ChatGPT** has been following you around. You’ve used it to quiz yourself on data types, to decode error messages, to explain your model’s R^2 score in plain English. Each time, you typed some words, and something on the other end typed words back — usually helpful ones, occasionally wrong ones, always *confident* ones.

We never stopped to ask the obvious question: **what is on the other end?**

Now you’re finally ready for the answer, because as of [Chapter 8](#), you’ve built a machine learning model with your own hands. You fed it examples, watched it learn a pattern nobody programmed into it, tested it on data it had never seen, and caught it being confidently wrong. Hold onto every one of those experiences, because here’s the punchline of this chapter: *the thing on the other end of the prompt is the same kind of thing you built* — trained the same basic way, evaluated the same basic way, and wrong in some of the same ways. The difference is scale. Enormous, almost absurd scale.

9.3 What Is a Large Language Model?

Large Language Model (LLM)

A **Large Language Model (LLM)** is a machine learning model trained on enormous amounts of text to predict what word (or word-fragment) is most likely to come next, given everything that came before it. ChatGPT, and tools like it, are LLMs with a chat interface on top.

Read that definition again slowly, because it should sound familiar. In Chapter 8, you trained a model by showing it examples — 216 rows of activity data — and it learned a pattern: *given* Steps, Age, Hour of Day, and the rest, *predict* HeartRate. Then you tested it on rows it had never seen.

An LLM is trained the same basic way — **learn a pattern from examples, then predict on unseen input** — with two things swapped out:

	Your Chapter 8 model	An LLM
Training examples	216 rows of activity data	A giant fraction of all publicly available text — trillions of words
Given...	Steps, Age, Hour of Day, ...	All the words so far (“The capital of France is...”)
Predict...	A heart rate number	The most likely next word (“Paris”)
Tested on	The 20% of rows it never saw	Text it never saw

That’s it. That’s the core mechanism. When you send ChatGPT a prompt, it predicts the most likely next word-fragment, adds it to the text, predicts the next one after *that*, and repeats — thousands of times per response. It turns out that when a model gets extraordinarily good at “what word comes next?” across trillions of words of human writing, the predictions start to look like explanations, translations, working Python code, and answers to your questions. Not because anyone wrote a rule for “how to explain `.loc`” — for the same reason nobody wrote a pedestrian-detection rule for the self-driving car in Chapter 8 — but because the pattern was in the examples, and the model learned it.

One more Chapter 8 idea carries over, and it’s the one to never forget: your heart-rate model didn’t *understand* cardiology — it found statistical patterns in its training rows, and when those patterns didn’t cover a situation, it confidently produced garbage (remember the R^2 of -1.8?). An LLM doesn’t *understand* the world either. It found statistical patterns in text. Most of the time those patterns produce genuinely useful output. Sometimes they produce fluent, confident garbage. We’ll get there in a few pages.

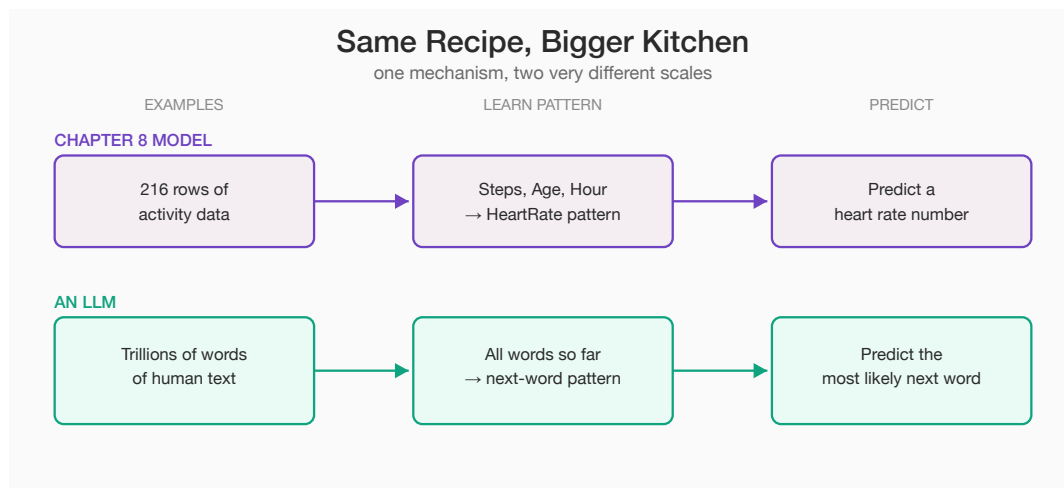


Figure 9.1: The exact same three-step pipeline — examples in, find the pattern, predict on new input — run at two wildly different scales.

9.4 Where Does the Training Data Come From?

i Training Data (AI)

Training data, for an AI model like an LLM, is the collection of examples it learns its patterns from: books, websites, news articles, encyclopedias, forum posts, computer code, and other text gathered at massive scale — often a large slice of the public internet.

Your Chapter 8 model learned from data you personally cleaned and could inspect row by row. An LLM's training data is far too large for any human to read — but it is still *data*, and that means everything you learned about data in this book applies to it.

Think back to [Chapter 1](#), where you learned that **every dataset has a point of view**: data reflects how it was collected, who it was collected from, and what the collector decided to measure. Your 24-hour activity log captured your screen time but not the joke that made your whole table laugh at lunch. Now scale that idea up. A pile of text scraped largely from the internet has a point of view too:

- **Some languages are overrepresented.** There is vastly more English on the web than most other languages, so LLMs are typically strongest in English and noticeably weaker in languages with less online text.
- **Some viewpoints and communities are overrepresented.** People who post a lot online — and topics that get written about a lot — dominate the examples. Perspectives that are rarely written down are, to the model, nearly invisible.

- **Some time periods are missing entirely.** Training data is collected up to a cutoff date. Ask about something that happened after that cutoff, and the model has literally zero examples to draw on — though it may answer anyway.

And now connect this to [Chapter 7’s](#) algorithmic bias lesson. Remember the flu-vaccine model: its hospital records were perfectly accurate, yet the model still made biased predictions, because the *collection* had a blind spot — people who never came to the hospital looked like people who never got sick. **Bias in, bias out.** An LLM works by the exact same mechanism. If its training text underrepresents a language, a community, or a viewpoint — or repeats a stereotype thousands of times — the model learns *that* pattern just as faithfully as it learns grammar, and its responses can reflect the same biases and gaps. Not because anyone programmed it to be biased, but for the same reason the hiring algorithm and the flu model were: **a model learns whatever patterns are in its examples, including the ones we wish weren’t there.**

Knowledge Check

A friend says: “ChatGPT read basically the whole internet, so its answers are neutral — it’s seen every side of everything.” Using Chapter 1’s “data has a point of view” and Chapter 7’s “bias in, bias out,” give two specific reasons this reasoning fails.

Reveal Answer

First, “the whole internet” is itself a dataset with a point of view: it overrepresents certain languages (especially English), certain communities (people who write a lot online), and certain time periods (nothing after the training cutoff) — so the model’s examples were never a balanced sample of “every side” to begin with. Second, bias in, bias out: a model learns whatever patterns dominate its examples, so if a stereotype or one-sided framing appears frequently in the training text, the model absorbs and can reproduce it — exactly the mechanism that made Chapter 7’s flu-vaccine model score the least-connected patients as “low risk.” Seeing *lots* of data is not the same as seeing *representative* data.

9.5 Prompting Is a Skill

Prompt

A **prompt** is the text you give an AI model to get a response — your question, plus any context, instructions, or examples you include with it. The prompt is the model’s *entire* window into what you want.

Here is a secret hiding in plain sight: every **Try With ChatGPT** box in this book has been quietly teaching you prompting this whole time. Go back and look at one — say, Chapter 8’s. It doesn’t say “ask ChatGPT about your model.” It says: **“I’m a high school student who just trained my first machine learning models. My model’s Mean Squared Error was _____ and its R^2 score was _____...”* Who you are. What you did. Your actual numbers. A specific question. That structure was never decoration. It’s the difference between a useful answer and a useless one, and now you’ll see exactly why.

An LLM predicts the most likely continuation *of your prompt*. A vague prompt is compatible with thousands of continuations, so you get the most generic one. A specific prompt narrows the field until the most likely continuation is the answer you actually need. Watch it happen with a real question from [Chapter 4](#):

The vague prompt:

explain loc and query

What does a model do with this? It doesn’t know you mean *pandas* (a “query” could be SQL, a database, a search engine). It doesn’t know what you already understand, what you’re stuck on, or how long an answer you want. So it predicts the most statistically typical response to a vague question about “loc and query”: a multi-paragraph, textbook-style overview — probably about pandas, if you’re lucky — that re-explains things you already know, in someone else’s example data, and never touches the thing you were actually wondering.

The specific prompt:

I’m a high school student learning pandas. I already know how to filter a DataFrame with `.loc`, like this:

```
df.loc[df['Steps'] > 8000, ['Name', 'Steps']]
```

My textbook says `.query()` can do similar filtering. Show me this exact filter rewritten with `.query()`, then give me a short two-column table of when `.loc` is the better choice vs. when `.query()` is. Keep the whole answer under 200 words.

Same topic. Completely different result — and you can name exactly why, piece by piece:

- **Context** (“high school student learning pandas,” “I already know `.loc`”) tells the model what level to pitch the answer at and what *not* to waste words re-explaining.
- **A concrete example** (your actual line of code) anchors the answer in *your* problem instead of a made-up one — the model can now rewrite *your* filter, not invent a random one.
- **A clear, specific question** (“show me this exact filter rewritten with `.query()`”) is compatible with very few continuations — most of them correct and on-target.
- **A stated format** (“a short two-column table,” “under 200 words”) shapes the response into something you can actually use, instead of an essay you have to dig through.

None of this is magic, and none of it requires secret keywords. It's the same skill as asking a good question of a teacher or writing a clear bug report: give context, be specific, say what form the answer should take. The payoff is real — and from now on, when you write your own prompts instead of using ours, you know the recipe.

9.6 When AI Gets It Confidently Wrong

Hallucination (AI)

A **hallucination** is when an AI model states something false as if it were fact — often fluently, in detail, and with total confidence. The model isn't "lying"; it's producing text that *sounds* right without any built-in guarantee that it *is* right.

Why does this happen? One plain sentence covers it: **an LLM is predicting plausible-sounding text, not looking up guaranteed-true facts — unless it has been specifically connected to a search engine or another tool that checks.** “Plausible” and “true” overlap most of the time, which is what makes the failures so sneaky: a hallucination doesn't look like an error. It looks exactly like a correct answer.

Here's what that looks like in *this course*. Suppose you're redoing your Chapter 5 cleaning work and ask an AI assistant:

What pandas method fills the missing values in a column with that column's median?

And it replies, smoothly and helpfully:

You can use the `.fill_missing()` method with the `strategy` parameter:
`df['HeartRate'].fill_missing(strategy='median').`

That answer has everything: a method name that follows pandas naming conventions, a plausible parameter, tidy example code. It is also completely made up. **There is no `.fill_missing()` method in pandas.** Run that line and Python answers instantly:

```
AttributeError: 'Series' object has no attribute 'fill_missing'
```

The real method — the one you actually used in Chapter 5 — is `.fillna()`:

```
df['HeartRate'] = df['HeartRate'].fillna(df['HeartRate'].median())
```

Notice what just happened, because it's the whole lesson in miniature. The hallucinated answer *sounded* more official than the real one. And notice what caught it: not squinting at the answer harder — **running the code**. This is exactly the “always verify” rule from [Using AI Responsibly](#), which warned you on page one that AI tools “can be confidently, fluently wrong — especially about specific numbers, dates, and how library functions behave.” Now you know *why* they can: the model is a next-word predictor trained on patterns, and `fill_missing` fits the pattern of pandas method names beautifully. Plausibility is what LLMs are built to produce. Truth is your job to check:

- **Claims about code:** run it. Two seconds of `AttributeError` beats twenty minutes of confusion.
- **Claims about facts, numbers, or dates:** cross-check against this book, class materials, or a source you trust.
- **“It sounded sure of itself”** is not evidence. It never was. Fluent confidence is precisely what a hallucination is made of.

Knowledge Check

A classmate asks ChatGPT for the pandas method that removes duplicate rows, gets back a confident answer with example code, and pastes it straight into their notebook because “it gave me the exact syntax and everything — it obviously knows.” Explain (a) why the detailed, confident style of the answer is *not* evidence that it's correct, using this chapter's one-sentence explanation of why hallucinations happen, and (b) the fastest way for your classmate to verify the answer.

Reveal Answer

- (a) An LLM produces plausible-sounding text by predicting likely next words — it is not looking up guaranteed-true facts. Fluency, detail, and exact-looking syntax are what the model generates *whether or not the content is true*, so a polished answer is exactly what both a correct response and a hallucination look like. Confidence is the model's style, not its evidence.
- (b) Run the code. If the method is real (it's `.drop_duplicates()`), it executes; if it was hallucinated, Python throws an `AttributeError` immediately. For code claims, execution is the fastest and most reliable verification there is — the same “always verify” rule from [Using AI Responsibly](#).

Try With ChatGPT

This one is bigger than usual, because this time the *prompting itself* is the experiment.

Step 1 — the vague prompt. Pick a real data-science question from this course — for example, how to handle outliers like Diana's heart-rate spike from Chapter 5, or when to

use a bar chart vs. a histogram from Chapter 3. Ask it badly, on purpose, in ten words or fewer: “*how do I deal with outliers*”. Save the response.

Step 2 — the specific prompt. Start a **new chat** (so the model can’t peek at Step 1) and ask the same question using this chapter’s recipe — context, a concrete example, a clear question, a stated format:

Prompt: “I’m a high school student working with a small activity dataset (216 rows: steps, heart rate, age). One person has a heart-rate reading far above all their others, and I think it might be a sensor error. Should I remove it, cap it, or keep it? Give me a short pros-and-cons list for each of the three options, then a one-sentence recommendation for my situation. Under 250 words.”

Step 3 — compare. Put the two responses side by side and write down: (1) two concrete ways the specific prompt’s answer is more useful — did the context, the example, or the stated format cause each improvement? (2) Anything the vague answer included that was generic filler.

Step 4 — the verification pass. Now read *both* responses with this chapter’s eyes and underline every part you should double-check before trusting it: any named method or function (run it), any specific number or statistical claim (check it against Chapters 5 and 7), any confident statement that came with no reasoning attached. A specific prompt gets you a *more useful* answer — it does not get you a *guaranteed-true* one. Even the good response gets the verification treatment.

9.7 Your Turn: Catch One in the Wild

Time to run the “always verify” rule for real, on your own prompt, start to finish.

1. **Ask.** Write your own prompt to ChatGPT (or whichever AI tool your class uses) about anything from this course — a pandas method, a statistics concept from Chapter 7, what some matplotlib function does, a fact about how train/test splits work. Use the specific-prompt recipe; you’ve earned it.
2. **Pick a claim.** From the response, choose one concrete, checkable factual claim. Not an opinion (“this method is convenient”) — a fact: a method name, what a parameter does, a definition, a number.
3. **Actually verify it.** Choose your weapon: run the code in your notebook, look it up in this book, or check an authoritative source (for pandas claims, the official pandas documentation is the referee). “It sounded right” does not count as verification — that’s the exact trap this chapter is about.
4. **Write the verdict.** In 2–3 sentences: What was the claim? Did it hold up — fully, partly, or not at all? And what was your evidence? If the claim survived, say what convinced you. If you caught a hallucination, congratulations — describe it, because you just did professional-grade fact-checking, and you’ll never read an AI answer the same way again.

9.8 Roadmap: Everything, All at Once

Look at the toolbox you're carrying out of Unit III. You can question where data comes from and whose point of view it carries. You can clean it, transform it, and pull it from APIs. You can summarize it honestly with statistics and make it speak with visualizations. You can train models that predict — and diagnose them when they fail. And as of this chapter, you understand the AI tools at your elbow well enough to use them the way a professional does: as a genuinely powerful assistant whose every factual claim gets verified before it gets trusted.

Unit IV is where all of it comes together. In Chapters 10 and 11, you stop practicing on our datasets and our questions, and you build something that's yours: an original data science project, carried from a raw question all the way to a finished analysis — and then presented to an audience that will ask *you* the follow-up questions. Every skill in the toolbox gets used, AI included, tutor-not-ghostwriter included. You've spent nine chapters learning data science. Now you get to do some.

Part IV

Unit IV: Communicating and Applying Data Science

10 Designing Your Own Data Science Project

10.1 Learning Objectives

By the end of this chapter, you will be able to:

- Select a dataset that is both personally interesting and rich enough to support a real analysis.
- Turn a vague topic into a specific, answerable **research question**.
- Map each stage of your capstone project to the skills you learned in Chapters 2 through 8.
- Evaluate a project plan against a concrete rubric — before investing hours of work in it.
- Decide whether your research question calls for a prediction model, or whether analysis and visualization alone will answer it.

10.2 Your Capstone Project

For nine chapters, we handed you the datasets and asked the questions. You filtered half a million Delaware traffic crashes, cleaned Diana’s messy fitness tracker data, pulled live weather from an API, computed statistics that don’t lie, and trained models that predict. Every one of those chapters was practice for this moment.

The capstone project flips the script: **you** choose the dataset, **you** ask the question, and **you** carry the analysis from raw CSV to finished answer. It’s the difference between doing exercises from a textbook and playing an actual game. Nobody knows in advance what you’ll find — not your teacher, not this book, not you. That’s not a bug; that’s the whole point of data science.

This chapter is your project blueprint. It walks through the five steps of the capstone, and at each step it points back to the chapter where you already learned the skill. Nothing here is new material — it’s your entire toolbox, finally used all at once, on a project you own.

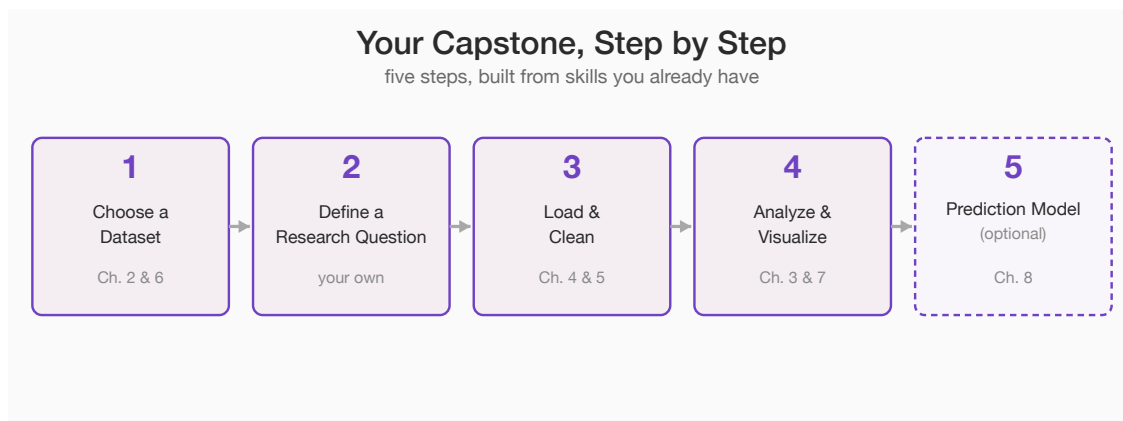


Figure 10.1: The five-step capstone pipeline, each step mapped back to the chapter where you first learned it.

10.3 Step 1: Choose a Dataset

Everything downstream depends on this choice, so choose with two rules in mind:

1. **Pick something you actually care about.** Sports, music, the environment, health, video games, local events, movies — if the topic bores you, the project will feel like a chore, and it will show in the final result. If the topic genuinely interests you, you’ll ask sharper questions and notice things a bored analyst would miss.
2. **Pick something with enough substance to explore.** A dataset with 12 rows and 2 columns runs out of questions fast. Aim for at least a few hundred rows and several columns — ideally a mix of **categorical** columns (types, days, counties, teams) and **numeric** columns (counts, scores, prices, temperatures), because that mix is what makes filtering, grouping, and charting possible.

Where do you find data like that? You already know:

- The **Delaware Open Data Portal** (<https://data.delaware.gov/>) — the source of our nearly 540,000-row traffic crash dataset, plus hundreds of others on health, education, environment, and public safety. [Chapter 2](#) is where you first downloaded a CSV from it.
- **Kaggle** (<https://www.kaggle.com/datasets>) — thousands of datasets on nearly any topic imaginable, from Spotify tracks to Pokémon stats.
- A **public API** — if you want live or very current data (weather, air quality, space launches), [Chapter 6](#) taught you how to request it, read the JSON, and check the terms of service before you do.

Download a candidate and open it. Skim the columns. If you find yourself already wondering things — *“huh, is that higher on weekends?”* — you’ve found your dataset.

10.4 Step 2: Define a Research Question

A dataset is not a project. A **question** is a project. This is the single most common place where capstone projects go wrong — not in the code, but in starting with a topic so vague that no amount of pandas can answer it.

Research Question

A **research question** is a specific, answerable question that your data can actually address — it names what you’re measuring, what you’re comparing, and what would count as an answer. A topic is not a research question; a topic is where research questions come from.

- **Vague topic:** “Traffic accidents.” → **Research question:** “Do traffic accidents in Delaware happen more often on weekends than on weekdays?”
- **Vague topic:** “Music popularity.” → **Research question:** “Do songs with a faster tempo have more streams, on average, than slower songs in this Spotify dataset?”

Notice what the transformation does in both examples: it picks a **measurable outcome** (number of accidents, number of streams), a **comparison** (weekend vs. weekday, faster vs. slower), and implicitly a **verdict** — after the analysis, you can say “yes,” “no,” or “it’s more complicated than that, and here’s how.” A vague topic can never be finished; a research question can.

One more test worth running: ask yourself “*what chart would answer this?*” If you can picture the chart — a bar per day of the week, a scatter plot of tempo vs. streams — the question is probably specific enough. If you can’t picture any chart, the question isn’t ready yet.

Knowledge Check

A classmate proposes this as their capstone research question: “*I want to study climate change.*” Is this a research question? If not, rewrite it into one, assuming they have a dataset of monthly average temperatures for Delaware from 1950 to 2020.

Reveal Answer

It’s a topic, not a research question — it names no measurement, no comparison, and no way to know when you’ve answered it. A rewrite using their actual dataset: “*Have Delaware’s average summer temperatures increased between 1950 and 2020, and by how much per decade?*” Now there’s a measurable outcome (average summer temperature), a comparison (across decades), and a chart you can already picture (a line chart of temperature over time).

Try With ChatGPT

Prompt: “I’m a high school student choosing a dataset for my data science capstone project. I’m considering a dataset about _____ (describe your candidate dataset — its topic and its main columns). Brainstorm 3 possible research questions I could investigate with it.”

Then do the important part yourself: for each of the three, ask “*what would I measure, what would I compare, and what chart would answer it?*” Pick the one that passes all three tests most clearly. If none of them do, tell ChatGPT why and ask it to sharpen them — you’re the editor here, not the audience.

10.5 Step 3: Load and Clean Your Data

You’ve done this twice before, in two different tools:

- [Chapter 4](#) taught you to load a CSV into **pandas** with `read_csv()`, inspect it with `head()` and `info()`, and filter and group it — the skills that tamed nearly 540,000 crash records without a single spinning wheel of doom.
- [Chapter 5](#) taught you the full **ETL** pipeline: extracting raw data, then transforming it — handling missing values, fixing formats, normalization, encoding — into something analysis-ready.

For a smaller dataset, Google Sheets ([Chapter 2](#)) is a legitimate choice too. Use the tool that fits your data’s size.

We won’t re-teach any of it here — just run the checklist you already know:

- ☐ **Missing values:** Are there empty cells or NaNs? Decide — and be ready to justify — whether to drop those rows or fill them.
- ☐ **Irrelevant rows and columns:** If your question is about weekends vs. weekdays, columns that can’t possibly bear on it are clutter. Trim them.
- ☐ **Correct formats:** Are dates actually dates, and numbers actually numbers — not text that merely looks like them?
- ☐ **Transformations, if needed:** If you’ll compare columns on wildly different scales, normalize. If a model will need categorical data as numbers, encode.

Keep a note of every cleaning decision you make and *why*. That “why” becomes part of your final project — real data scientists show their cleaning work, because every dropped row is a judgment call someone might question.

10.6 Step 4: Analyze and Visualize

Now the payoff: making the data answer your question.

- **Filter, group, and calculate.** Slice the data by your comparison — weekend vs. weekday, one category vs. another — and compute the numbers that matter. Reach for the descriptive statistics from [Chapter 7](#): mean, median, and mode each tell a different story, and Chapter 7 showed you how to pick the honest one (and to remember that correlation is not causation when you interpret what you find).
- **Chart it.** A finding nobody can see is a finding nobody believes. [Chapter 3](#) taught you the chart-choosing logic — bar charts for categories, line charts for change over time, scatter plots for relationships, pie charts (sparingly!) for parts of a whole — and gave you CODAP for building them by dragging and dropping. For data living in pandas, the plotting skills from Chapters 4 and 5 do the same job in code.

The standard to aim for: at least one chart that *directly* answers part of your research question. Not a chart of the data in general — a chart where someone can look at it and see the answer. If your question is “do accidents happen more on weekends?”, the bar chart of crashes per day of the week *is* the answer, in picture form.

10.7 Step 5 (Optional): Build a Prediction Model

This step is **optional**, and whether it belongs in your project depends entirely on your research question.

Ask yourself: *is my question a prediction question?* “Do accidents happen more on weekends?” is a **description** question — Steps 3 and 4 fully answer it, and bolting a model onto it adds nothing. But “can I predict tomorrow’s crash count from the day of week and the weather?” is a **prediction** question, and that’s when [Chapter 8](#) enters the game.

If your question is a prediction question, you know the drill:

1. Split your data into training and test sets.
2. Train a **Linear Regression** model, and a **Decision Tree** as a challenger.
3. Compare them with **MSE** and **R²** — and interpret the scores honestly, including when they’re bad. (Remember: Chapter 8’s first heart-rate model scored a *negative* R², and diagnosing why was the most valuable part of the chapter.)

A project with no model is not a lesser project. A description question answered rigorously beats a prediction model duct-taped onto a question that never asked for one.

10.8 What Makes a Good Project?

Here is the rubric. Check your project against it — every item is something you can point to and verify, not a vibe:

- **A specific research question** — it names a measurable outcome and a comparison, and you can picture the chart that would answer it.
- **Real data, actually loaded and explored** — the dataset is open in pandas or Sheets and you’ve run real operations on it, not just described what it contains.
- **At least one cleaning or transformation step, justified in your own words** — for example: *“I dropped 340 rows with missing dates because my question depends on the day of the week,”* not just *“I cleaned the data.”*
- **At least one chart that answers part of the research question** — someone can look at the chart and see the answer, without you standing next to it explaining.
- **An honest account of at least one limitation** — missing data, a small sample size, a possible confounding variable, or a reason your finding might not generalize. Naming a weakness is a strength; pretending there isn’t one is the actual weakness.

Knowledge Check

A student’s project summary reads: *“I found a really interesting dataset about local restaurants and made some nice charts of the data. The analysis was good and I learned a lot.”* Using the rubric above, name at least three specific items this summary gives you no evidence for.

Reveal Answer

It fails to show evidence for nearly every rubric item: (1) no **research question** is stated — “restaurants” is a topic; (2) no **cleaning or transformation step** is described, let alone justified; (3) “some nice charts of the data” doesn’t show a chart that **answers a specific question** — charts *of* data aren’t the same as charts that answer something; and (4) no **limitation** is acknowledged. Words like “interesting,” “nice,” and “good” are exactly the vague adjectives the rubric is designed to replace with checkable evidence.

10.9 Your Turn: Stake Your Claim

Time to put a first stake in the ground for your capstone.

1. **Name a candidate dataset.** Browse the [Delaware Open Data Portal](#), Kaggle, or an API from Chapter 6, and pick one dataset that genuinely interests you. Write down its

name, its source, roughly how many rows it has, and 3–5 of its columns (note which are categorical and which are numeric).

2. **Draft a research question.** Write one first-draft research question for that dataset. Apply the Step 2 tests: What are you measuring? What are you comparing? What chart would answer it?
3. **Run the rubric.** Go through “What Makes a Good Project?” item by item and mark each one: *can my dataset and question deliver this?* If the dataset is too thin to support a cleaning step, a comparison, and a chart — better to find out now. Revise the question (or swap the dataset) until every rubric item is at least *reachable*.
4. **Keep it.** This page of notes is the seed of your actual capstone. You’ll build on it, not start over.

10.10 Roadmap: From Analysis to Story

Suppose you finish every step in this chapter: clean data, a sharp question, a chart that answers it, maybe even a model. You now have something valuable — and if it stays in your notebook, almost nobody will ever benefit from it. An analysis that isn’t communicated might as well not exist.

Chapter 11 covers the final skill of the course: **data storytelling** — turning your capstone from a stack of correct results into a story that someone who has never seen your dataset actually wants to hear, understands on the first pass, and remembers afterward. Your analysis answers the question. Your story is what makes anyone care about the answer.

11 Telling Your Data Story

11.1 Learning Objectives

By the end of this chapter, you will be able to:

- Explain why a finding presented as a **story** sticks with an audience when the same finding presented as a bare number doesn't.
- Identify the four elements of a data story — context, narrative, data, and visual design — and point to each one in your own capstone project.
- Write a one-to-two-sentence **key message** that tells your audience exactly why your finding matters to them.
- Apply five concrete ChatGPT techniques to sharpen your communication — while keeping every claim exactly as accurate as your analysis found it to be.
- Outline and deliver a short presentation of your capstone project, including answers to the tough questions your audience is most likely to ask.

11.2 Data Alone Isn't Enough

You've spent ten chapters learning to get the numbers *right*. This final chapter is about a stubborn fact of human nature: being right isn't enough. People forget numbers. People remember stories.

Try it on yourself. Here is a finding, stated the way an analysis notebook states it:

“Accident rates increased by 20% in 2024.”

True, precise, verifiable — and gone from your memory by tomorrow. Now the same finding, told as a story:

“In 2024, as more people returned to offices, rush-hour traffic surged, leading to a 20% increase in accidents — most of them during the evening commute.”

Same number. Same data. But now the number has a *cause* you can picture (offices reopening), a *scene* you can see (the evening commute), and an *implication* you might act on (be extra careful driving home at 5:30). The first version informs; the second version lands. That's the difference between reporting data and telling a data story — and the tip behind it is worth

memorizing: **always connect your findings to real-life implications your audience cares about.**

i Data Storytelling

Data storytelling is the practice of combining data, visual design, and narrative so that an audience understands *and cares about* your findings — instead of just seeing a chart. The data supplies the evidence; the visuals make it visible; the narrative makes it matter.

This is not decoration, and it is not spin. [Chapter 10](#) ended with the blunt truth that an analysis nobody hears about might as well not exist. Data storytelling is how you make sure yours exists — for an audience who has never seen your dataset, will hear your findings exactly once, and gets to decide in about three minutes whether any of it was worth remembering.

11.3 The Four Elements of a Data Story

Every effective data story — a three-minute class presentation or a front-page investigation — is built from the same four elements:

1. **Context** — *why this matters to this audience*. Before anyone sees a chart, they need to know why they should look. For your capstone, this means opening not with your dataset but with the real-world situation your research question lives in — the thing your classmates already care about or should.
2. **Narrative** — *the story arc*. A question raised, evidence examined, an answer reached: beginning, middle, end. For your capstone, this means presenting your project in the order of your *curiosity* — the question you asked, what the data showed, what it means — not the order of your notebook cells.
3. **Data** — *the actual evidence*. A story without data is just an opinion with confidence. For your capstone, this means the real numbers your Chapter 10 analysis produced — cleaned, computed, and quoted exactly, because the story is only as trustworthy as the analysis underneath it.
4. **Visual design** — *how it's shown*. The right chart makes your answer visible in a glance; the wrong one buries it. For your capstone, this means choosing the one chart from your project that most directly answers your research question — using the chart-choosing logic from [Chapter 3](#) — and labeling it so it works even without you standing next to it.

Take away any one of the four and the story fails in a predictable way. No context? Nobody knows why they're looking. No narrative? It's a data dump. No data? It's an anecdote. No visual design? Your audience has to take your word for what they could have *seen*.

All four elements appear together in one of the most famous data stories ever told. In 1854, physician John Snow was trying to convince London that cholera spread through contaminated

water, not “bad air” — the accepted theory of the time. He plotted every death from a single outbreak as a dot on a street map:

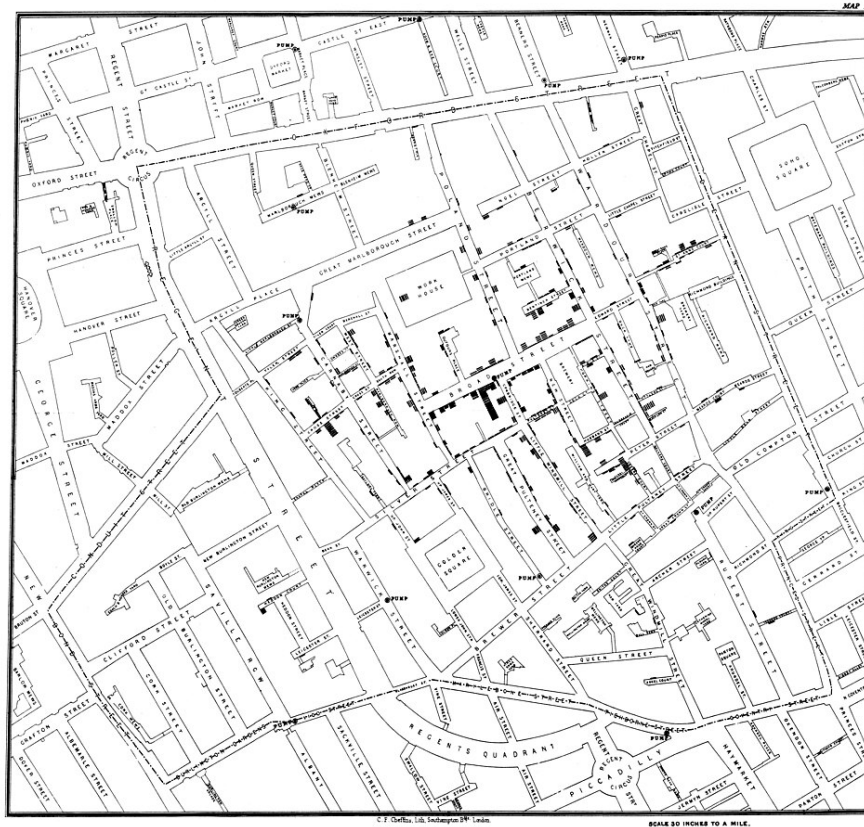


Figure 11.1: John Snow’s 1854 map of the Soho cholera outbreak. Each black bar is a death, stacked at the house where it happened — and they cluster tightly around one water pump on Broad Street.

Context: a deadly outbreak nobody could explain. Narrative: a physician’s contested theory, tested against the evidence. Data: every recorded death, real and exact. Visual design: one map that made the pattern impossible to miss. The city removed the Broad Street pump handle soon after — and modern epidemiology traces its origin to this single, unforgettable chart.

Knowledge Check

A student opens their capstone presentation like this: *“I used pandas to load a CSV with 4,200 rows, dropped the rows with missing values, grouped by day of the week, and made this bar chart.”* Every word is true. Which of the four elements of a data story are missing from this opening, and what would you add first?

Reveal Answer

Context and **narrative** are missing. The opening has data (4,200 real rows, real operations) and a visual (the bar chart), but it never says why the topic matters to the audience or what question drove the work — it narrates the *notebook*, not the *story*. Add context first: one or two sentences about the real-world situation and the question it raised (“*Ever notice traffic feels worse on Fridays? I wanted to know if the crash data agrees*”). Then let the narrative carry the audience from that question to the chart that answers it. The pandas details can wait — or live in the backup slides.

11.4 Define Your Key Message

Here’s a hard question every presenter should answer *before* building a single slide: if your audience forgets everything else, what is the one thing you need them to remember?

Key Message

Your **key message** is the one-to-two-sentence core takeaway you want your audience to remember after your presentation — stated so that it answers the question “*why should my audience care?*” If a listener can repeat your key message to a friend the next day, your data story worked.

A key message is not your research question, and it’s not your dataset description — it’s your *answer*, with its implication attached. Suppose a student named Maya did her capstone on the Delaware crash dataset from [Chapter 2](#), asking whether accidents happen more on weekends. Her key message might be:

“Delaware’s most dangerous driving window isn’t Saturday night — it’s Friday evening rush hour, which has more crashes than any weekend period. If you drive home from practice or a job on Fridays, that’s when you should be most careful.”

Notice what those two sentences do: they state the finding (Friday evening beats the weekend), they include the surprise (not what you’d guess), and they tell the audience what it means *for them* (be careful Friday at 5 PM). Every slide, every chart, every sentence in Maya’s presentation now has one job — supporting that message. Anything that doesn’t support it is a candidate for cutting.

Write your key message first, and the rest of the presentation becomes an organizing problem instead of a blank page.

11.5 Using ChatGPT to Sharpen Your Story

Chapter 9 taught you the prompting recipe — context, a concrete example, a specific question, a stated format — and this section is that recipe applied to communication. Each of the five techniques below is a specific, well-structured prompt aimed at one weak spot presentations tend to have.

One rule frames all five, and it comes straight from [Using AI Responsibly](#): **always verify**. ChatGPT is helping you *phrase* findings, not *produce* them — and a suggestion that makes your message “more engaging” must never make it less accurate. If a punchier phrasing says “crashes doubled” when your analysis found a 20% increase, the punchier phrasing is wrong, and it’s your name on the presentation. After every technique below, check the reworted version against your actual numbers before you use it. You are the analyst here; ChatGPT is the editor you fact-check.

11.5.1 1. Check Clarity

Paste your written analysis or presentation script into ChatGPT and ask: *“Is this easy to follow for a high school audience? Point out anywhere a reader could get lost.”* You’ve been staring at your project for weeks — you can no longer see which parts only make sense because you already know the answer.

- **Before:** “After normalization of the temporal features, aggregation by weekday revealed elevated incident frequency.”
- **After:** “Once I grouped the crashes by day of the week, one day clearly stood out.”

Verify: the clearer version must still describe what you actually did — simpler words, same facts.

11.5.2 2. Make It Engaging

Give ChatGPT your key message and ask for more attention-grabbing ways to phrase it: **“Here’s the main finding of my project: _____. Suggest three more engaging ways to open a presentation with it.”**

- **Before:** “My project found that accident rates were higher on Fridays.”
- **After:** “The most dangerous day to drive in Delaware isn’t the one you’d guess — and the data is unambiguous about it.”

Verify: this is the technique where the “never less accurate” rule earns its keep. “Higher on Fridays” can legally become “the most dangerous day” only if Friday really *was* the highest — not merely high. Check every superlative (“most,” “always,” “doubled”) against your actual results before it goes on a slide.

11.5.3 3. Polish Language

Titles, captions, and conclusions are the three places your audience actually reads word-for-word, so they deserve the most polish. Ask: **“Here’s my chart title and caption: _____. Suggest tighter versions that state the finding, not just the topic.”**

- **Before:** Chart title — “Accident Data by Day.”
- **After:** Chart title — “Friday Evenings Have the Most Crashes in Delaware.”

Verify: the polished title now makes a *claim*, and a claim can be wrong. Reread your chart and confirm the title states exactly what the bars show — no more, no less.

11.5.4 4. Simulate Audience Q&A

Before you present, ask ChatGPT: **“Here’s a summary of my data science project: _____. What tough questions might people ask about this?”* — then draft an answer to each one. This is a rehearsal you can run at 11 PM the night before, and it converts the scariest part of presenting (the questions you didn’t see coming) into a part you’ve already practiced.

- **Before:** A classmate asks “But couldn’t that just be because more people drive on Fridays?” — and you freeze, because you never thought about it.
- **After:** You saw that exact question in your simulated Q&A, and you have an answer ready: “Great question — my data counts crashes, not crash *rates* per driver, so I can’t rule that out. That’s the main limitation of my project.”

Verify: draft the answers *yourself*, from your own analysis. If ChatGPT’s simulated question exposes something you genuinely can’t answer, that’s not a phrasing problem — it’s a finding about your project, and honesty (“my data can’t answer that”) beats improvisation.

11.5.5 5. Match Visuals to Narrative

Describe your chart to ChatGPT — what type it is, what’s on each axis, what pattern it shows — along with your key message, and ask: *“Does this chart match the story I’m telling? Is there a mismatch between what I’m claiming and what the chart shows?”*

- **Before:** “This is a line chart of accidents.”
- **After:** “This chart shows how Friday night accidents spike right when people are driving home from work.”

Verify: ChatGPT never saw your actual chart — only your description of it. Before adopting its suggestion, put the reworded caption next to the real chart and confirm the spike it describes is actually there, at the actual time it names.

Knowledge Check

A student's analysis found that ice cream sales and drowning incidents both rise in summer months. They ask ChatGPT to make their key message "more engaging," and it suggests: *"Shocking data reveals: ice cream sales are driving a surge in drownings!"* The student thinks it sounds great. Using this section's rules — and something you learned in Chapter 7 — explain what's wrong and write a version that is engaging *and* accurate.

Reveal Answer

The suggestion fails the "always verify" rule in the worst way: it converts a **correlation** into a **causation** — exactly the trap Chapter 7 warned about. Ice cream doesn't cause drowning; summer (the confounding variable) drives both. "More engaging" made the claim *false*, which means it's not usable no matter how good it sounds. An engaging *and* accurate version: *"Ice cream sales can 'predict' drownings — not because dessert is dangerous, but because both spike in summer. It's the clearest everyday proof that correlation isn't causation."* Still surprising, still attention-grabbing, and every word of it is true.

11.6 Preparing for Audience Questions

The question-and-answer moment after a presentation is where data stories are really tested — and it's also the easiest part to prepare for, because you already have the tool: technique #4, **simulate audience Q&A**, run on your own project summary a day or two before you present.

Pair it with one more preparation that requires no AI at all. Chapter 10's rubric required you to give *an honest account of at least one limitation* — missing data, a small sample, a possible confounding variable. Go back to the limitation you named, and be ready to explain it out loud if asked. There is a real difference between an audience member *discovering* your project's weakness and you *already knowing* it: the first makes you look caught; the second makes you look like a scientist. "That's a limitation I identified — here's what it means for my conclusion" is one of the strongest sentences you can say in a Q&A.

Try With ChatGPT

Run this right before you present — it's the dress rehearsal for the Your Turn below.

Prompt: "I'm a high school student about to present my data science capstone project. My research question was , *my dataset was* , and my key message is: _____. Act as a skeptical audience member: ask me the 5 toughest questions you'd ask after this

presentation.”

Then answer all five **out loud, in your own words, without ChatGPT’s help** — that part is the actual practice. For any question you can’t answer, check it against your analysis: some will have answers hiding in work you already did, and at least one will probably touch the limitation from Chapter 10’s rubric — which you should be ready to own, not dodge. Remember the verification rule: if ChatGPT’s questions assume something about your project that isn’t true, correcting it is part of the exercise.

11.7 Your Turn: Present Your Data Story

This is it — **the final deliverable of your capstone project**. In [Chapter 10](#) you chose the dataset, defined the research question, cleaned the data, and built the chart that answers it. This Your Turn is where all of that becomes something an audience experiences. Three steps:

1. **Write your key message.** One to two sentences, following this chapter’s definition: your finding, plus why your audience should care. Test it against Maya’s example — does it state the answer (not the question), and does it pass “why should my audience care?” Sharpen it with techniques #1–#3 if it helps, and verify every reworded claim against your actual numbers.
2. **Pick your single best chart.** Not all of your charts — *one*. The chart from your Chapter 10 project that most directly answers your research question, with a title that states the finding (technique #3) and a match between what it shows and what you claim (technique #5).
3. **Outline a short presentation**, three to five minutes, in this arc:
 - **Context** — why this topic matters, in terms your audience already cares about (no dataset details yet);
 - **Key message** — the takeaway, stated early and plainly, not saved for a dramatic ending;
 - **The chart** — walk your audience through it until they can *see* the answer for themselves;
 - **The takeaway** — what you’d ask the audience to remember or do, plus the one limitation you’re prepared to discuss.

Then run the Try With ChatGPT box above as your dress rehearsal, and present. When you finish — when a room full of people who never saw your dataset understands your question, believes your evidence, and remembers your message — you will have done, end to end, the actual job of a data scientist.

11.8 You’ve Built a Full Pipeline

Step back and look at how far you’ve come — not just in this chapter, but across the whole book.

In **Unit I**, you learned what data actually is, tamed it in spreadsheets, and made your first charts — the foundations. In **Unit II**, you outgrew the spreadsheet: Python and pandas handled half a million rows without flinching, ETL turned messy real-world data into something trustworthy, and APIs let you pull live data from the wider world. In **Unit III**, you made data *mean* something — statistics that don’t lie, machine learning models you trained and honestly evaluated, and a clear-eyed understanding of the AI tools everyone else treats as magic. And in **Unit IV**, you put it all together: a capstone project that was yours from raw CSV to finished chart, and now a story that makes an audience care about what you found.

That sequence isn’t just a table of contents. It’s a **pipeline** — question → data → cleaning → analysis → model → story — and it’s the same pipeline running inside every data science team at every hospital, newsroom, sports franchise, and research lab on Earth. The tools get fancier out there; the pipeline doesn’t change. And as of today, you own it end to end. Not one link of it. All of it.

So here’s what comes next, and it isn’t a chapter: a real dataset, a real question, and a real audience. The world produces more data every day than anyone can analyze, which means it is permanently full of questions nobody has gotten around to asking — about your town, your team, your music, your future. You now have everything you need to pick one and find out.

Go ask a good question. You know exactly what to do with the answer.

A Glossary

Every term introduced in this book, collected in one place. Each entry links back to the chapter where it's first taught, if you want the fuller explanation and worked example.

A.1 Algorithmic Bias

Algorithmic bias happens when a computer system produces systematically unfair results because the data it learned from contained bias. The algorithm isn't malicious — it's *obedient*. It faithfully learns whatever patterns are in its training data, including the unfair ones. Data scientists summarize it in four words: **bias in, bias out**.

See [Descriptive Statistics & Data Ethics](#).

A.2 API

An **API** (Application Programming Interface) is a way for one program to ask another program for data — usually over the internet — and get back a structured answer. The requesting program sends a specially formatted URL; the serving program replies with machine-readable data instead of a human-readable web page.

See [Where Data Comes From: APIs & Real-World Sources](#).

A.3 Bias (in data)

Bias (in data) is a systematic tilt in a dataset — caused by how, where, or from whom the data was collected — that makes it misrepresent the real world. A biased dataset isn't “wrong numbers”; it's true numbers that tell a slanted story.

See [What Is Data?](#).

A.4 Big Data

In this context, **Big Data** refers to datasets that are too large or complex to be dealt with efficiently by traditional data-processing software, necessitating more robust tools like Python and Pandas.

See [Python & Pandas: Working with Big Data](#).

A.5 Boolean Masking

Bracket notation works by creating a **Boolean Mask**—a series of True/False values for every row. Python then only displays the rows marked “True.”

See [Part 4: Filtering Like a Pro](#).

A.6 Causation

Causation means one thing actually *makes* the other happen — changing the first genuinely changes the second. Causation is a claim about how the world *works*, and it’s much, much harder to prove than correlation.

See [Descriptive Statistics & Data Ethics](#).

A.7 Cell

A **cell** is one block of a notebook. A *code cell* holds Python code you can run; the output appears directly underneath it. You can add as many cells as you like with the + **Code** button.

See [Getting Set Up](#).

A.8 Cell Reference

A **cell reference** is the address of a cell or range of cells, written as a column letter plus a row number. B2 means “column B, row 2.” A range like B2:B540000 means “every cell in column B from row 2 down to row 540000.” Formulas use references so they can work on data wherever it lives.

See [Data Wrangling with Spreadsheets](#).

A.9 Classification

Classification is predicting a *category* — a label from a fixed set of choices, like “spam / not spam” or “cat / dog / hamster.”

See [Introduction to Machine Learning](#).

A.10 Correlation

Correlation means two things tend to move together: when one goes up, the other tends to go up (or reliably down). Correlation describes a *pattern* you can see in the data. It says nothing about *why* the pattern exists.

See [Descriptive Statistics & Data Ethics](#).

A.11 Dashboard

A **dashboard** is a visual display that gathers one or more charts (and sometimes tables or maps) in a single view, so a reader can absorb the most important results of a dataset at a glance — without ever touching the raw rows.

See [Data Visualization Basics](#).

A.12 Data

Data is information that can be collected, stored, and analyzed. It comes from our actions, our devices, and the world around us — every step counted, song skipped, and door opened can become a data point.

See [What Is Data?](#).

A.13 Data Logging

Data logging is the practice of recording observations or measurements over time, at regular intervals or as events happen, so the record can be analyzed later. A fitness tracker logs your steps automatically; a scientist logging tide heights every hour is doing the same thing by hand.

See [What Is Data?](#).

A.14 Data Preparation

Data Preparation is the process of cleaning and transforming raw data prior to processing and analysis. It often involves reformatting types and handling missing values.

See [Part 4: Filtering Like a Pro](#).

A.15 Data Privacy

Data privacy is your ability to control information about yourself — what gets collected, who can see it, how it's used, and how long it's kept. It rests on **consent**: the idea that companies should tell you what they collect and get your agreement before collecting it.

See [Descriptive Statistics & Data Ethics](#).

A.16 Data Storytelling

Data storytelling is the practice of combining data, visual design, and narrative so that an audience understands *and cares about* your findings — instead of just seeing a chart. The data supplies the evidence; the visuals make it visible; the narrative makes it matter.

See [Telling Your Data Story](#).

A.17 Data Visualization

Data visualization is the practice of representing data as pictures — charts, graphs, maps — so that patterns, trends, and outliers can be seen at a glance instead of computed in your head. It is the last step of the summarize-then-show pipeline you started in Chapter 2.

See [Data Visualization Basics](#).

A.18 DataFrame

A **DataFrame** is the table Pandas stores your data in — rows and columns, like a spreadsheet, but handled entirely through code. `pd.read_csv()` is the function that reads a CSV file *into* a DataFrame.

See [Part 3: Meet Your New Best Friend—Pandas](#).

A.19 Dataset

A **dataset** is an organized collection of data — often arranged in rows and columns, like a giant spreadsheet — that we can explore, question, and analyze.

See [How to Use This Book](#).

A.20 Decision Tree

A **decision tree** is a model that makes predictions by asking a sequence of yes/no questions about the features — like a flowchart it learns from the data. For example: *Is Steps greater than 800? If yes, is Hour of Day past 6 PM?* — and each path of answers ends in a prediction. Because each question splits the data into pieces that get their own predictions, a tree can capture patterns that bend, jump, and zigzag — no straight-line assumption anywhere.

See [Introduction to Machine Learning](#).

A.21 Dual-Axis Chart

A **dual-axis chart** is a chart-literacy term for a graph with *two different* y-axis scales — one on the left, one on the right — so two variables with very different ranges (say, crash counts in the thousands and rainfall in inches) can share one picture. CODAP supports adding a second, right-side y-axis as a separate option, but handle the idea with care wherever you meet it: as you’ll see in “How Charts Can Mislead,” two independently-scaled axes are one of the easiest ways to accidentally lie with a chart.

See [Data Visualization Basics](#).

A.22 Encoding

Encoding is the process of converting categorical data (text-based labels) into numerical values. This ensures that non-numeric features can be used as inputs for mathematical models.

See [Part 4: Encoding, Merging, and the ETL Endgame](#).

A.23 Endpoint

An **endpoint** is one specific URL (or URL pattern) that an API exposes for one specific kind of data. Open-Meteo's `/v1/forecast` endpoint serves weather forecasts; its `/v1/air-quality` endpoint serves air-quality data. One API, many endpoints — like one restaurant menu with separate sections for entrées and desserts.

See [Where Data Comes From: APIs & Real-World Sources](#).

A.24 ETL

ETL stands for **Extract, Transform, and Load** — a three-stage process that turns raw, messy data into something clean, structured, and ready for analysis or machine learning. It's the invisible backbone behind every dashboard and prediction model, ensuring that data is trustworthy, consistent, and analysis-ready.

See [ETL in Action: Cleaning & Preprocessing](#).

A.25 Filter (spreadsheet)

A **filter** is a spreadsheet feature that temporarily hides rows that don't match conditions you choose, so you only see the subset you care about. The hidden rows aren't deleted — turn the filter off and they all come back.

See [Data Wrangling with Spreadsheets](#).

A.26 Formula

A **formula** is an instruction you type into a spreadsheet cell, always starting with `=`, that tells the spreadsheet to calculate a value for you. When the underlying data changes, the formula automatically recalculates — you never have to redo the work by hand.

See [Data Wrangling with Spreadsheets](#).

A.27 Google Colab

Google Colab (short for “Colaboratory”) is a free website from Google that lets you write and run Python code in your browser. Your code runs on Google’s computers in the cloud, and your work saves automatically to your Google Drive — just like a Google Doc.

See [Getting Set Up](#).

A.28 Hallucination (AI)

A **hallucination** is when an AI model states something false as if it were fact — often fluently, in detail, and with total confidence. The model isn’t “lying”; it’s producing text that *sounds* right without any built-in guarantee that it *is* right.

See [How AI Learns From Data](#).

A.29 Integrated Development Environment (IDE)

An **IDE** is a super-powered digital notebook designed specifically for writing, running, and managing code in one centralized location.

See [Part 2: Getting Started with PyCharm](#).

A.30 JSON

JSON (JavaScript Object Notation) is a text format for structured data. Despite the “JavaScript” in its name, every programming language can read it — and to a Python programmer, JSON looks almost exactly like nested **dictionaries** (curly braces, “`key`”: value pairs) and **lists** (square brackets).

See [Where Data Comes From: APIs & Real-World Sources](#).

A.31 Key Message

Your **key message** is the one-to-two-sentence core takeaway you want your audience to remember after your presentation — stated so that it answers the question “*why should my audience care?*” If a listener can repeat your key message to a friend the next day, your data story worked.

See [Telling Your Data Story](#).

A.32 Large Language Model (LLM)

A **Large Language Model (LLM)** is a machine learning model trained on enormous amounts of text to predict what word (or word-fragment) is most likely to come next, given everything that came before it. ChatGPT, and tools like it, are LLMs with a chat interface on top.

See [How AI Learns From Data](#).

A.33 Library

A **Library** is like a box of LEGO bricks—ready-made code you snap together to add functionality to your programs rather than building every tool from scratch.

See [Part 3: Meet Your New Best Friend—Pandas](#).

A.34 Machine Learning

Machine learning is teaching a computer to find patterns in data and make predictions from them — by showing it many examples, instead of writing exact rules by hand.

See [Introduction to Machine Learning](#).

A.35 Mean

The **mean** (what most people call the “average”) is the sum of all the values divided by how many values there are. It uses every single data point — which is both its superpower and its weakness.

See [Descriptive Statistics & Data Ethics](#).

A.36 Mean Squared Error (MSE)

Mean Squared Error is the average size of the model’s mistakes, *squared* — for each test row, take the gap between the predicted value and the actual value, square it, then average all those squared gaps. Squaring means big mistakes count extra: missing by 20 hurts far more than four times as much as missing by 5. **Lower is better**, and an MSE of 0 would mean every prediction was perfect. This is exactly the “spread of a model’s mistakes” idea we promised you at the end of [Chapter 7’s section on spread](#).

See [Introduction to Machine Learning](#).

A.37 Median

The **median** is the middle value when the data is sorted from smallest to largest. Half the values sit below it, half above. (If there's an even number of values, the median is the mean of the two middle ones.) Because it only cares about *position*, the median barely notices outliers.

See [Descriptive Statistics & Data Ethics](#).

A.38 Mode

The **mode** is the value that appears most often in a dataset. It's the only measure of the middle that works on *categorical* data — data made of labels and categories instead of numbers.

See [Descriptive Statistics & Data Ethics](#).

A.39 Overfitting

Overfitting is when a model learns its training data *too* specifically — memorizing its quirks and noise instead of the real underlying pattern — so it scores brilliantly on data it has seen and falls apart on data it hasn't. It's the model that memorized the practice questions but fails the real test.

See [Introduction to Machine Learning](#).

A.40 pip

pip stands for “Package Installer for Python.” Think of it as Python's own App Store. You tell it what you need, and it fetches and installs everything for you automatically.

See [Part 3: Meet Your New Best Friend—Pandas](#).

A.41 Pivot Table

A **pivot table** is a spreadsheet tool that automatically groups rows by the categories in one or more columns and calculates a summary (count, sum, average...) for each group. It turns a long list of raw rows into a compact grid of totals — no formulas required.

See [Data Wrangling with Spreadsheets](#).

A.42 Prompt

A **prompt** is the text you give an AI model to get a response — your question, plus any context, instructions, or examples you include with it. The prompt is the model’s *entire* window into what you want.

See [How AI Learns From Data](#).

A.43 Rate Limit

A **rate limit** is a cap on how many requests you may send an API in a given time window (for example, Open-Meteo’s free tier allows up to 10,000 requests per day). Exceed it and the API starts refusing you.

See [Where Data Comes From: APIs & Real-World Sources](#).

A.44 Regression

Regression is predicting a *number* — a continuous value that can land anywhere on a scale, like 72.4 beats per minute or \$312,500.

See [Introduction to Machine Learning](#).

A.45 Research Question

A **research question** is a specific, answerable question that your data can actually address — it names what you’re measuring, what you’re comparing, and what would count as an answer. A topic is not a research question; a topic is where research questions come from.

- **Vague topic:** “Traffic accidents.” → **Research question:** “Do traffic accidents in Delaware happen more often on weekends than on weekdays?”
- **Vague topic:** “Music popularity.” → **Research question:** “Do songs with a faster tempo have more streams, on average, than slower songs in this Spotify dataset?”

See [Designing Your Own Data Science Project](#).

A.46 R² Score

The **R² score** (“R-squared”) says what fraction of the pattern in the data the model actually explains. An R² of 1 is a perfect model; an R² of 0 means the model is no better than ignoring the features entirely and just guessing the average heart rate for everyone. And yes — R² can go **negative**, which means the model’s predictions are actively *worse* than guessing the average every time.

See [Introduction to Machine Learning](#).

A.47 String

A **String** is a data type used in programming to represent text. In Python, strings are always wrapped in quotation marks.

See [Part 2: Getting Started with PyCharm](#).

A.48 Structured Data

Structured data is neatly organized information with a consistent format — usually rows and columns — which makes it easy for computers to search, sort, and analyze. Examples:

1. Your daily step counts (a number, tied to a date).
2. GPS coordinates from a phone’s map app (latitude, longitude, timestamp).
3. Calendar events (a title, a date, a start time, an end time — always the same fields).

See [What Is Data?](#).

A.49 The Load Phase

The **Load** phase is the final stage of ETL where transformed data is saved to a permanent destination, ensuring it is preserved for future analysis or application.

See [Part 5: The Load Phase](#).

A.50 Trace

A **trace** is one layer of data drawn on a chart — one line on a line chart, one set of same-colored points on a graph. A chart comparing two variables has two traces; the legend tells you which is which.

See [Data Visualization Basics](#).

A.51 Train/Test Split

A **train/test split** divides your dataset into two piles before any learning happens: **training data** (usually around 80%), which the model learns its patterns from, and **test data** (the remaining ~20%), which is kept hidden from the model and used afterward to check its predictions on data it has never seen. The test set is the model’s real exam.

See [Introduction to Machine Learning](#).

A.52 Training Data (AI)

Training data, for an AI model like an LLM, is the collection of examples it learns its patterns from: books, websites, news articles, encyclopedias, forum posts, computer code, and other text gathered at massive scale — often a large slice of the public internet.

See [How AI Learns From Data](#).

A.53 Transform

Transform is the process of converting raw data into a structured format by applying operations like filtering, encoding, normalization, or feature engineering.

See [Part 3: The Transform Engine Room](#).

A.54 Unstructured Data

Unstructured data is information with no fixed format, which makes it much harder for computers to process directly. Examples:

1. Photos in your camera roll (a grid of millions of colored pixels — nowhere does it *say* “birthday party”).

2. Videos (thousands of those photos per minute, plus sound).
3. Voice memos (sound waves, not sentences — a computer has to work hard just to figure out what words were said).

See [What Is Data?](#).

B Getting Set Up

Before you can do data science with Python, you need a place to actually write and run Python code. The good news: you have options. This appendix walks you through two of them — **PyCharm**, a program you install on your own computer, and **Google Colab**, which runs entirely in your web browser. Not every school issues laptops that allow software installs (many issue Chromebooks, which don't), so pick whichever path your school and your computer support. Both will get you to the exact same place: writing real Python code.

B.1 Option A: PyCharm (If You Can Install Software)

If you have a laptop where you're allowed to install programs — your own computer, or a school-issued one without install restrictions — PyCharm is a fantastic choice. It's the same kind of professional tool (an **IDE**) that working data scientists use every day.

Here's the short version of getting started:

- **Download PyCharm** from the [official JetBrains website](#), choosing the version for your operating system (Windows, macOS, or Linux).
- **Pick the Community Edition** — it's completely free and has everything a beginner needs.
- **Run the installer** and follow the prompts to finish setup.
- **Create a new project** when PyCharm first launches — and remember the “no spaces” rule: name it something like `my_first_project`, never `my first project`.
- **Write and run your first program** — a one-line `print('hello Python')` — and look for the “Process finished with exit code 0” message, the computer's way of giving you a thumbs up.

That's the trailer; the full movie is in [Part 2 of the Python chapter](#), which walks through every step with details on the interface, naming conventions, and how to read your first console output. If you're going the PyCharm route, head there for the complete walkthrough.

B.2 Option B: Google Colab (Chromebook-Friendly, Nothing to Install)

If you're on a Chromebook — or any computer where you can't install software — you can still write and run Python. Google Colab runs Python entirely in your web browser, using Google's computers to do the actual work. All you need is a browser and a Google account.

Google Colab

Google Colab (short for “Colaboratory”) is a free website from Google that lets you write and run Python code in your browser. Your code runs on Google's computers in the cloud, and your work saves automatically to your Google Drive — just like a Google Doc.

B.2.1 Step 1: Go to the Colab Website

Open your browser and go to colab.research.google.com. If you're not already signed in, click **Sign in** in the top-right corner and log in with your Google account (your school Google account works, as long as your school hasn't blocked Colab — if it has, ask your teacher).

Instructor Insight: Notebooks, Not Files

PyCharm organizes code into `.py` files inside a project folder. Colab instead uses **notebooks** — documents made of individual blocks called **cells**, where each cell can hold code you run one chunk at a time. Notebooks are hugely popular in data science because you can run a step, look at the result, and then decide what to do next. Same Python, different container.

B.2.2 Step 2: Create a New Notebook

When Colab opens, it usually greets you with an **Open notebook** window listing recent and example notebooks. To start fresh:

1. Click **New notebook** (in the Open notebook window), *or*
2. If you're already looking at a notebook, use the menu: **File** → **New notebook in Drive**.

A brand-new notebook appears, named something like `Untitled0.ipynb`. Click that name at the top of the page and rename it — the “no spaces” rule from PyCharm is a good habit here too, so try `my_first_notebook`. Colab saves it to your Google Drive automatically as you work.

Cell

A **cell** is one block of a notebook. A *code cell* holds Python code you can run; the output appears directly underneath it. You can add as many cells as you like with the **+ Code** button.

B.2.3 Step 3: Run Your First Code

Your new notebook starts with one empty code cell, cursor already blinking inside it. Type:

```
print('hello Python')
```

Now run it, either way works:

- Click the **round play button** () on the left edge of the cell, or
- Press **Ctrl+Enter** (**Cmd+Enter** on a Mac) to run the cell, or **Shift+Enter** to run it and move to the next cell.

The very first time you run a cell, Colab pauses for a few seconds while it says **Connecting** or **Allocating** — that’s Colab starting up a computer in the cloud just for you. This is normal; it only happens once per session. Then you’ll see the output appear right below the cell:

hello Python

If you see that, congratulations — you’re running Python in the cloud. A small green checkmark next to the cell is Colab’s version of PyCharm’s “exit code 0” thumbs up.

Instructor Insight: When the Runtime Falls Asleep

If you leave a Colab tab idle for a while, Colab disconnects your cloud computer to save resources. Your *code* is always safe (it’s saved in Drive), but anything the code *created* — variables, uploaded files — gets wiped. No panic needed: just run your cells again from the top. This is the single most common “wait, it was working a minute ago!” moment for new Colab users.

B.2.4 Step 4: Working with Data Files (Where Colab Differs from PyCharm)

Later in this book you’ll load data from **CSV files**. In PyCharm this is simple: you drop the CSV into your project folder and read it by name. In Colab, your code runs on Google’s

computer, not yours — so that computer can't see the files on your laptop until you hand them over. You have two ways to do that.

Way 1: Upload the file directly. Run this in a code cell:

```
from google.colab import files
uploaded = files.upload()
```

A **Choose Files** button appears below the cell. Click it, pick your CSV, and once it uploads you can read it just like in PyCharm:

```
import pandas as pd
df = pd.read_csv('my_data.csv')
```

The catch: uploaded files vanish whenever the runtime disconnects (see the sleepy-runtime note above), so you'll re-upload at the start of each session. That's fine for small class datasets.

Way 2: Mount your Google Drive. For files you'll use again and again, store them in your Google Drive and connect Drive to your notebook:

```
from google.colab import drive
drive.mount('/content/drive')
```

Colab will pop up a permission window asking you to allow access to your Drive. Once you approve, everything in your Drive appears under the folder path `/content/drive/MyDrive/`, and you can read a file like this:

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/my_data.csv')
```

Files in Drive stick around between sessions — no re-uploading needed.

B.3 Which Should I Use?

Our Recommendation

If your computer lets you install software and you like the idea of a professional desktop tool, go with **PyCharm** — it's the closest match to how the Python chapter's screenshots and walkthroughs are written. If you're on a Chromebook, a shared computer, or you just want the fastest possible start, go with **Google Colab** — zero installation, and your

work follows you to any computer you sign into. Either way, the Python you write is identical, and everything in this book works in both.

C Dataset & Tool Directory

A quick reference to every tool and data source this book uses, gathered in one place.

C.1 Where to Find Data

- [Delaware Open Data Portal](#) — real government datasets (traffic crashes, school enrollment, park locations, COVID-19 trends, and more). Used throughout this book, starting in Chapter 2.
- [Kaggle](#) — a huge library of datasets and a learning community, mentioned as an alternative data source starting in Chapter 2.

C.2 Tools Used in This Book

- [CODAP](#) — free, no-login drag-and-drop data visualization, used in Chapter 3. See also CODAP’s official [How-to Guides](#).
- [Google Sheets](#) — spreadsheet formulas, filtering, and pivot tables, used in Chapter 2.
- [PyCharm](#) — a Python IDE, set up in Chapter 4 (see also [Getting Set Up](#)).
- [Google Colab](#) — a no-install alternative to PyCharm, covered in [Getting Set Up](#).
- [Open-Meteo](#) — a genuinely free weather API (no account, no API key), used in Chapter 6 to teach how APIs work. See its [documentation](#) for the full range of endpoints. Chapter 6 also points curious readers to the [Open Notify ISS position API](#) as a second free, no-key API to try.
- [scikit-learn](#) — the machine learning library used in Chapter 8.

D Image Credits

Every historical chart and photograph reproduced in this book is sourced from Wikimedia Commons under a Creative Commons or public-domain license. This page collects the source, author, and license for each one, so the chapters themselves can stay focused on the ideas.

Diagrams *without* an entry below — the flowcharts, anatomy diagrams, and pipeline illustrations throughout the book — were built by hand for this book and need no external credit.

Figure	Chapter	Source title	Author	License
Diagram of the Causes of Mortality in the Army in the East (1858)	Chapter 3, Data Visualization Basics	Nightingale-mortality.jpg	Florence Nightingale	Public Domain
Anscombe's Quartet	Chapter 7, Descriptive Statistics & Data Ethics	Anscombe's quartet 3.svg	Avenue, based on data by Francis Anscombe	CC BY-SA 3.0
1854 Broad Street Cholera Map	Chapter 11, Telling Your Data Story	Snow-cholera-map-1.jpg	John Snow	Public Domain